

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Гжельский государственный университет»

(ГГУ)

Колледж ГГУ

Специальность 09.02.07 Информационные системы и программирование
Форма обучения – очная

ДИПЛОМНЫЙ ПРОЕКТ

на тему: **«Разработка информационной системы пищевого
производства»**

Выполнил:

студент группы ИСП-О-22

Попов Андрей Викторович

(подпись)

РАБОТА ДОПУЩЕНА К ЗАЩИТЕ

Заместитель директора по УР

М.В. Казакова

(подпись)

МП

Руководитель:

преподаватель колледжа,

Вальтер И.В.

(подпись)

пос. Электроизлятор

2026 год

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. АНАЛИТИЧЕСКАЯ ЧАСТЬ	6
1.1. Описание и анализ предметной области	6
1.2. Принципы разработки ИС	10
1.3. Характеристика проблемы разработки	12
1.4. Обзор и сравнительный анализ готовых решений	14
2. ПРОЕКТНАЯ ЧАСТЬ	16
2.1. Определение требований к информационной системе	16
2.2. Обзор и выбор средств проектирования и разработки ИС	18
2.3. Методы моделирования системы	22
2.3.1. ER-диаграмма	23
2.3.2. Контекстная диаграмма	28
2.3.3. Диаграмма декомпозиции IDEF0	31
2.3.4. Диаграмма потоков данных	34
2.3.5. Диаграмма вариантов использования (Use Case Diagram)	38
2.4. Макеты интерфейса приложения	43
2.5. Описание логики функционирования ПО	64
3. СПЕЦИАЛЬНАЯ ЧАСТЬ	68
3.1. Формирование требований к ПО	68
3.2. Разработка интерфейса ПО	70
3.3. Проектирование структуры базы данных	82
3.4. Отладка и тестирование программы	94
3.5. Руководство пользователя	98
ЗАКЛЮЧЕНИЕ	101
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	103
ПРИЛОЖЕНИЕ	106

ВВЕДЕНИЕ

Современные условия функционирования предприятий пищевой промышленности требуют внедрения эффективных информационных систем, обеспечивающих автоматизацию управления производственными процессами, оптимизацию логистических потоков, контроль качества продукции и повышение общей эффективности бизнеса. В условиях высокой конкуренции, жёстких требований к качеству и безопасности пищевых продуктов, а также необходимости соблюдения нормативно-правовых актов, использование традиционных методов управления становится всё менее эффективным.

Разработка информационной системы для пищевого производства направлена на решение комплекса задач, связанных с автоматизацией учёта сырья, управления производственными циклами, отслеживания этапов изготовления продукции, анализа эффективности работы сотрудников и формирования отчётности. В рамках дипломного проекта была выбрана тема: "Разработка информационной системы для пищевого производства", что обусловлено актуальностью цифровизации малых и средних предприятий, стремлением к повышению прозрачности и управляемости производственных процессов.

Целью дипломного проекта является разработка программного решения, позволяющего автоматизировать ключевые бизнес-процессы предприятия пищевой промышленности, включая приём заказов, управление складскими запасами, планирование производства, контроль качества и аналитику деятельности.

Для достижения поставленной цели были определены следующие задачи:

- Провести анализ предметной области и существующих решений в сфере управления пищевым производством;
- Сформировать функциональные и нефункциональные требования к системе;

- Спроектировать архитектуру информационной системы, включая диаграммы IDEF0, DFD, Use Case и ER-модель базы данных;
- Разработать пользовательский интерфейс для различных ролей (администратор, менеджер, повар);
- Реализовать базу данных и ключевые модули приложения;
- Провести тестирование и анализ функциональности системы.

Объектом исследования является производственно-хозяйственная деятельность предприятия, осуществляющего выпуск пищевой продукции.

Предметом исследования — информационные процессы, происходящие в рамках управления производством, складом, заказами и персоналом.

Дипломный проект состоит из введения, трёх глав, заключения, списка использованных источников и приложений. Первая глава посвящена анализу предметной области, включая описание бизнес-процессов предприятия, проблем ручного управления, обзор аналогов и формулирование требований к системе. Вторая глава содержит проектную часть: проектирование архитектуры, интерфейса, базы данных, а также описание реализации и тестирования разработанного программного обеспечения. Третья глава содержит описание процесса проектирования информационной системы, её тестирование и документирования.

Практическая значимость проекта заключается в том, что разработанная система может быть внедрена на небольшом или среднем предприятии, занимающемся пищевым производством, с целью повышения прозрачности управления, снижения издержек и повышения конкурентоспособности продукции.

1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1. Описание и анализ предметной области

Предметом исследования данного дипломного проекта является деятельность предприятия пищевого производства, специализирующегося на выпуске готовой продукции для розничных и оптовых клиентов. Пищевое производство представляет собой сложную систему, включающую в себя технологические процессы, логистику, управление персоналом, финансовую отчётность и маркетинг. Эффективность работы такого предприятия напрямую зависит от слаженности взаимодействия всех подразделений и точности учёта всех операций — от закупки сырья до отгрузки готовой продукции.

Современные условия функционирования предприятий пищевой промышленности требуют внедрения информационных систем, позволяющих автоматизировать ключевые бизнес-процессы: планирование производства, учёт сырья и готовой продукции, управление заказами, контроль качества, расчёт себестоимости и формирование финансовой отчётности. Отсутствие единой информационной системы приводит к ряду проблем:

- Дублирование данных между отделами (например, склад и бухгалтерия используют разные источники информации).
- Задержки в обработке заказов из-за ручного ввода и согласования.
- Низкая прозрачность бизнес-процессов — сложно отследить, на каком этапе находится заказ или почему возникла задержка.
- Ошибки при учёте из-за человеческого фактора.
- Отсутствие аналитики — невозможно оценить эффективность работы сотрудников, рентабельность продуктов или загруженность оборудования.

Разработка информационной системы для пищевого производства направлена на решение этих задач за счёт создания единого цифрового пространства, где все данные хранятся в одной базе, доступны в реальном времени и обновляются автоматически при изменении состояния процесса.

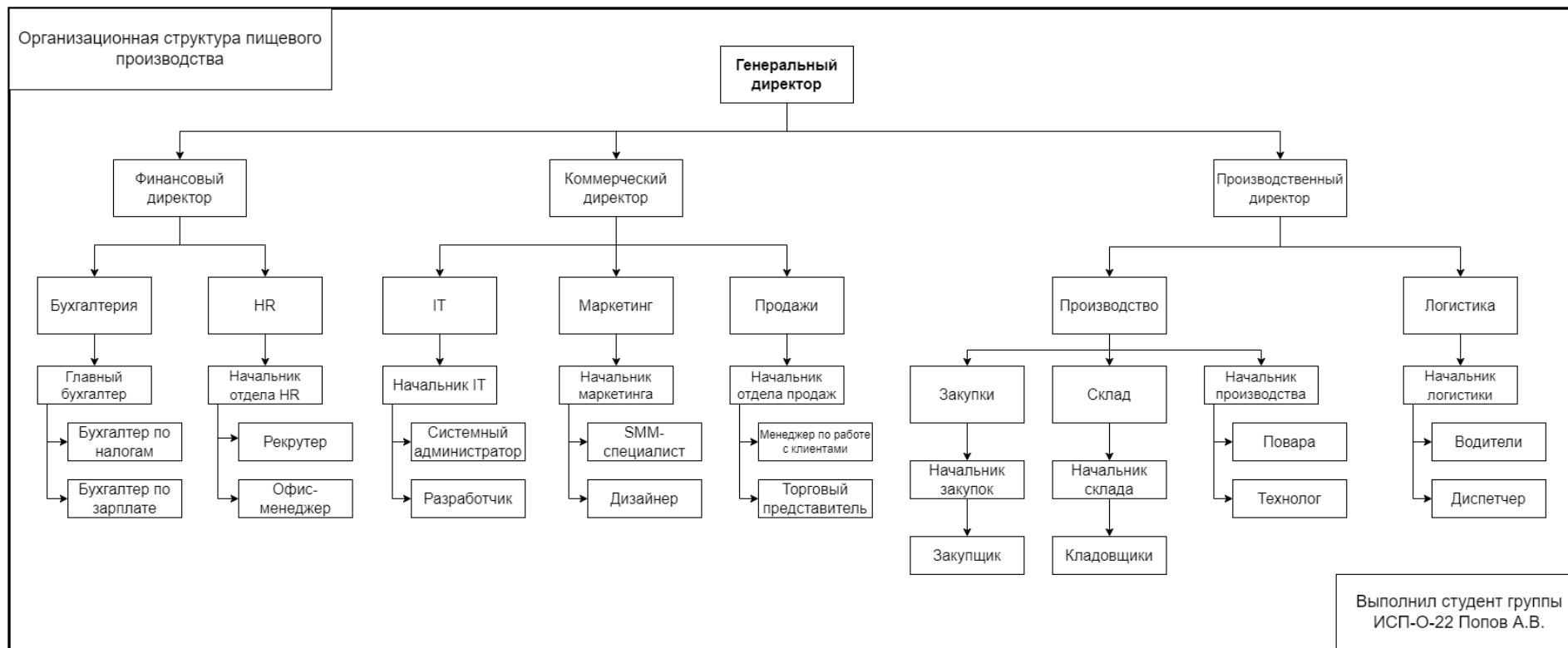


Рисунок 1.1. Организационная структура пищевого производства

На предприятии действует иерархическая организационная структура, в которой каждый сотрудник имеет чётко определённые обязанности и зону ответственности. Структура построена по функциональному принципу: каждый директор отвечает за свой блок деятельности, а внутри блока выделены отделы и должности, обеспечивающие выполнение задач.

Как видно из структуры, управление предприятием осуществляется тремя ключевыми директорами:

Финансовый директор — отвечает за финансовую устойчивость, бухгалтерский учёт, налоги, зарплату и бюджетирование. В его подчинении находятся:

- Бухгалтерия (Главный бухгалтер, бухгалтер по налогам, бухгалтер по зарплате);
- HR-отдел (начальник отдела, рекрутер, офис-менеджер).

Коммерческий директор — управляет продажами, маркетингом и IT-инфраструктурой. Его подразделения:

- Продажи (начальник отдела, менеджеры по работе с клиентами, торговые представители);
- Маркетинг (начальник, SMM-специалист, дизайнер);
- IT-отдел (начальник, системный администратор, разработчик).

Производственный директор — отвечает за выпуск продукции, обеспечение качества и логистику. В его подчинении:

- Производство (начальник, повара, технолог);
- Закупки (начальник, закупщик);
- Склад (начальник, кладовщики);
- Логистика (начальник, водители, диспетчер).

Такая структура обеспечивает чёткое разделение обязанностей и позволяет каждому подразделению сосредоточиться на своих задачах, что повышает общую эффективность предприятия.

Основными бизнес-процессами на предприятии являются:

1) Управление закупками — поиск поставщиков, оформление заказов на сырьё, приём и учёт поступившего товара.

2) Производство — планирование выпуска, составление рецептур, контроль качества, учёт расхода сырья.

3) Управление складом — учёт остатков, формирование заявок на пополнение, отпуск продукции.

4) Управление заказами — приём заказов от клиентов, назначение менеджера и повара, отслеживание статуса, отгрузка.

5) Финансовый учёт — учёт затрат, расчёт себестоимости, начисление зарплат, формирование отчётности.

6) Маркетинг и продажи — продвижение продукции, работа с клиентами, анализ спроса.

7) Управление персоналом — подбор, обучение, расстановка задач, учёт рабочего времени.

Эти процессы тесно связаны между собой. Например, для начала производства необходимо наличие сырья, которое должно быть закуплено и доставлено на склад. Для формирования заказа клиенту нужно знать, какие продукты доступны, а для этого — актуальные остатки на складе. Финансовая отчётность строится на основе данных о продажах, закупках и зарплатах.

Проблемы, с которыми сталкивается предприятие

Несмотря на чёткую организационную структуру, предприятие сталкивается с рядом проблем, связанных с отсутствием единой информационной системы:

- Ручной учёт — данные ведутся в Excel-таблицах, что приводит к ошибкам и несогласованности.

- Отсутствие интеграции — отделы работают изолированно, информация передаётся по почте или вручную.

- Нехватка аналитики — нет возможности быстро получить отчёт о рентабельности продукта, загрузке поваров или оборачиваемости запасов.

- Задержки в принятии решений — руководству приходится ждать данных из разных источников.
- Риск потери данных — отсутствие резервного копирования и централизованного хранения.

Решением этих проблем может стать внедрение информационной системы, которая объединит все бизнес-процессы в единое целое, обеспечит автоматизацию рутинных операций и предоставит инструменты для анализа и управления.

1.2. Принципы разработки ИС

Разработка информационной системы для предприятия, занимающегося пищевым производством, должна опираться на определённые фундаментальные принципы, направленные на достижение максимальной эффективности, надёжности и соответствия специфике бизнес-процессов. Эти принципы служат основой для принятия проектных решений, выбора архитектурных решений, технологий и инструментов, а также определяют подходы к обеспечению качества и удобства использования системы.

Целью реализации информационной системы является повышение прозрачности управления, оптимизация ресурсопотребления, ускорение обработки заказов, обеспечение контроля качества продукции и поддержка аналитики. Для достижения этих целей разрабатываемое программное обеспечение должно обладать рядом ключевых характеристик, основанных на современных подходах к проектированию и разработке ПО.

1) **Функциональность и соответствие требованиям.** Система должна в полной мере отвечать задачам, выявленным в ходе анализа предметной области. Это означает, что все основные бизнес-процессы предприятия — от приёма заказов до учёта расхода сырья и анализа эффективности сотрудников — должны быть автоматизированы и отражены в функционале системы. При этом важно обеспечить гибкость и масштабируемость, чтобы в будущем можно было легко адаптировать систему под изменяющиеся бизнес-требования или расширять её функциональность.

2) Надёжность и устойчивость. Информационная система должна быть устойчивой к сбоям, обеспечивать целостность и сохранность данных, а также корректно обрабатывать ошибки. Это особенно важно для пищевого производства, где сбой в системе может привести к нарушению технологического процесса, потере информации о заказах или остатках на складе. Необходимо предусмотреть резервное копирование и восстановление данных, а также протоколирование критических операций.

3) Безопасность информации. Поскольку система будет содержать конфиденциальные данные (информация о клиентах, поставщиках, сотрудниках, финансах, рецептурах), необходимо обеспечить защиту от несанкционированного доступа. Это достигается путём реализации системы аутентификации и авторизации пользователей, разграничения прав доступа в зависимости от роли (например, администратор, менеджер, повар), а также шифрования чувствительных данных при хранении и передаче.

4) Удобство использования (Usability). Интерфейс системы должен быть интуитивно понятным и удобным для пользователей, обладающих разным уровнем компьютерной грамотности. Это особенно важно для сотрудников производственных и складских подразделений, которые не являются профессиональными IT-специалистами. Простота навигации, ясность надписей, логичная структура окон и форм — всё это способствует снижению времени обучения и повышению общей производительности.

5) Интеграция и совместимость. Система должна быть спроектирована с учётом возможности интеграции с другими программными решениями, используемыми на предприятии (например, бухгалтерские программы, учётные системы, ERP-решения). Это обеспечит согласованность данных и исключит дублирование информации.

6) Поддержка и сопровождение. Разработка должна учитывать долгосрочную перспективу использования. Это подразумевает создание системы, которую будет легко поддерживать, обновлять и развивать. Структура кода, документация, а также архитектура приложения должны быть

ориентированы на лёгкость внесения изменений и устранения неполадок в будущем.

7) Экономическая эффективность. При проектировании и разработке ИС учитывается оптимальное соотношение стоимости и функциональности. Необходимо избегать излишней сложности и перегрузки системой ненужными функциями. Это позволяет снизить затраты на внедрение, обучение и сопровождение, при этом сохранив все преимущества автоматизации.

Таким образом, соблюдение указанных принципов позволяет создать качественную, надёжную и удобную информационную систему, которая будет соответствовать специфике пищевого производства и способствовать повышению эффективности управления предприятием.

1.3. Характеристика проблемы разработки

Разработка информационной системы для предприятия, специализирующегося на пищевом производстве, представляет собой сложную и многогранную задачу. Её успешная реализация требует не только технических знаний и навыков программирования, но и глубокого понимания специфики пищевой отрасли, нормативно-правового регулирования, особенностей бизнес-процессов и потребностей конечных пользователей. Несмотря на очевидную пользу от автоматизации, проект может столкнуться с рядом существенных проблем, которые необходимо учитывать и прогнозировать на всех этапах — от начального анализа до сдачи в эксплуатацию и последующего сопровождения.

1) Проблема сложности и разнообразия бизнес-процессов. Пищевое производство включает в себя множество взаимосвязанных процессов: закупка и приёмка сырья, учёт и хранение на складах, соблюдение технологических карт и рецептур, контроль качества на различных этапах, планирование производства, упаковка, отгрузка готовой продукции и формирование отчётности. Каждый из этих процессов может иметь свои подпроцессы, исключения, зависимости и нормативные требования.

Моделирование и последующая автоматизация такого разнообразия требуют высокой степени детализации и чёткого понимания логики взаимодействия между различными подразделениями. Ошибки в проектировании могут привести к несогласованности данных, задержкам и нарушению технологического цикла.

2) Проблема неоднозначности и изменчивости требований. В начале проекта требования к системе могут быть сформулированы не до конца чётко. Пользователи могут не осознавать, как именно система повлияет на их работу, или не учитывать все нюансы текущих процессов. В процессе разработки могут выявляться новые аспекты или изменяться приоритеты, что влечёт за собой необходимость пересмотра проектных решений. Такая изменчивость усложняет планирование, увеличивает сроки и стоимость проекта, а также повышает риск отклонения от изначальной цели.

3) Проблема интеграции с существующими системами и процессами. Предприятие может уже использовать некоторые программные продукты (например, бухгалтерские программы, системы учёта персонала или отдельные Excel-модули). Новая информационная система должна безопасно и корректно интегрироваться с уже имеющейся инфраструктурой, не нарушая текущую работу. Это требует дополнительного анализа и проектирования механизмов обмена данными, что увеличивает сложность разработки.

4) Проблема обеспечения безопасности и соответствия нормативным требованиям. Пищевая промышленность строго регулируется государственными органами. Система должна обеспечивать контроль за соблюдением санитарных норм, учёт сроков годности, хранение информации о составе продуктов, участие в системах прослеживаемости и т.п. Кроме того, необходимо защищать конфиденциальные данные клиентов, поставщиков, сотрудников и коммерческую информацию. Реализация этих требований усложняет разработку и требует постоянного контроля.

5) Проблема сопротивления изменениям со стороны персонала. Внедрение новой ИС часто сопровождается сопротивлением со стороны

сотрудников, привыкших к старым методам работы. Это может проявляться в отказе использовать систему, снижении продуктивности в период адаптации или негативном отношении к изменениям. Успешное внедрение требует не только технически качественной системы, но и грамотной коммуникации, обучения и поддержки пользователей, что также является отдельной задачей в рамках проекта.

6) Проблема выбора и использования технологий. Выбор технологий (языка программирования, СУБД, фреймворков, архитектурных паттернов) должен учитывать не только текущие требования, но и долгосрочную перспективу. Неправильный выбор может привести к трудностям в сопровождении, масштабировании, интеграции или найме специалистов. Также важно учитывать стоимость владения решением, включая лицензии, оборудование и трудозатраты на поддержку.

7) Проблема тестирования и отладки. Из-за сложности взаимосвязей между модулями системы, большое количество сценариев использования и зависимость от внешних факторов (например, данных от поставщиков или изменений в рецептуре), тестирование становится трудоёмким процессом. Необходимо предусмотреть тестирование не только функциональности, но и производительности, безопасности, корректности обработки ошибок и восстановления после сбоев.

Таким образом, разработка информационной системы для пищевого производства — это не просто задача программирования, а комплексный проект, требующий междисциплинарного подхода. Успешное его завершение возможно лишь при чётком понимании всех потенциальных сложностей и разработке стратегии их преодоления на ранних этапах проекта.

1.4. Обзор и сравнительный анализ готовых решений

Для обоснования целесообразности создания собственной информационной системы предприятия пищевого производства, проведён обзор и анализ существующих программных решений.

1) Универсальные ERP-системы (например, SAP, 1C:ERP, Oracle NetSuite) предлагают широкий функционал для автоматизации всех аспектов бизнеса. Их преимущество — полнота и интеграция данных. Однако высокая стоимость, сложность внедрения и потенциальная избыточность функций делают их не всегда рациональным выбором для небольших или средних предприятий.

2) Специализированные решения (например, WMS для склада, iiko для общепита) глубоко покрывают конкретные сферы. Они проще в внедрении, но редко охватывают все процессы предприятия. Использование нескольких таких систем может привести к фрагментации данных и усложнению управления.

3) Open Source решения (например, Odoo, ERPNext) предоставляют гибкость и низкие лицензионные затраты. Однако требуют значительных усилий на настройку, кастомизацию и сопровождение, а также могут иметь ограниченную поддержку и повышенные риски безопасности.

4) Облачные SaaS-платформы (например, Microsoft Dynamics 365 Business Central, Bitrix24) обеспечивают быстрое развертывание и минимальные начальные затраты. Их недостатками могут быть ограниченная кастомизация, зависимость от провайдера и ежемесячные платежи.

Анализ показал, что универсальные или специализированные готовые решения не всегда полностью соответствуют специфике пищевого производства (контроль сроков годности, рецептуры, нормативы). Следовательно, разработка собственной системы, адаптированной под конкретные бизнес-процессы, является наиболее рациональным подходом.

2. ПРОЕКТНАЯ ЧАСТЬ

2.1. Определение требований к информационной системе

Этап определения требований к информационной системе играет ключевую роль в успешности всего проекта. Он включает в себя анализ потребностей предприятия, выявление проблем, формулирование функций, которые должна выполнять система, и установление характеристик, которым она обязана соответствовать. От точности и полноты сформированных требований зависит, насколько эффективно будет построена архитектура, реализованы модули, разработан интерфейс и в конечном итоге — насколько успешно система решит поставленные задачи автоматизации.

Для предприятия пищевого производства требования к системе формировались на основе анализа предметной области, выявленных проблем в текущих бизнес-процессах и специфики отрасли. Основной целью является создание программного решения, способного объединить все ключевые аспекты деятельности: от приёма заказов клиентов до контроля за расходом сырья и анализа эффективности сотрудников.

Требования к информационной системе можно разделить на функциональные и нефункциональные.

Функциональные требования описывают конкретные действия, которые система должна выполнять. Они отражают основные бизнес-процессы предприятия и определяют, что система делает.

- Управление заказами клиентов: создание, редактирование, отслеживание статуса (новый, в процессе, готово, завершён, отменён), назначение менеджеров и поваров, расчёт стоимости.
- Управление складскими запасами: учёт остатков сырья и готовой продукции, контроль минимальных уровней, отражение операций приёма и списания.
- Управление производством: планирование выпуска, отслеживание этапов, учёт расхода сырья на основе рецептов, контроль качества.

- Управление рецептами и составом продукции: хранение и редактирование состава готовых изделий, связь с сырьем и остатками.
- Управление поставками: регистрация поступлений сырья от поставщиков, учёт количества и качества.
- Управление клиентами и поставщиками: хранение контактной информации, истории заказов и поставок.
- Управление персоналом: учёт сотрудников, их ролей (администратор, менеджер, повар), логинов и прав доступа.
- Формирование отчётов и аналитика: генерация отчётов по продажам, популярным товарам, расходу сырья, эффективности сотрудников (менеджеров, поваров, админов), прибыли.

Нефункциональные требования определяют характеристики системы, не связанные напрямую с её функциями, но критически важные для её успешной эксплуатации. Они описывают, как система должна работать.

- Производительность: система должна быстро отвечать на запросы пользователя (например, открытие списка заказов или товара — не более 2-3 секунд).
- Надёжность: система должна быть устойчивой к сбоям, обеспечивать целостность данных и иметь механизмы резервного копирования.
- Безопасность: реализация аутентификации пользователей, разграничение доступа по ролям, защита конфиденциальной информации.
- Удобство использования: интерфейс должен быть интуитивно понятным, на русском языке, с логичной навигацией, чтобы минимизировать время на обучение сотрудников.
- Совместимость: приложение должно корректно работать в выбранной операционной системе (например, Windows).
- Масштабируемость: архитектура должна предусматривать возможность добавления новых функций и роста объёма данных без значительной переработки системы.

Сформированные требования служат основой для дальнейших этапов — проектирования архитектуры, разработки базы данных, создания интерфейсов и программного кода. Они обеспечивают единое понимание целей и задач проекта между всеми участниками и позволяют избежать отклонений от поставленных целей.

2.2. Обзор и выбор средств проектирования и разработки ИС

Для успешной реализации информационной системы предприятия пищевого производства необходимо обоснованно выбрать программные инструменты, охватывающие все этапы — от проектирования архитектуры и моделирования данных до написания кода, отладки, тестирования и сопровождения. В рамках данного раздела рассматриваются ключевые категории инструментов, анализируются различные варианты, и обосновывается выбор конкретных технологий и программных продуктов, которые будут использованы в проекте.

Язык программирования и среда разработки

Выбор языка программирования и среды разработки (IDE) является одной из фундаментальных задач на этапе планирования проекта. Язык определяет подход к написанию кода, его структуру, особенности взаимодействия с базой данных и операционной системой, а также доступность разработчиков, знакомых с технологией.

Для проекта информационной системы было решено использовать язык C#, разработанный компанией Microsoft. C# — это современный объектно-ориентированный язык, обладающий рядом преимуществ:

- Интеграция с .NET Framework/.NET Core: C# тесно интегрирован с экосистемой Microsoft, что обеспечивает хорошую совместимость с Windows-приложениями и другими продуктами Microsoft.
- Надёжность и безопасность: язык включает встроенную поддержку управления памятью, исключений и строгую типизацию, что способствует созданию стабильного и безопасного кода.

- Производительность: C# компилируется в промежуточный язык (IL), что обеспечивает высокую производительность приложения.

- Поддержка GUI-фреймворков: язык отлично подходит для разработки десктопных приложений, включая Windows Forms и WPF.

В качестве среды разработки выбрана Microsoft Visual Studio Community. Это мощная IDE, предоставляющая широкий набор инструментов:

- Встроенный отладчик.
- Поддержка IntelliSense.
- Интеграция с системами контроля версий (Git).
- Визуальные конструкторы интерфейсов.
- Интеграция с Microsoft SQL Server.

Альтернативы: также можно рассматривать JetBrains (отличная IDE, но коммерческая) или Visual Studio Code (лёгкий редактор, требующий установки расширений для полноценной разработки на C#).

Технология интерфейса (GUI Framework)

Для создания десктопного интерфейса в экосистеме .NET доступны два основных фреймворка:

Windows Forms (WinForms): Простой и проверенный способ создания интерфейсов с привычным внешним видом для Windows. Подходит для приложений с умеренными требованиями к интерактивности и дизайну. Быстрее осваивается, особенно для новичков. Для проекта пищевого производства, где основной упор делается на функциональность, а не на сложный UI, WinForms представляется рациональным выбором.

WPF (Windows Presentation Foundation): Более современная и гибкая технология, позволяющая создавать богатые интерфейсы с анимацией, стилями и привязкой данных. Однако требует больше времени на изучение и разработку.

В рамках данного проекта был выбран Windows Forms, как более простой и быстрый в разработке инструмент, соответствующий целям и задачам системы.

Хранение, обработка и управление данными — одна из ключевых задач информационной системы. Выбор СУБД определяет производительность, надёжность, масштабируемость и безопасность системы.

В качестве СУБД для проекта выбрана Microsoft SQL Server, разработанная компанией Microsoft. Основные причины выбора:

- Высокая производительность и надёжность: SQL Server — ведущее корпоративное решение, способное обрабатывать большие объёмы данных и обеспечивать стабильную работу.
- Интеграция с .NET/C#: отлично интегрируется с приложениями, написанными на C#, благодаря встроенным драйверам (ADO.NET, Entity Framework).
- Безопасность: Встроенная поддержка ролей, разграничения доступа, шифрования данных.
- Развитые средства администрирования и анализа: SQL Server Management Studio (SSMS) предоставляет мощные инструменты для управления БД.
- Версия Express: для небольших проектов подходит бесплатная версия SQL Server Express, что снижает затраты.

Альтернативы: PostgreSQL (мощная open-source СУБД), MySQL (популярна, но может уступать PostgreSQL в сложных задачах), SQLite (лёгковесная файловая СУБД, не подходит для многопользовательских систем, как в нашем случае).

Для визуализации архитектуры, структуры данных, процессов и взаимодействия с системой используются специализированные инструменты моделирования.

Для диаграмм и схем (IDEF0, DFD, ER-модель, Use Case):

- draw.io (diagrams.net): Бесплатный инструмент с поддержкой различных нотаций, включая IDEF0, DFD, UML, ERD. Удобен для совместной работы и интеграции в документацию.

- Альтернативы: Lucidchart (веб-инструмент с совместной работой), StarUML, Enterprise Architect (мощные CASE-средства, часто коммерческие).

Система контроля версий

Система контроля версий необходима для отслеживания изменений в коде, совместной работы нескольких разработчиков и возможности отката к предыдущим версиям.

- Git: Стандарт де-факто в современной разработке. Распределённая система, мощные возможности слияния изменений, интеграция с IDE и платформами хостинга кода (GitHub, GitLab).

- Интеграция: Git встроен в Visual Studio и поддерживается через командную строку и графические интерфейсы (например, встроенный в VS или внешние, как TortoiseGit).

На основе проведённого анализа были выбраны следующие средства для разработки информационной системы:

Язык программирования: C# — выбран как основной язык разработки благодаря своей производительности, надёжности, интеграции с .NET и экосистеме Microsoft, что соответствует целевой платформе Windows и потенциальной инфраструктуре предприятия.

Среда разработки: Microsoft Visual Studio Community — выбрана как наиболее полная и интегрированная среда разработки для C# и .NET.

Технология интерфейса: Windows Forms — выбрана для быстрой и удобной разработки десктопного интерфейса с понятной логикой взаимодействия.

СУБД: Microsoft SQL Server — выбрана как надёжная, безопасная и хорошо интегрируемая СУБД.

Инструменты моделирования: draw.io — для создания диаграмм IDEF0, DFD, ER-модели и Use Case.

Система контроля версий: Git — для управления исходным кодом.

Выбранные инструменты позволяют реализовать все поставленные функциональные и нефункциональные требования к системе и создать эффективное, удобное в сопровождении программное обеспечение для предприятия пищевого производства.

2.3. Методы моделирования системы

Моделирование системы — это важный этап проектирования, заключающийся в создании упрощённого, но формализованного представления реальной системы (в данном случае — информационной системы предприятия пищевого производства). Цель моделирования — анализ, структуризация и документирование ключевых аспектов будущей системы: её структуры данных, функций, бизнес-процессов и взаимодействия с пользователями и внешней средой. Использование различных методов и нотаций моделирования позволяет наглядно и точно описать систему, избежать недопонимания между разработчиками и заказчиком, а также заложить прочный фундамент для последующих этапов — проектирования базы данных, разработки интерфейса и программной реализации.

В рамках дипломного проекта применяются различные методы моделирования, каждый из которых отражает определённый аспект системы. Для описания структуры данных используется ER-диаграмма (сущность-связь), которая визуализирует сущности (таблицы), их атрибуты и связи, и служит основой для проектирования базы данных. Контекстная диаграмма и диаграмма декомпозиции IDEF0 отражают функциональные аспекты системы, показывая, какие процессы выполняются, какие данные в них поступают и выводятся, а также как система взаимодействует с внешними субъектами. Диаграмма потоков данных (DFD) помогает понять, как информация перемещается внутри системы между процессами и хранилищами. Наконец, диаграмма вариантов использования (Use Case) описывает взаимодействие между пользователями (акторами) и системой, выделяя основные функции, доступные каждой роли.

2.3.1. ER-диаграмма

ER-диаграмма (от англ. Entity-Relationship Diagram, диаграмма "Сущность-Связь") — это графическая модель, используемая для описания структуры данных информационной системы. Она отображает сущности (объекты реального мира, например, "Продукт", "Клиент", "Заказ"), их атрибуты (свойства, такие как "Название продукта", "Имя клиента", "Дата заказа") и связи между этими сущностями (например, "Клиент сделал Заказ", "Заказ включает Продукт").

Назначение ER-диаграммы заключается в формализации структуры будущей базы данных, наглядном представлении сущностей, их свойств и взаимосвязей, обеспечении понимания структуры данных как для разработчиков, так и для заказчиков, а также в службе основной для проектирования реляционной базы данных (например, в Microsoft SQL Server).

ER-диаграмма состоит из следующих элементов:

- Сущности (Entities) — объекты, о которых система хранит информацию. На диаграмме обычно изображаются в виде прямоугольников.
- Атрибуты (Attributes) — свойства сущностей. Изображаются в виде овалов, соединённых линией с соответствующей сущностью.
- Связи (Relationships) — отношения между сущностями. Изображаются в виде ромбов, соединённых линиями с сущностями.
- Кардинальность (Cardinality) — показывает, сколько экземпляров одной сущности может быть связано с экземпляром другой.

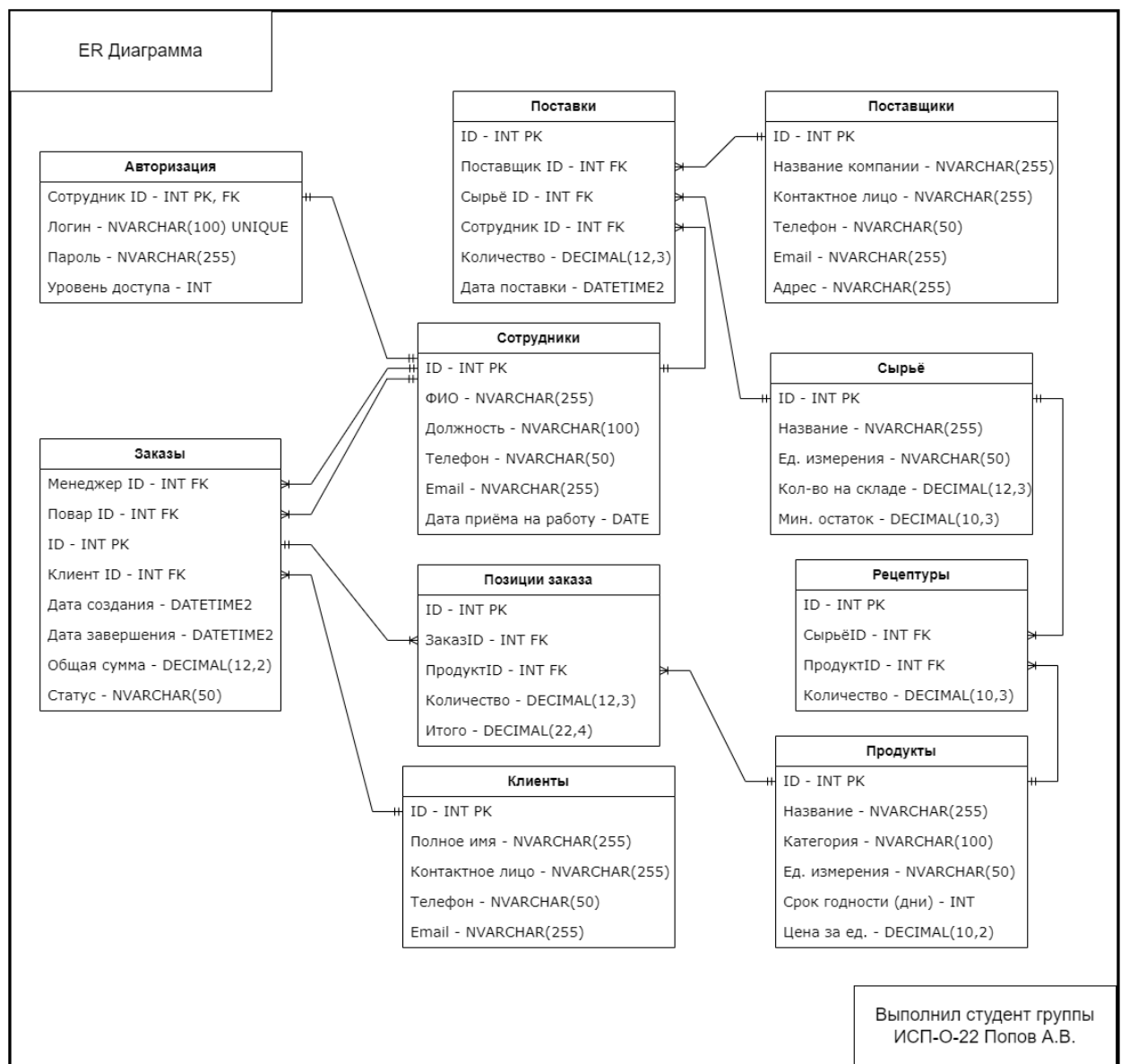


Рисунок 2.1. ER Диаграмма

Представленная ER-диаграмма (Entity-Relationship Diagram) представляет собой концептуальную модель данных информационной системы предприятия пищевого производства. Она визуально отображает основные сущности, их атрибуты и логические связи между ними, формируя фундамент для проектирования реляционной базы данных.

Диаграмма построена в соответствии с нотацией Питера Чена, где:

- Сущности (объекты реального мира) изображены в виде прямоугольников.
- Атрибуты (характеристики сущностей) перечислены внутри прямоугольников.

- Связи между сущностями показаны линиями, на которых указаны кардинальности (множественность связей): «один» (одна вертикальная черта) и «многие» (три черты, напоминающие "H").

Основные сущности и их атрибуты:

1) Поставщики — содержит информацию о компаниях-поставщиках сырья. Атрибуты: ID (первичный ключ, уникальный идентификатор), Название компании (NVARCHAR(255), обязательное поле), Контактное лицо (NVARCHAR(255)), Телефон (NVARCHAR(50)), Email (NVARCHAR(255)), Адрес (NVARCHAR(255)).

2) Сырьё — хранит данные о компонентах, используемых в производстве. Атрибуты: ID (первичный ключ), Название (NVARCHAR(255), обязательное), Ед. измерения (NVARCHAR(50)), Кол-во на складе (DECIMAL(12,3)), Мин. остаток (DECIMAL(10,3)).

3) Продукты — описывает готовую продукцию, выпускаемую предприятием. Атрибуты: ID (первичный ключ), Название (NVARCHAR(255), обязательное), Категория (NVARCHAR(100)), Ед. измерения (NVARCHAR(50)), Срок годности (INT, количество дней), Цена за ед. (DECIMAL(10,2), обязательное, ≥ 0).

4) Рецептуры — определяет состав продуктов из сырья. Атрибуты: ID (первичный ключ), ПродуктID (внешний ключ, ссылка на таблицу "Продукты"), СырьёID (внешний ключ, ссылка на таблицу "Сырьё"), Количество (DECIMAL(10,3), обязательное, ≥ 0).

5) Сотрудники — содержит информацию о персонале предприятия. Атрибуты: ID (первичный ключ), ФИО (NVARCHAR(255), обязательное), Должность (NVARCHAR(100)), Телефон (NVARCHAR(50)), Email (NVARCHAR(255)), Дата приёма на работу (DATE, обязательное).

6) Авторизация — хранит учётные данные сотрудников для входа в систему. Атрибуты: Сотрудник ID (первичный ключ, одновременно внешний ключ, связь с "Сотрудники"), Логин (NVARCHAR(100), уникальный, обязательный), Пароль (NVARCHAR(255), обязательный), Уровень доступа

(INT, обязательный, значения: -1 — заблокирован, 0 — админ, 1 — менеджер, 2 — повар).

7) Клиенты — содержит данные о заказчиках продукции. Атрибуты: ID (первичный ключ), Полное имя (NVARCHAR(255), обязательное), Контактное лицо (NVARCHAR(255)), Телефон (NVARCHAR(30)), Email (NVARCHAR(255)).

8) Заказы — основная сущность, фиксирующая заказы клиентов. Атрибуты: ID (первичный ключ), Клиент ID (внешний ключ, ссылка на "Клиенты"), Менеджер ID (внешний ключ, ссылка на "Сотрудники"), Повар ID (внешний ключ, ссылка на "Сотрудники"), Дата создания (DATETIME2, обязательная), Дата завершения (DATETIME2, может быть NULL), Общая сумма (DECIMAL(12,2), обязательная, ≥ 0), Статус (NVARCHAR(50), обязательный, значения: "Новый", "В процессе", "Готово", "Завершён", "Отменён").

9) Позиции заказа — детализирует состав каждого заказа. Атрибуты: ID (первичный ключ), ЗаказID (внешний ключ, ссылка на "Заказы"), ПродуктID (внешний ключ, ссылка на "Продукты"), Количество (DECIMAL(12,3), обязательное, ≥ 0), Итого (DECIMAL(22,4), вычисляемое поле, = Количество * Цена продукта на момент заказа).

10) Поставки — фиксирует факт поступления сырья от поставщиков. Атрибуты: ID (первичный ключ), Поставщик ID (внешний ключ, ссылка на "Поставщики"), Сырьё ID (внешний ключ, ссылка на "Сырьё"), Сотрудник ID (внешний ключ, ссылка на "Сотрудники"), Количество (DECIMAL(12,3), обязательное, ≥ 0), Дата поставки (DATETIME2, обязательная).

Ключевые связи между сущностями:

- Поставщики — Поставки: Связь «один-ко-многим». Один поставщик может выполнять множество поставок. Реализована через внешний ключ ПоставщикID в таблице Поставки.

- Сырьё — Поставки: Связь «один-ко-многим». Одно сырьё может поступать от разных поставщиков. Реализована через внешний ключ СырьёID в таблице Поставки.
- Сырьё — Рецептуры: Связь «один-ко-многим». Одно сырьё может использоваться в рецептуре нескольких продуктов. Реализована через внешний ключ СырьёID в таблице Рецептуры.
- Продукты — Рецептуры: Связь «один-ко-многим». Один продукт состоит из нескольких ингредиентов (сырья). Реализована через внешний ключ ПродуктID в таблице Рецептуры.
- Сотрудники — Авторизация: Связь «один-к-одному». Каждый сотрудник имеет одну учётную запись. Реализована через первичный ключ СотрудникID, который одновременно является внешним ключом и ссылается на ID в таблице Сотрудники.
- Сотрудники — Заказы: Связь «один-ко-многим». Один сотрудник (менеджер или повар) может участвовать в множестве заказов. Реализована через внешние ключи МенеджерID и ПоварID в таблице Заказы.
- Клиенты — Заказы: Связь «один-ко-многим». Один клиент может сделать несколько заказов. Реализована через внешний ключ КлиентID в таблице Заказы.
- Заказы — Позиции заказа: Связь «один-ко-многим». Один заказ состоит из нескольких позиций. Реализована через внешний ключ ЗаказID в таблице Позиции заказа.
- Продукты — Позиции заказа: Связь «один-ко-многим». Один продукт может быть включён в множество заказов. Реализована через внешний ключ ПродуктID в таблице Позиции заказа.
- Сотрудники — Поставки: Связь «один-ко-многим». Один сотрудник может оформлять множество поставок. Реализована через внешний ключ СотрудникID в таблице Поставки.

Эта ER-диаграмма служит основой для создания физической модели базы данных и определяет структуру таблиц, их поля и внешние ключи, гарантируя целостность и корректность хранения информации в системе.

2.3.2. Контекстная диаграмма

Контекстная диаграмма — это высокоуровневое представление предприятия, отражающее его взаимодействие с внешней средой. Она показывает, какие входы поступают в предприятие, какие выходы оно формирует, какими управляющими воздействиями руководствуется и какие ресурсы (механизмы) использует для выполнения своей основной функции. Контекстная диаграмма обычно создаётся в нотации IDEF0 и обозначается как диаграмма А-0.

Центральный блок диаграммы обозначает основную функцию предприятия (например, "Производство и реализация пищевой продукции"). Входы — это ресурсы, данные или материалы, поступающие извне (сырьё, заказы, нормативные документы). Выходы — результаты деятельности (готовая продукция, отчёты, доход). Управление — нормативы, правила, стандарты, законодательные акты. Механизмы — ресурсы, с помощью которых предприятие осуществляет свою функцию (персонал, оборудование, IT-системы, склады).

Диаграмма позволяет определить границы предприятия, проанализировать взаимодействие с внешними субъектами и формализовать общее понимание его деятельности. Также она служит основой для дальнейшей декомпозиции процессов и построения более детализированных моделей.

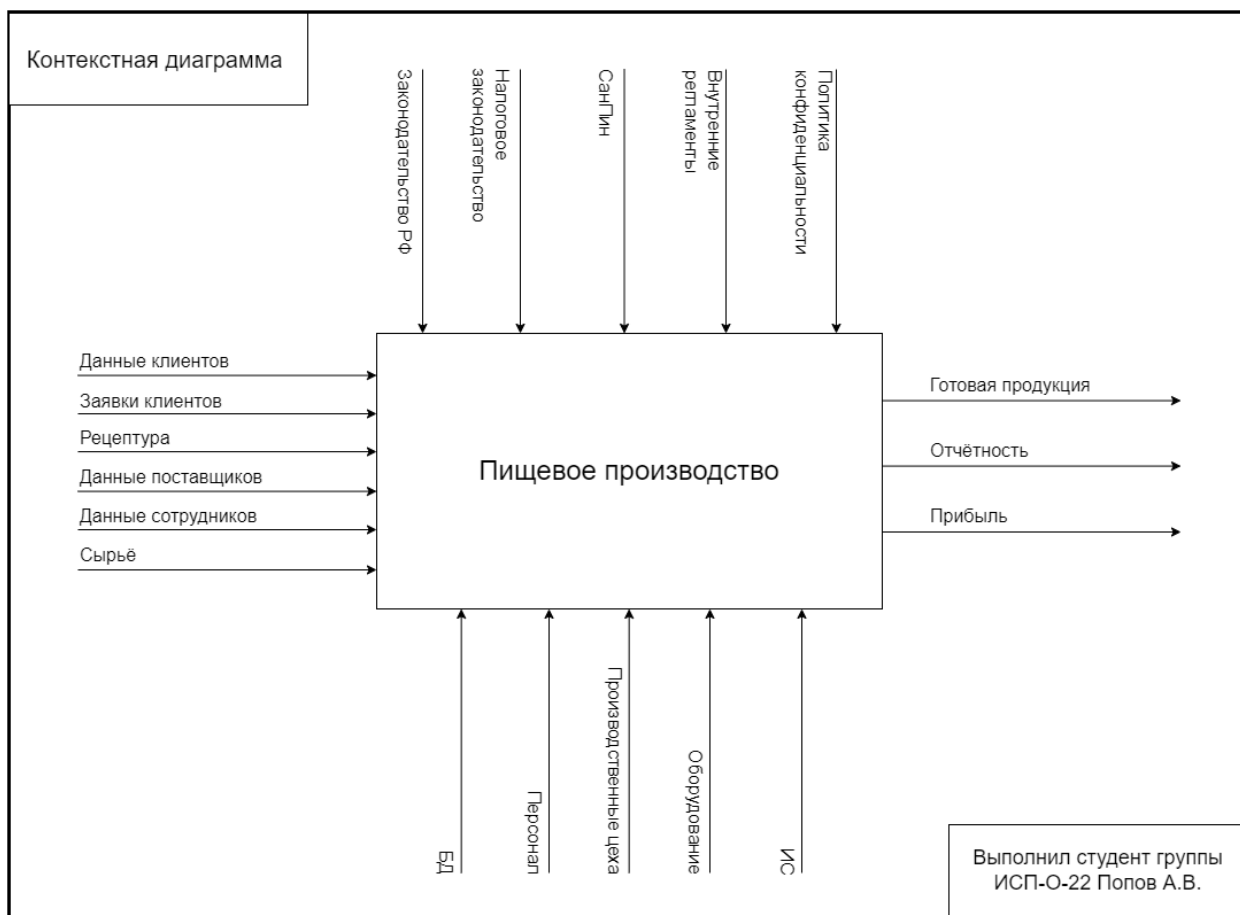


Рисунок 2.2. Контекстная диаграмма

Контекстная диаграмма предприятия «Пищевое производство», представленная на рисунке 2.2, отражает его взаимодействие с внешней средой в нотации IDEF0 и соответствует уровню А-0. Диаграмма наглядно демонстрирует, какие ресурсы поступают в предприятие, какими нормативами оно руководствуется, какие механизмы использует для выполнения своей основной функции и какие результаты выдает во внешнюю среду.

Центральный блок диаграммы — «Пищевое производство» — обозначает основную функцию предприятия: выпуск и реализацию пищевой продукции.

Входы (Inputs) — ресурсы, необходимые для функционирования:

- Сырьё — базовый материал, используемый для производства готовой продукции.
- Данные клиентов — информация о клиентах, необходимая для оформления заказов и взаимодействия.

- Заявки клиентов — запросы на производство продукции.
- Рецептура — технологические документы, определяющие состав и способ приготовления продукции.
- Данные поставщиков — информация о поставщиках, используемая для организации закупок.
- Данные сотрудников — персональные и должностные данные, необходимые для управления персоналом.

Управление (Controls) — правила и ограничения, которым подчиняется предприятие:

- Законодательство РФ — нормативно-правовая база, регулирующая деятельность предприятия.
- Налоговое законодательство — требования к ведению учёта и отчислению налогов.
- СанПиН — санитарно-эпидемиологические правила и нормы, обязательные для соблюдения в пищевой промышленности.
- Внутренние регламенты — локальные правила и стандарты, установленные на предприятии.
- Политика конфиденциальности — требования к защите персональных данных и коммерческой тайны.

Механизмы (Mechanisms) — ресурсы, обеспечивающие выполнение функции:

- Персонал — люди, выполняющие работу.
- Производственные цеха — физическое пространство и технологические линии.
- Оборудование — машины и инструменты.
- Разрабатываемая ИС — информационная система, поддерживающая управление процессами.
- БД — база данных, обеспечивающая хранение и обработку информации.

Выходы (Outputs) — результаты деятельности предприятия, поступающие во внешнюю среду:

- Готовая продукция — основной результат.
- Отчётность — финансовая, налоговая, управленческая отчётность.
- Прибыль — экономический результат деятельности.

Таким образом, контекстная диаграмма формирует целостное представление о предприятии, определяя его границы, ключевые потоки информации и ресурсов, а также роль нормативных и технических факторов в его функционировании. Она служит отправной точкой для дальнейшего детального моделирования бизнес-процессов и разработки информационной системы, способной эффективно поддерживать все аспекты деятельности предприятия.

2.3.3. Диаграмма декомпозиции IDEF0

Диаграмма декомпозиции IDEF0 — это графическое представление, которое используется для детализации и разбиения сложной системы на более мелкие подпроцессы или функции. В отличие от контекстной диаграммы (A-0), которая показывает систему как единое целое, диаграмма декомпозиции разбивает основную функцию на составляющие части, каждая из которых может быть проанализирована отдельно. Это позволяет глубже понять структуру и логику работы системы, а также выявить взаимосвязи между различными подпроцессами.

В нотации IDEF0 диаграммы декомпозиции обозначаются как A-1, A-2, A-3 и т.д., где цифра после "A" указывает на уровень детализации. Каждый подпроцесс на диаграмме декомпозиции также описывается с помощью четырёх компонентов: входов (Inputs), выходов (Outputs), управления (Controls) и механизмов (Mechanisms), что позволяет сохранить единообразие и согласованность на всех уровнях моделирования.

Таким образом, диаграмма декомпозиции IDEF0 позволяет постепенно углубляться в детали функционирования системы, обеспечивая понимание и структурирование её внутренних процессов.

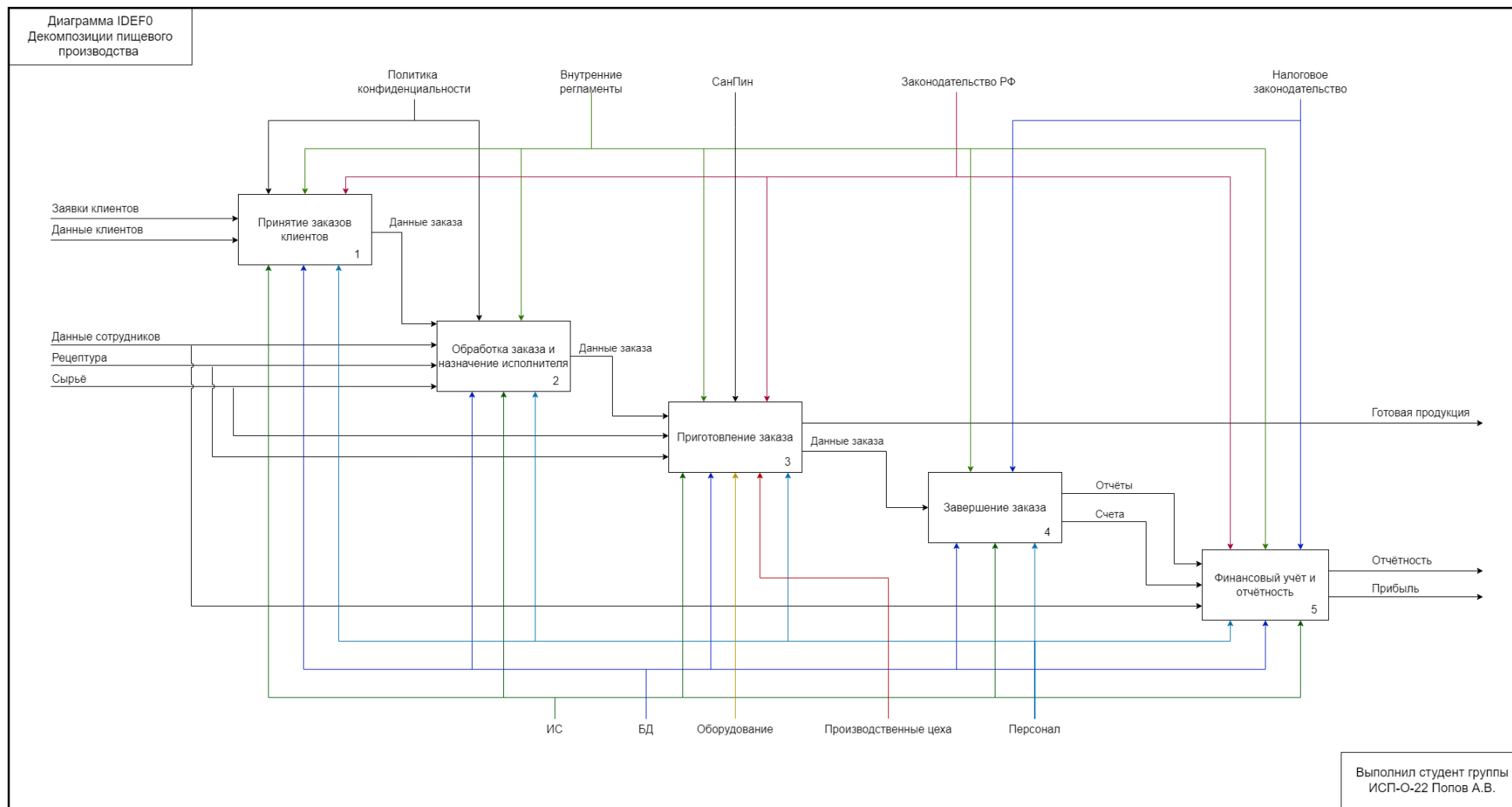


Рисунок 2.3. Диаграмма декомпозиции IDEF0

Диаграмма IDEF0 декомпозиции пищевого производства, представленная на рисунке 2.3, представляет собой детализированную модель основных бизнес-процессов предприятия и является следующим уровнем разбиения после контекстной диаграммы (А-0). Она раскрывает внутреннюю структуру деятельности предприятия, разбивая его основную функцию — «Пищевое производство» — на пять ключевых подпроцессов, каждый из которых описан с помощью стандартных компонентов IDEF0: входов, выходов, управления и механизмов.

Описание процессов:

1) Принятие заказов клиентов

Процесс включает получение и регистрацию заявок от клиентов.

- Входы: Заявки клиентов, Данные клиентов.
- Управление: Политика конфиденциальности, Внутренние регламенты, Законодательство РФ.
- Механизмы: ИС (информационная система), БД (база данных), персонал.
- Выходы: Данные заказа — передаются на следующий этап.

2) Обработка заказа и назначение исполнителя

На этом этапе происходит анализ заказа, проверка наличия сырья и назначение ответственного сотрудника.

- Входы: Данные заказа, Данные сотрудников, Рецепт, Сырьё.
- Управление: Внутренние регламенты, Политика конфиденциальности.
- Механизмы: ИС, БД, Персонал.
- Выходы: Данные заказа — передаются на этап приготовления.

3) Приготовление заказа

Основной производственный процесс, включающий подготовку, изготовление и упаковку продукции.

- Входы: Данные заказа, Рецепт, Сырьё.
- Управление: СанПиН, Внутренние регламенты, Законодательство РФ.

- Механизмы: Оборудование, Производственные цеха, Персонал, БД, ИС.

- Выходы: Готовая продукция, Данные заказа.

4) Завершение заказа

Процесс фиксирует факт выполнения заказа, формирует счёт и готовит документы для отгрузки.

- Входы: Данные заказа.

- Управление: Внутренние регламенты, Налоговое законодательство.

- Механизмы: ИС, БД, Персонал.

- Выходы: Отчёты, Счета — направляются в финансовый учёт.

5) Финансовый учёт и отчётность

Завершающий процесс, в котором формируются финансовые документы, рассчитывается прибыль и составляется отчётность.

- Входы: Отчёты, Счета, Данные сотрудников.

- Управление: Налоговое законодательство, Законодательство РФ, Внутренние регламенты.

- Механизмы: ИС, БД, Персонал.

- Выходы: Отчётность, Прибыль — передаются во внешнюю среду.

Таким образом, диаграмма декомпозиции IDEF0 позволяет наглядно представить последовательность и взаимосвязь бизнес-процессов предприятия, выявить точки контроля и ответственности, а также определить, какие ресурсы и нормативные требования задействованы на каждом этапе. Это служит основой для дальнейшего проектирования информационной системы, которая будет автоматизировать эти процессы.

2.3.4. Диаграмма потоков данных

Диаграмма потоков данных (DFD, Data Flow Diagram) — это графическая модель, используемая для описания движения и обработки информации в системе. Она показывает, как данные перемещаются между различными компонентами системы: внешними субъектами (акторами), процессами (функциями), хранилищами данных (базами данных, файлами) и

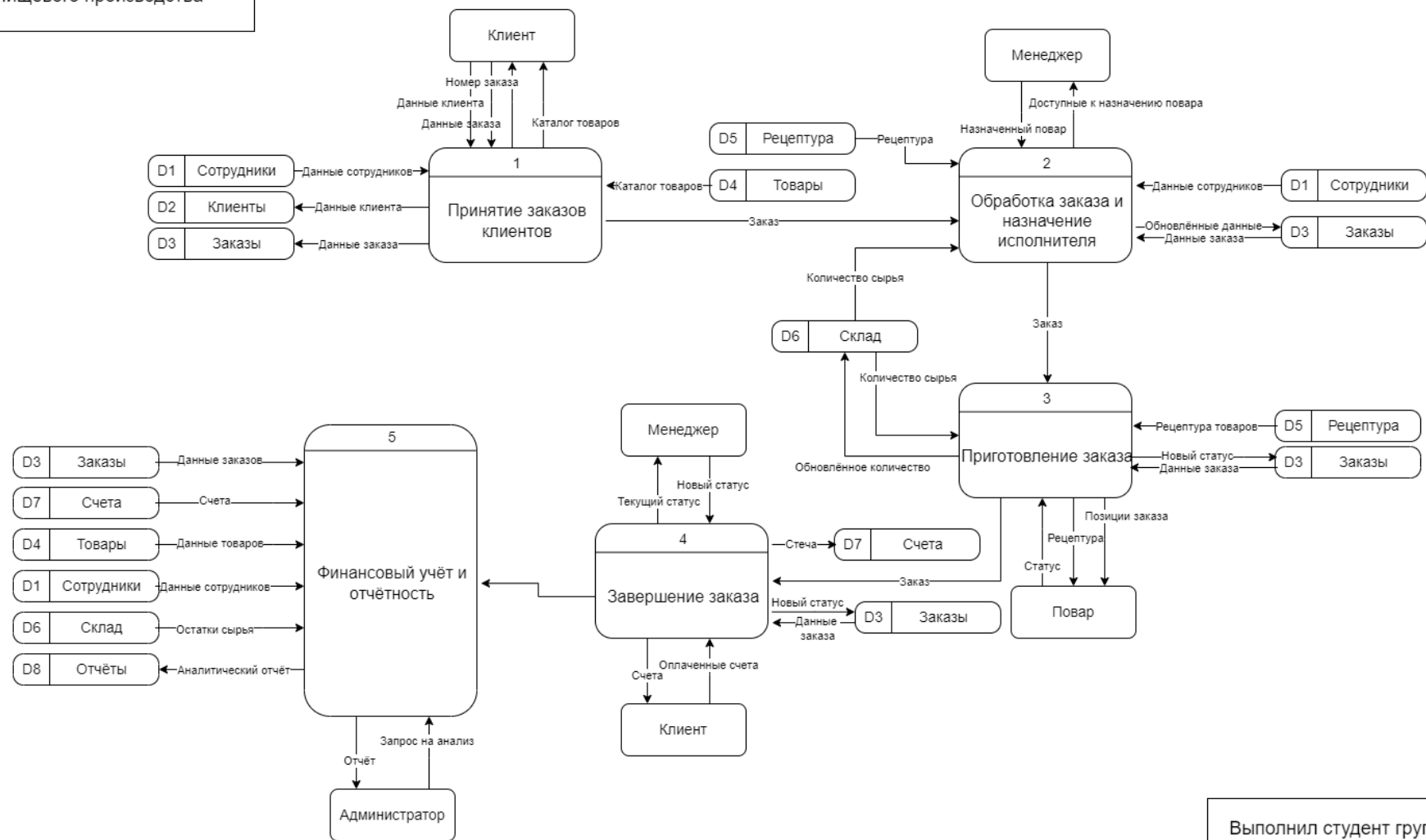
друг с другом. DFD позволяет наглядно представить информационные потоки, источники и приёмники данных, а также операции, которые выполняются над ними.

В диаграммах потоков данных используются стандартные обозначения:

- Акторы — внешние субъекты, взаимодействующие с системой (например, пользователи, поставщики, клиенты).
- Процессы — действия, которые обрабатывают данные (например, «Обработать заказ», «Рассчитать прибыль»).
- Хранилища данных — места, где данные хранятся (например, базы данных, файлы).
- Потоки данных — стрелки, показывающие направление передачи информации между компонентами системы.

DFD применяется на этапах проектирования информационных систем для анализа и документирования бизнес-процессов, определения границ системы, идентификации данных, необходимых для выполнения функций, и проверки логики обработки. Диаграмма может быть построена на нескольких уровнях детализации: от общего обзора системы до детального описания отдельных подпроцессов.

Диаграмма потоков данных DFD
Пищевого производства



Выполнил студент группы
ИСП-О-22 Попов А.В.

Рисунок 2.4. Диаграмма потоков данных

Диаграмма потоков данных (DFD) пищевого производства, представленная на рисунке 2.4, является логической моделью, визуализирующей движение информации между ключевыми процессами, хранилищами данных и внешними субъектами предприятия. В нотации DFD она показывает, как данные поступают в систему, обрабатываются, хранятся и передаются дальше — от клиента до финансового учёта, а также как система взаимодействует с менеджерами, поварами и администратором.

Основные компоненты диаграммы:

Актеры (внешние субъекты):

- Клиент — инициирует заказ, получает номер заказа и копию счёта.
- Менеджер — участвует в обработке и завершении заказа, получает доступ к назначению повара и текущему статусу.
- Повар — выполняет приготовление, отправляет статус и позиции заказа.
- Администратор — запрашивает аналитические отчёты и получает финальную отчётность.

Процессы (функции системы):

1) Принятие заказов клиентов (1) Получает от клиента данные о заказе и клиента, формирует номер заказа и сохраняет данные в хранилище Заказы (D3). Также выдаёт клиенту каталог товаров и номер заказа.

2) Обработка заказа и назначение исполнителя (2) На основе данных заказа и рецептуры определяет необходимое сырьё, проверяет его наличие на складе (через хранилище D6), назначает повара и обновляет статус заказа в Заказы (D3).

3) Приготовление заказа (3) Повар использует рецептуру (D5) и данные заказа (D3), списывает сырьё со склада (обновляет D6), формирует позиции заказа и передаёт информацию о новом статусе.

4) Завершение заказа (4) Фиксирует факт выполнения, формирует счёт (D7), обновляет статус заказа и передаёт данные клиенту.

5) Финансовый учёт и отчётность (5) Собирает данные из всех хранилищ: заказы (D3), счета (D7), товары (D4), сотрудники (D1), склад (D6), генерирует аналитические отчёты (D8) и предоставляет их администратору по запросу.

Хранилища данных (D1–D8):

- D1 Сотрудники — содержит данные о персонале.
- D2 Клиенты — хранит информацию о клиентах.
- D3 Заказы — основное хранилище заказов с их статусами.
- D4 Товары — каталог продукции и её характеристики.
- D5 Рецепттура — состав продуктов и нормы расхода сырья.
- D6 Склад — остатки сырья и логистические данные.
- D7 Счета — финансовые документы, связанные с заказами.
- D8 Отчёты — аналитическая информация для управления.

Таким образом, DFD-диаграмма позволяет наглядно представить логику обработки данных, взаимодействие между участниками и системой, а также структуру хранения информации. Она служит основой для проектирования базы данных и разработки программной логики информационной системы, обеспечивая целостность и согласованность потоков данных на всех этапах работы предприятия.

2.3.5. Диаграмма вариантов использования (Use Case Diagram)

Use Case диаграмма (диаграмма вариантов использования) — это графическое представление, используемое в объектно-ориентированном анализе и проектировании, для описания функций системы с точки зрения пользователей. Она показывает, какие действия (варианты использования) может выполнять система и кто (акторы) взаимодействует с этими функциями. Диаграмма отражает внешние интерфейсы системы, не вдаваясь в детали реализации, и помогает понять, что делает система, а не как она это делает.

Основные элементы Use Case диаграммы:

- Акторы — внешние субъекты (люди, устройства, другие системы), взаимодействующие с системой. Например: клиент, менеджер, повар, администратор.

- Вариант использования (Use Case) — действие или функция, которую система может выполнить по запросу актора. Обозначается овалом с кратким описанием, например: «Создать заказ», «Назначить повара», «Сформировать отчёт».

- Связи — линии, соединяющие акторов с вариантами использования, показывающие, какие функции доступны каждому актору.

Назначение Use Case диаграммы:

- Формализовать требования к системе с точки зрения пользователей.
- Определить функции, доступные каждому типу пользователя.
- Служить основой для проектирования интерфейса и разработки программной логики.

- Обеспечить понимание системы как набора сервисов, предоставляемых различным акторам.

Таким образом, Use Case диаграмма позволяет наглядно представить взаимодействие между пользователями и системой, а также разграничить права доступа и функциональные возможности для каждой роли.

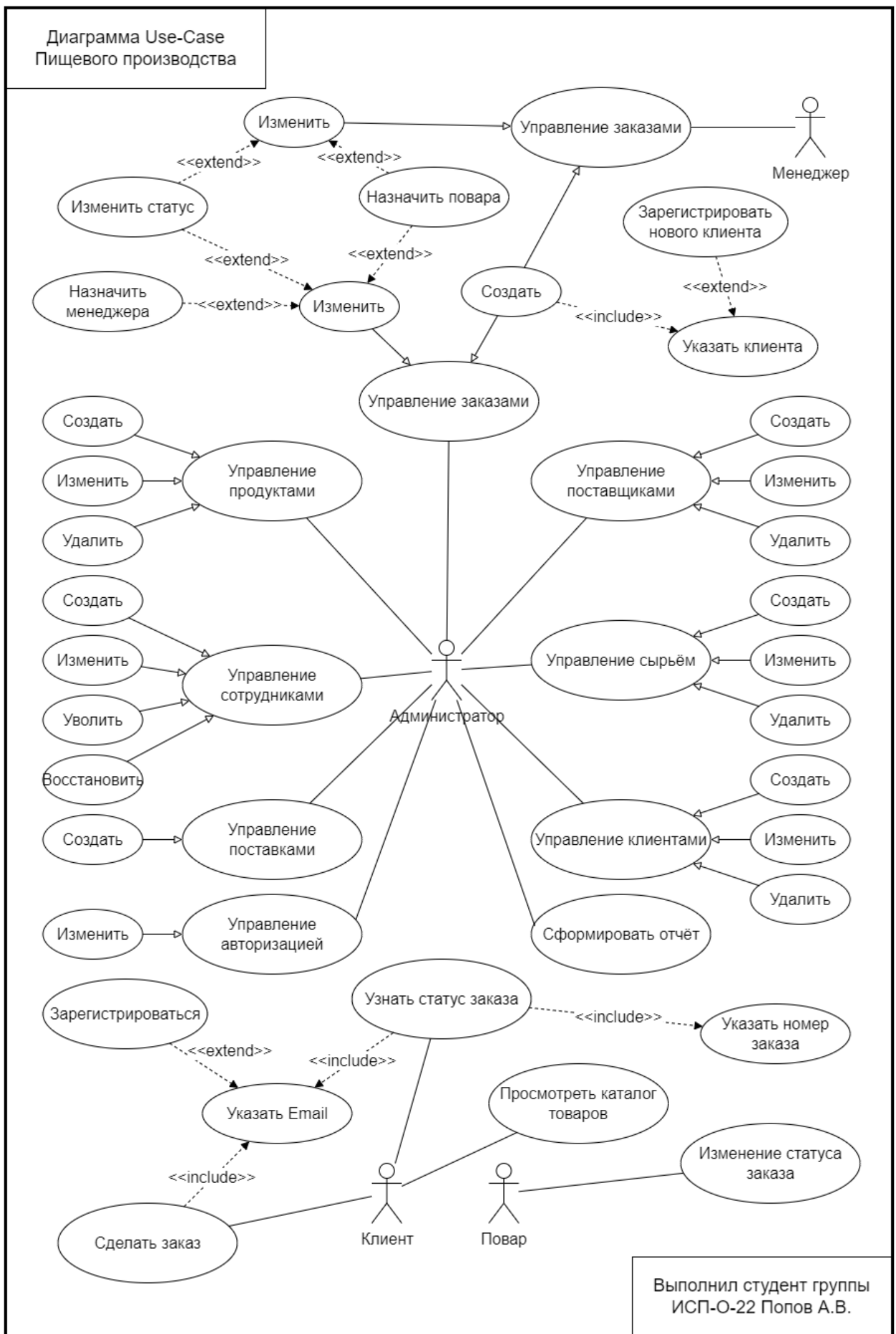


Рисунок 2.5. Диаграмма вариантов использования

Диаграмма вариантов использования (Use Case) для предприятия пищевого производства, представленная на рисунке 2.5, является ключевым инструментом моделирования функциональных требований к информационной системе. Она описывает взаимодействие между пользователями (акторами) и системой, отражая какие функции система должна предоставлять каждому типу пользователя. Диаграмма выполнена в нотации UML (Unified Modeling Language) и позволяет сформировать единое понимание того, что должна уметь делать система с точки зрения её конечных пользователей.

Актеры (Actors). На диаграмме выделены четыре основные роли пользователей, взаимодействующих с системой:

- Администратор — центральная фигура, обладающая полным доступом ко всем функциям системы. Отвечает за управление всеми аспектами деятельности: заказами, продуктами, сотрудниками, поставщиками, сырьём, клиентами и авторизацией.
- Менеджер — пользователь, ответственный за работу с заказами: их создание, изменение, назначение повара и управление статусами.
- Клиент — внешний пользователь, который может зарегистрироваться, сделать заказ и проверить его статус.
- Повар — пользователь, чья основная задача — просматривать каталог товаров и изменять статус заказа на «Готово».

Варианты использования (Use Cases). Каждый эллипс на диаграмме представляет собой конкретную функцию системы, которую может выполнить актер.

Основные группы функций:

1) Управление заказами (для Администратора и Менеджера):

- Создать, Изменить — базовые операции над заказами.
- Назначить повара, назначить менеджера, изменить статус — подоперации Изменения заказа.

2) Управление продуктами, сотрудниками, поставщиками, сырьём и клиентами (для Администратора):

- Управление продуктами: Создать, Изменить, Удалить.
- Управление сотрудниками: Создать, Изменить, Уволить, Восстановить.

- Управление поставками: Создать.
- Управление поставщиками: Создать, Изменить, Удалить.
- Управление сырьём: Создать, Изменить, Удалить.
- Управление клиентами: Создать, Изменить, Удалить.
- Управление авторизацией: Изменить.

3) Функции для Клиента:

- Зарегистрироваться — создание учётной записи.
- Сделать заказ — основная функция клиента.
- Узнать статус заказа — проверка текущего состояния заказа.
- Просмотреть каталог товаров — ознакомление с ассортиментом.

4) Функции для Повара:

- Изменение статуса заказа — переход заказа в статус «Готово».

5) Общие функции:

- Сформировать отчёт — генерация аналитических данных (доступно только администратору).

- Указать клиента — действие, включаемое при создании заказа (обязательный шаг).

Типы отношений между Use Case:

- Ассоциация (Association) — прямая линия, соединяющая актора с вариантом использования. Показывает, что актор использует данную функцию.

- Отношение включения (<<include>>) — пунктирная стрелка, указывающая, что один вариант использования всегда включает в себя другой. Например, при создании заказа обязательно нужно Указать клиента.

- Отношение расширения (<<extend>>) — пунктирная стрелка, указывающая, что один вариант использования может дополнить другой в определённых условиях. Например, при работе с заказом можно Изменить статус или Назначить повара — эти действия не обязательны, но возможны.

Таким образом, диаграмма Use Case наглядно демонстрирует функциональные возможности системы и разграничение прав доступа между различными ролями пользователей. Она служит основой для дальнейшего проектирования интерфейса, разработки программного кода и создания тестовых сценариев, обеспечивая единое понимание целей и задач проекта всеми участниками.

2.4. Макеты интерфейса приложения

Разработка интерфейса приложения требует чёткого понимания того, как пользователи будут взаимодействовать с системой. На этом этапе создаются визуальные макеты, которые демонстрируют структуру окон, расположение элементов управления, навигацию и общее представление о пользовательском опыте. Макеты позволяют заранее определить, как будет выглядеть и функционировать приложение, и служат мостом между проектными решениями и их программной реализацией.

Для информационной системы пищевого производства особое внимание уделяется удобству и простоте использования, так как интерфейс должен быть доступен сотрудникам с разным уровнем подготовки. Макеты отражают разграничение прав между ролями (администратор, менеджер, повар), предусматривают эффективную навигацию и быстрый доступ к ключевым функциям, таким как работа с заказами, товарами, складом и аналитикой. Также они учитывают специфику бизнес-процессов, чтобы минимизировать время на выполнение задач и снизить вероятность ошибок.

В рамках проекта были разработаны основные макеты интерфейса, соответствующие ключевым сценариям использования. Они отражают как общие принципы дизайна, так и особенности взаимодействия для каждой роли. Эти макеты стали основой для создания программного кода и

обеспечивают согласованность между проектной документацией и финальным продуктом.

The image displays three sequential mockups of a software window titled 'Авторизация' (Authorization) for a system named 'Пищевое производство' (Food Production). The first mockup shows the main interface with two buttons: 'Я сотрудник' (I am an employee) and 'Я клиент' (I am a client). The second mockup, which appears as a floating panel, shows the login form with fields for 'Логин' (Login) and 'Пароль' (Password, masked with asterisks), and a 'Войти' (Login) button. The third mockup, also a floating panel, shows the registration or order form with fields for 'Email' and '№ Заказа' (Order Number), and buttons for 'Каталог' (Catalog) and 'Проверить заказ' (Check order).

Рисунок 2.6. Макет окна авторизации

Начальный экран приложения, предназначенный для определения роли пользователя и предоставления доступа к соответствующим функциям системы. Окно имеет заголовок «Авторизация» и отображает название предприятия — «Пищевое производство».

В центре расположены две основные кнопки:

«Я сотрудник» — активирует панель входа в систему для внутренних пользователей (администраторов, менеджеров, поваров). При нажатии открывается всплывающая панель с полями ввода:

- Логин — текстовое поле для ввода уникального имени пользователя;
- Пароль — защищённое поле;

- Кнопка «Войти» — подтверждает введенные данные и выполняет аутентификацию.

«Я клиент» — активирует панель для внешних пользователей. Открывает всплывающую форму с возможностью:

- Проверить статус заказа: поля Email и № Заказа, а также кнопка «Проверить заказ»;
- Перейти к каталогу товаров: кнопка «Каталог».

В правом верхнем углу каждой всплывающей панели расположен крестик (X), позволяющий закрыть её без сохранения или выполнения действий.

Данный макет обеспечивает разграничение доступа на уровне интерфейса: сотрудники получают доступ к полному функционалу после авторизации, а клиенты — к ограниченным возможностям просмотра и оформления заказов. Это позволяет упростить взаимодействие и избежать избыточной информации для каждой категории пользователей.

Название	Цена	Количество	Итого

Итого: 0

Рисунок 2.7 Макет окна «Проверить заказ»

Данное окно предназначено для клиентов, желающих получить информацию о текущем статусе своего заказа. Оно открывается после ввода Email и номера заказа на начальном экране авторизации и представляет собой простой, информационный интерфейс, не требующий от пользователя дополнительных действий, кроме просмотра данных.

В верхней части окна отображаются два основных поля:

- Дата заказа — показывает дату создания заказа (в формате ДД.ММ.ГГГГ ЧЧ:ММ).
- Статус — отражает текущее состояние заказа (например, «Новый», «В процессе», «Готово», «Завершён», «Отменён»).

Основную часть окна занимает таблица с детализацией заказа, содержащая следующие столбцы:

- Название — наименование товара.
- Цена — стоимость единицы товара.
- Количество — количество единиц товара в заказе.
- Итого — сумма по позиции (Цена × Количество).

В нижней части окна отображается общий итог по заказу — «Итого: 0» (или соответствующая сумма), который рассчитывается автоматически как сумма всех значений в столбце «Итого».

Такой макет обеспечивает максимальную простоту и доступность информации для конечного пользователя, минимизируя нагрузку на интерфейс и фокусируясь на ключевых данных.

Каталог товаров		
Товар 1	0	Состав
Товар 1	0	Состав
Товар 1	0	Состав
Товар 1	0	Состав
Товар 1	0	Состав
Товар 1	0	Состав
Товар 1	0	Состав

Заказать

Рисунок 2.8 Макет окна «Каталог товаров»

Данное окно предназначено для клиентов желающих ознакомиться с ассортиментом продукции и оформить заказ. Оно открывается из начального экрана авторизации по кнопке «Каталог» или из окна проверки заказа.

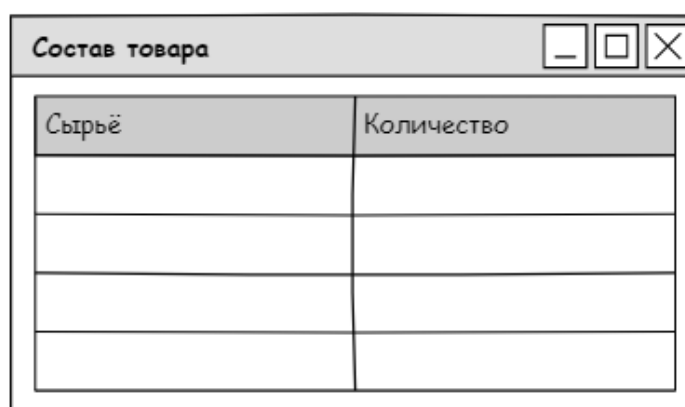
Основную часть занимает список доступных товаров, представленный в виде вертикального списка строк, каждая из которых содержит:

- Название товара — текстовое поле, отображающее наименование продукта (например, «Товар 1»).
- Поле количества — элемент управления типа `NumericUpDown`, позволяющий пользователю выбрать количество единиц товара. Значение может изменяться с помощью стрелок или прямого ввода.
- Кнопка «Состав» — позволяет просмотреть детализированный состав продукта.

Пользователь может пролистывать список и выбирать нужные товары, указывая их количество.

В нижней части окна расположена кнопка «Заказать». При нажатии она инициирует создание нового заказа на основе выбранных товаров и их количества. Если все поля количества равны нулю, система должна выдавать предупреждение: «Выберите хотя бы один товар».

Этот макет обеспечивает простой и интуитивно понятный интерфейс для клиента, позволяя ему быстро ознакомиться с ассортиментом, подобрать необходимые позиции и оформить заказ с минимальным количеством действий.



Сырьё	Количество

Рисунок 2.9 Макет окна «Состав товара»

Данное окно предназначено для просмотра детализированного состава выбранного продукта. Основную часть занимает таблица с двумя столбцами:

- Сырьё — перечень компонентов, входящих в состав продукта.

- Количество — объём каждого компонента, необходимого для производства единицы товара.

Окно не содержит кнопок редактирования или сохранения, так как его задача — исключительно информировать пользователя о составе продукта.

Такой макет обеспечивает простоту восприятия и быстрый доступ к информации о составе, что особенно важно для клиентов, интересующихся аллергенами, калорийностью или качеством продукции.

Заказать

Email

ФИО

Контактное лицо

Телефон

Название	Цена	Количество	Итого

Итого: 0

Рисунок 2.10 Макет окна «Заказать»

Данное окно предназначено для клиентов или менеджера, желающих оформить новый заказ. Оно открывается непосредственно после выбора товаров в каталоге.

В верхней части окна расположены поля для ввода контактной информации клиента:

- Email — поле для ввода адреса электронной почты (обязательное).
- ФИО — поле для ввода полного имени клиента.
- Контактное лицо — дополнительное поле, если заказ делается от лица организации.
- Телефон — поле для ввода номера телефона (обязательное).

Справа от поля Email находится кнопка «Проверить», которая выполняет поиск клиента и заполняет данные автоматически если находит.

Основную часть окна занимает таблица с выбранными товарами, состоящая из следующих столбцов:

- Название — наименование товара.
- Цена — стоимость единицы товара.
- Количество — количество единиц, выбранное клиентом.
- Итого — сумма по позиции (Цена × Количество).

В нижней части окна отображается общая сумма заказа — «Итого: 0», которая рассчитывается автоматически как сумма всех значений в столбце «Итого».

В правом нижнем углу расположена кнопка «Заказать». При нажатии она инициирует создание нового заказа в системе на основе выбранных товаров и введённых данных клиента. Если все поля не заполнены или количество товаров равно нулю, система должна выдавать предупреждение: «Заполните все обязательные поля и выберите хотя бы один товар».

Этот макет обеспечивает простой и понятный интерфейс для клиента, позволяя ему быстро оформить заказ, указав необходимые данные и подтвердив выбор товаров.



Рисунок 2.11 Макет окна «Панель управления» (панель администратора)

Данное окно представляет собой центральный навигационный хаб для администратора системы. Оно открывается после успешной авторизации и служит основой для доступа ко всем ключевым модулям информационной системы пищевого производства.

Основное пространство разделено на восемь крупных кнопок, расположенных в виде сетки 4×2, каждая из которых ведёт к отдельному функциональному блоку:

- Клиенты — управление базой данных клиентов.
- Сырьё — учёт и контроль запасов сырья.
- Заказы — полное управление заказами: просмотр, создание, изменение статуса, назначение исполнителей.
- Продукты — управление ассортиментом готовой продукции.
- Поставки — фиксация и учёт поставок сырья от поставщиков.
- Сотрудники — управление персоналом: добавление, редактирование, увольнение, восстановление.
- Поставщики — учёт и управление данными о компаниях-поставщиках.
- Авторизация — администрирование учётных записей пользователей и уровней доступа.

В нижней части окна расположена кнопка «Аналитика», открывающая доступ к отчётам и аналитическим инструментам.

Такой макет обеспечивает удобную и логичную навигацию для администратора, позволяя ему за один клик перейти к нужному функционалу без необходимости просматривать сложные меню или вкладки.

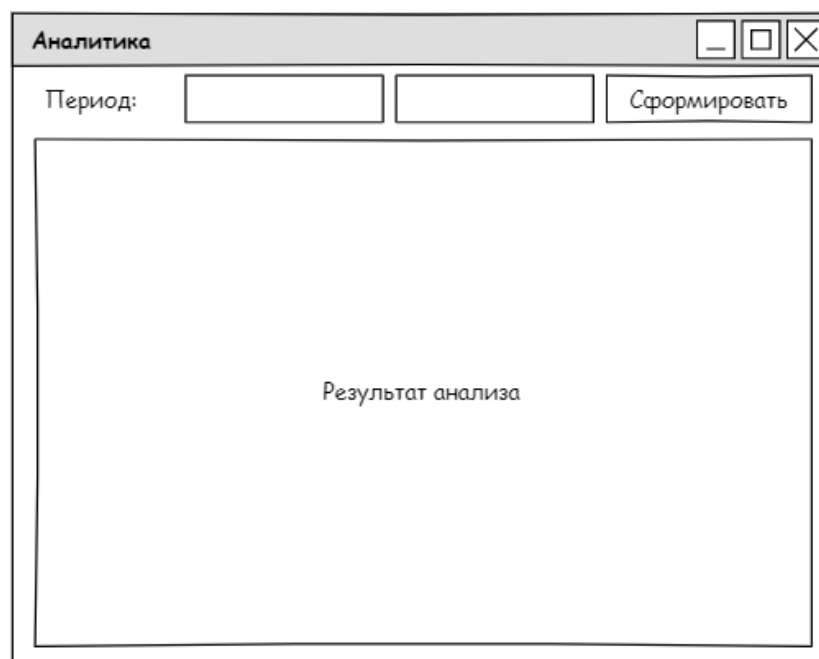


Рисунок 2.12 Макет окна «Аналитика»

Данное окно служит центральным инструментом для получения сводной информации о деятельности предприятия. Оно позволяет формировать отчёты за выбранный временной интервал.

В верхней части окна расположен заголовок «Аналитика». Ниже находится блок настройки отчёта:

- Поле «Период» — состоит из двух текстовых полей, в которые пользователь вводит дату начала и дату окончания периода анализа.
- Кнопка «Сформировать» — запускает процесс сбора и агрегации данных из базы данных за указанный период.

Основную часть окна занимает большая область с надписью «Результат анализа». После нажатия кнопки «Сформировать» в этой области отображается итоговый отчёт.

Такой макет обеспечивает простой и целенаправленный интерфейс, позволяя администратору быстро получить ключевые показатели эффективности бизнеса без необходимости просматривать множество таблиц или диаграмм.

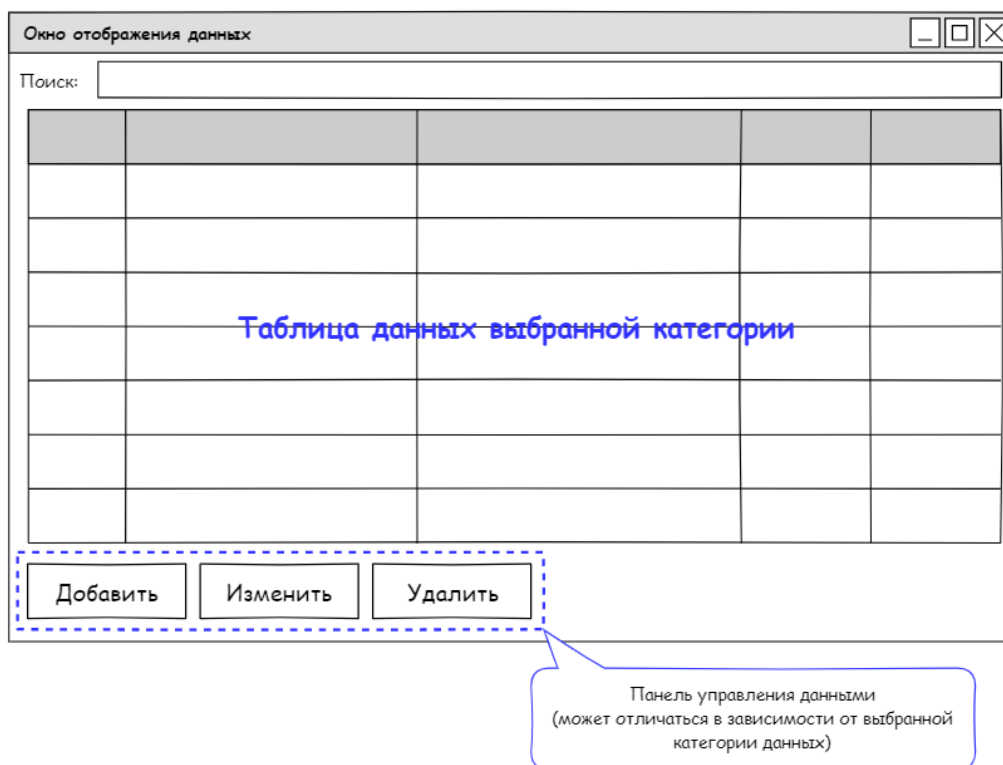


Рисунок 2.13 Макет окна отображения данных

Данное окно является универсальным интерфейсом для просмотра и управления данными в системе. Оно открывается из «Панели управления» по нажатию любой из кнопок: «Клиенты», «Заказы», «Сотрудники», «Продукты», «Сырьё», «Поставщики» или «Поставки». В зависимости от выбранной категории, содержимое окна динамически меняется — меняются заголовки столбцов таблицы и набор доступных действий в нижней панели.

В верхней части окна расположен заголовок «Окно отображения данных», а под ним — поле «Поиск», предназначенное для фильтрации строк в таблице по любому значению.

Основную часть окна занимает таблица с данными, структура которой зависит от выбранной категории:

- Для «Клиенты» — столбцы: ID, Полное имя, Контактное лицо, Телефон, Email.
- Для «Заказы» — столбцы: ID, Клиент, Менеджер, Повар, Дата создания, Статус, Общая сумма.
- Для «Сотрудники» — столбцы: ID, ФИО, Должность, Телефон, Email, Дата приёма.

- Для «Продукты» — столбцы: ID, Название, Категория, Цена, Срок годности.
- Для «Сырьё» — столбцы: ID, Название, Ед. измерения, Кол-во на складе, Мин. остаток.
- Для «Поставщики» — столбцы: ID, Название компании, Контактное лицо, Телефон, Email, Адрес.
- Для «Поставки» — столбцы: ID, Поставщик, Сырьё, Количество, Дата поставки, Сотрудник.

В нижней части окна расположена панель управления данными, которая может меняться в зависимости от категории данных.

Такой макет обеспечивает гибкость и единообразие интерфейса: пользователь всегда видит один и тот же шаблон окна, но его содержимое и функциональность адаптируются под текущую задачу. Это снижает порог входа для новых пользователей и упрощает обучение персонала.

The image shows a window titled "Клиенты - Добавить/Изменить". Inside the window, there are four text input fields arranged vertically, each with a label to its left: "ФИО", "Контактное лицо", "Номер телефона", and "Электронная почта". Below these fields, centered, is a button with the text "Добавить/Изменить". The window has a standard title bar with minimize, maximize, and close buttons.

Рисунок 2.14 Макет окна добавления/изменения клиентов

Данное окно предназначено для создания новой записи о клиенте или редактирования существующей.

Основную часть окна занимают четыре поля ввода:

- ФИО — текстовое поле для ввода полного имени клиента.
- Контактное лицо — поле для указания лица, с которым следует связываться.
- Номер телефона — поле для ввода контактного номера.

- Электронная почта — поле ввода адреса электронной почты клиента.

В нижней части окна расположена универсальная кнопка «Добавить/Изменить». Её название отражает двойственное назначение:

- При создании нового клиента — она выполняет действие «Добавить» и сохраняет введённые данные в базу данных.
- При редактировании существующей записи — она выполняет действие «Изменить» и обновляет данные в базе.

Такой подход упрощает интерфейс, так как не требует создания двух отдельных окон для добавления и редактирования.

Этот макет обеспечивает единообразие и простоту для администратора, позволяя быстро добавлять или корректировать информацию о клиентах без необходимости переключаться между разными формами.

The image shows a screenshot of a software window titled "Сырьё - Добавить/Изменить". The window has a standard Windows-style title bar with minimize, maximize, and close buttons. Inside the window, there are four text input fields arranged vertically, each preceded by a label: "Наименование", "Единица измерения", "Мин. остаток", and "На складе". Below these input fields, centered, is a button with the text "Добавить/Изменить".

Рисунок 2.15 Макет окна добавления/изменения сырья

Данное окно предназначено для создания новой записи о сырье или редактирования существующей.

Основную часть окна занимают четыре поля ввода:

- Наименование — текстовое поле для ввода названия сырья.
- Единица измерения — поле для указания единицы измерения сырья.
- Мин. остаток — поле для установки минимального допустимого количества сырья на складе.
- На складе — поле ввода текущего количества сырья на складе.

В нижней части окна расположена универсальная кнопка «Добавить/Изменить». Её название отражает двойственное назначение:

- При создании нового сырья — она выполняет действие «Добавить» и сохраняет введённые данные в базу данных.

- При редактировании существующей записи — она выполняет действие «Изменить» и обновляет данные в базе.

Такой подход упрощает интерфейс, так как не требует создания двух отдельных окон для добавления и редактирования.

Продукты - Добавить/Изменить	
Наименование	
Категория	
Ед. измерения	
Срок годности	
Цена за ед.	
Состав:	
Сырьё	Количество
Добавить Удалить Добавить/Изменить	

Рисунок 2.16 Макет окна добавления/изменения продуктов

Данное окно предназначено для создания новой записи о продукте или редактирования существующей.

Основную часть окна занимают пять полей ввода:

- Наименование — текстовое поле для ввода названия готового продукта.

- Категория — поле для указания категории продукта.

- Ед. измерения — поле для указания единицы измерения продукта.

- Срок годности — поле для ввода срока годности продукта в днях.

- Цена за ед. — поле для указания стоимости одной единицы продукта.

Ниже этих полей находится секция «Состав», которая представляет собой таблицу с двумя столбцами:

Сырьё — список ингредиентов, входящих в состав продукта.

Количество — объём каждого компонента, необходимого для производства единицы продукта.

В нижней части окна расположены три кнопки:

- Добавить — открывает диалог для добавления нового ингредиента в состав продукта.
- Удалить — удаляет выбранную строку из таблицы состава.
- Добавить/Изменить — универсальная кнопка, выполняющая действие в зависимости от режима: при создании — сохраняет новый продукт, при редактировании — обновляет существующую запись.

Такой подход позволяет использовать одну форму как для создания, так и для редактирования продуктов, что упрощает интерфейс и снижает количество окон в системе.

Этот макет обеспечивает гибкость и точность при работе с продукцией, позволяя администратору не только задать основные параметры товара, но и детально прописать его рецептуру, что является ключевым требованием для пищевого производства.

Рисунок 2.17 Макет окна добавления поставок

Данное окно предназначено для оформления новой поставки сырья от поставщика. В отличие от других форм, это окно не имеет режима редактирования — каждая поставка фиксируется как отдельная операция и не подлежит изменению после создания.

В верхней части окна расположен заголовок «Поставки — Добавить», указывающий на назначение формы.

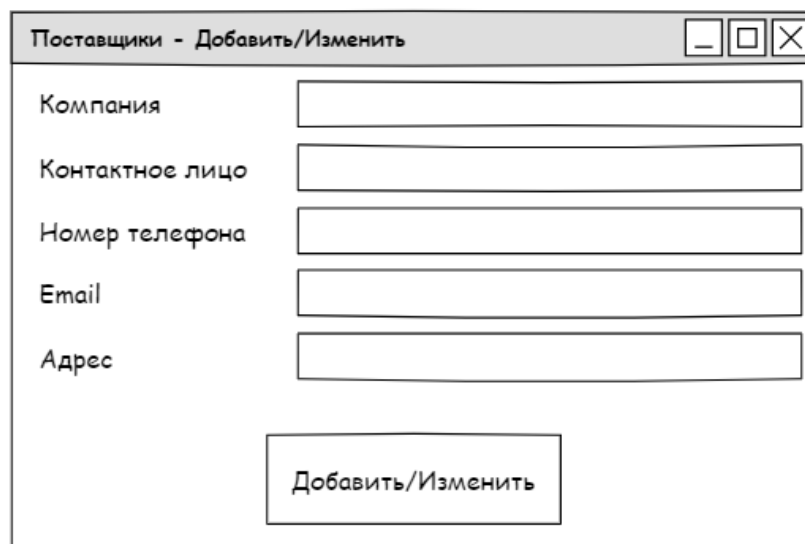
Основную часть окна занимают три поля:

- Поставщик — выпадающий список, содержащий список всех зарегистрированных поставщиков.
- Сырьё — выпадающий список, содержащий перечень всех видов сырья, учтённых в системе.
- Количество — текстовое поле с возможностью ввода числового значения, указывающее объём поступившего сырья.

В нижней части окна расположена кнопка «Добавить», которая:

- Сохраняет данные о поставке в базу данных.
- Автоматически обновляет остатки сырья на складе.
- Фиксирует дату поставки и ответственного сотрудника.

Такой макет обеспечивает простоту и надёжность при работе с поставками: пользователь быстро выбирает поставщика и сырьё, указывает количество и подтверждает операцию. Это минимизирует ошибки и гарантирует актуальность данных в системе.



The image shows a screenshot of a software window titled "Поставщики - Добавить/Изменить". The window has a standard Windows-style title bar with minimize, maximize, and close buttons. Inside the window, there are five text input fields arranged vertically, each with a label to its left: "Компания", "Контактное лицо", "Номер телефона", "Email", and "Адрес". Below these fields, centered, is a button labeled "Добавить/Изменить".

Рисунок 2.18 Макет окна добавления/изменения поставщиков

Данное окно предназначено для создания новой записи о поставщике или редактирования существующей.

Основную часть окна занимают пять полей ввода:

- Компания — текстовое поле для ввода названия компании-поставщика.

- Контактное лицо — поле для указания имени сотрудника, с которым следует связываться.
- Номер телефона — поле для ввода контактного номера.
- Email — поле для ввода адреса электронной почты.
- Адрес — поле для указания юридического или фактического адреса поставщика.

В нижней части окна расположена универсальная кнопка «Добавить/Изменить». Её название отражает двойственное назначение:

- При создании нового поставщика — она выполняет действие «Добавить» и сохраняет данные в базу.
- При редактировании существующей записи — она выполняет действие «Изменить» и обновляет информацию.

Рисунок 2.19 Макет окна изменения сотрудников

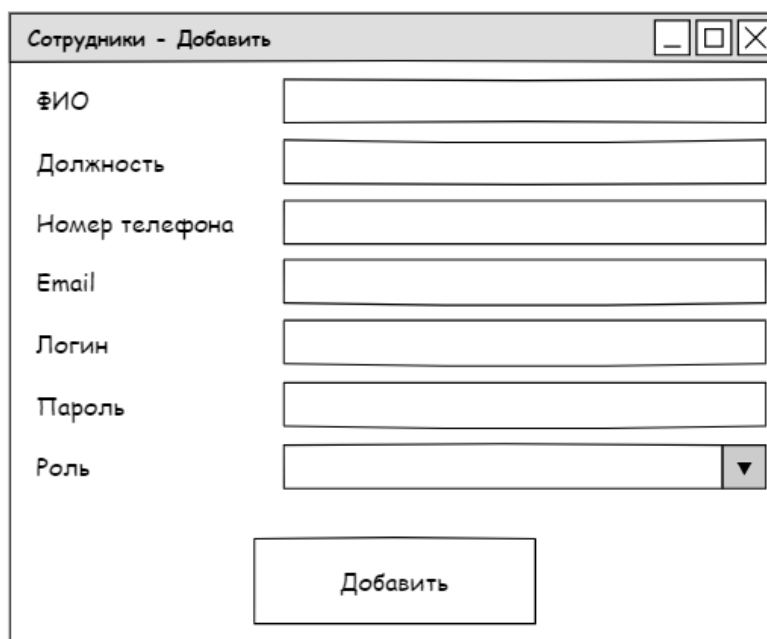
Данное окно предназначено исключительно для редактирования данных существующего сотрудника. В отличие от других форм, здесь отсутствует режим создания — новая запись о сотруднике создаётся через отдельную форму, а это окно служит только для корректировки уже внесённой информации.

Основную часть окна занимают четыре поля ввода, расположенные вертикально:

- ФИО — текстовое поле для редактирования полного имени.
- Должность — поле для изменения должности.
- Номер телефона — поле для обновления контактного номера.

- Email — поле для редактирования адреса электронной почты.

В нижней части окна расположена кнопка «Изменить», которая сохраняет внесённые правки в базу данных.



The image shows a window titled "Сотрудники - Добавить". It contains the following fields and controls:

- ФИО: text input field
- Должность: text input field
- Номер телефона: text input field
- Email: text input field
- Логин: text input field
- Пароль: text input field
- Роль: dropdown menu with a downward arrow
- Добавить: button at the bottom center

Рисунок 2.20 Макет окна добавления сотрудников

Данное окно предназначено исключительно для создания новой записи о сотруднике. В отличие от формы редактирования, здесь нет режима изменения — это чисто вводная форма, предназначенная для регистрации нового пользователя системы.

Основную часть окна занимают шесть полей ввода:

- ФИО — текстовое поле для ввода полного имени сотрудника.
- Должность — поле для указания занимаемой.
- Номер телефона — поле для ввода контактного номера
- Email — поле для ввода адреса электронной почты.
- Логин — уникальное имя пользователя для входа в систему.
- Пароль — поле для установки пароля.

Ниже этих полей расположен элемент «Роль» — выпадающий список, позволяющий выбрать уровень доступа сотрудника: 0 — Администратор, 1 — Менеджер, 2 — Повар, -1 — Заблокирован (неактивный пользователь).

В нижней части окна расположена кнопка «Добавить», которая создаёт новую запись в таблице Сотрудники и Авторизация.

Рисунок 2.21 Макет окна изменения авторизации сотрудников

Данное окно предназначено для редактирования учётных данных сотрудника — логина, пароля и уровня доступа.

Основную часть окна занимают три поля:

- Логин — текстовое поле для изменения уникального имени пользователя. При редактировании логина система должна проверять его уникальность.
- Пароль — поле для смены пароля (символы маскируются). Позволяет администратору сбросить или изменить пароль сотрудника.
- Роль — выпадающий список, позволяющий выбрать уровень доступа.

В нижней части окна расположена кнопка «Изменить», которая сохраняет внесённые изменения в таблице Авторизация.

Название	Цена	Количество	Итого

Рисунок 2.22 Макет окна редактирования заказа

Данное окно предназначено для просмотра и редактирования данных заказа. Оно доступно всем сотрудникам системы — администратору, менеджеру и повару — но с различным уровнем прав доступа, что обеспечивает контроль над изменениями в зависимости от роли пользователя.

В верхней части окна расположен заголовок «Редактор заказа», а ниже — информация о заказе:

- Заказ — номер заказа
- Клиент — имя клиента.
- Менеджер — имя ответственного менеджера.
- Повар — имя назначенного повара.
- Дата создания — дата и время оформления заказа.
- Дата завершения — дата, когда заказ был завершён (заполняется автоматически при изменении статуса).

- Статус — текущий статус заказа.

Ниже расположена таблица с составом заказа, содержащая столбцы:

- Название — наименование товара.
- Цена — стоимость единицы товара.
- Количество — количество единиц в заказе.
- Итого — сумма по позиции (Цена × Количество).

Уровни доступа:

Администратор — имеет полный доступ к редактированию всех полей: может изменить клиента, менеджера, повара, статус, дату завершения. Исключение: не может изменять данные после того, как заказ был завершён или отменён — в этом случае все поля становятся недоступными для редактирования, чтобы сохранить целостность истории.

Менеджер — может:

- Изменять статус заказа (например, с «Новый» на «В процессе»).
- Назначать повара через выпадающий список.
- Не может изменять клиентские данные.

Повар — может:

- Изменять статус заказа только с «В процессе» на «Готово».
- Не может назначать других поваров, менять менеджера или клиентские данные.

В правом нижнем углу окна расположена кнопка «Применить», которая сохраняет внесённые изменения.

Такой макет обеспечивает гибкое управление заказами с учётом бизнес-логики предприятия: каждый сотрудник работает в рамках своей зоны ответственности, что снижает риск ошибок и повышает прозрачность процессов.

№	Клиент	Менеджер	Повар	Дата заказа	Дата завершения	Сумма	Статус

Добавить Изменить Обновить

Рисунок 2.23 Макет окна менеджера

Данное окно предназначено для работы менеджера с заказами, назначенными на него. Оно открывается после авторизации и служит основным рабочим интерфейсом для управления своими заказами — просмотра, редактирования статуса и назначения поваров.

В верхней части окна расположен заголовок «Окно Менеджера», а под ним — поле «Поиск», позволяющее фильтровать заказы по любому значению.

Основную часть окна занимает таблица с данными заказов, в которой отображаются следующие столбцы:

- № — номер заказа.
- Клиент — имя клиента, сделавшего заказ.
- Менеджер — имя ответственного менеджера (текущий пользователь или отсутствие менеджера).
- Повар — имя назначенного повара (если есть).
- Дата заказа — дата и время оформления заказа.
- Дата завершения — дата, когда заказ был завершён.
- Сумма — общая стоимость заказа.
- Статус — текущее состояние заказа: «Новый», «В процессе», «Готово», «Завершён», «Отменён».

В нижней части окна расположены три кнопки:

- Добавить — открывает форму создания нового заказа.
- Изменить — открывает окно редактора заказа.
- Обновить — обновляет данные в таблице.

Такой макет обеспечивает удобный и целенаправленный интерфейс для менеджера, позволяя ему быстро просматривать свои заказы, контролировать их статус и оперативно назначать исполнителей. Это повышает эффективность работы и снижает время на выполнение задач.

Окно Повара

Поиск:

№	Клиент	Менеджер	Повар	Дата заказа	Дата завершения	Сумма	Статус

Изменить Обновить

Рисунок 2.24 Макет окна повара

Данное окно предназначено для работы повара с заказами, назначенными на него. Оно открывается после авторизации и служит основным рабочим интерфейсом для просмотра текущих задач и обновления статусов выполнения.

В верхней части окна расположен заголовок «Окно Повара», а под ним — поле «Поиск», позволяющее фильтровать заказы по любому значению.

Основную часть окна занимает таблица с данными заказов, в которой отображаются следующие столбцы:

- № — номер заказа.
- Клиент — имя клиента, сделавшего заказ.
- Менеджер — имя ответственного менеджера (текущий пользователь или отсутствие менеджера).
- Повар — имя назначенного повара (если есть).
- Дата заказа — дата и время оформления заказа.
- Дата завершения — дата, когда заказ был завершён.
- Сумма — общая стоимость заказа.
- Статус — текущее состояние заказа: «Новый», «В процессе», «Готово», «Завершён», «Отменён».

В нижней части окна расположены две кнопки:

- Изменить — открывает окно редактора заказа.
- Обновить — обновляет данные в таблице.

Такой макет обеспечивает минимальный и целенаправленный интерфейс для повара, позволяя ему быстро находить свои заказы и оперативно отмечать их как готовые. Это снижает нагрузку на пользователя, исключает ошибки и повышает прозрачность производственного процесса.

2.5. Описание логики функционирования ПО

Логика функционирования разработанного программного обеспечения информационной системы пищевого производства основана на чётко определённых бизнес-процессах предприятия, разграничении прав доступа между пользователями и целостности данных. Программное обеспечение

реализует автоматизацию ключевых операций: приём заказов, учёт сырья, управление производством, формирование отчётов и взаимодействие с клиентами. Программное обеспечение построено по модельно-ориентированному принципу, где данные хранятся в централизованной базе данных (Microsoft SQL Server), а приложение предоставляет интерфейс для взаимодействия с этими данными. Основные модули системы включают модуль авторизации и управления пользователями, модуль работы с заказами, модуль управления складом и производством, модуль клиентского взаимодействия и модуль аналитики. Каждый модуль связан с базой данных через запросы (SELECT, INSERT, UPDATE, DELETE), и при этом реализованы проверки целостности, бизнес-правила и валидация вводимых данных.

При запуске приложения пользователь попадает на окно авторизации. В зависимости от выбранной роли (сотрудник или клиент) предоставляется доступ к различным функциям. Для клиента доступны ограниченные функции: проверка статуса заказа (ввод email и номера) и просмотр каталога товаров. Для сотрудника запрашиваются логин и пароль. Система проверяет их в таблице Авторизация базы данных. Если данные верны, система определяет уровень доступа (уровень — 1: заблокирован, 0: администратор, 1: менеджер, 2: повар) и открывает соответствующий интерфейс. Каждая форма и действие в системе проверяют уровень доступа пользователя. Только администратор может управлять всеми данными (сотрудники, продукты, поставки, авторизация), менеджер может работать с заказами: изменять статус, назначать поваров, а повар может изменить статус заказа с «В процессе» на «Готово».

Процесс создания и обработки заказа включает в себя следующую логику. Клиент может создать заказ через окно «Каталог» -> «Заказать», указав свои данные и выбрав товары с количеством. Система проверяет, все ли обязательные поля заполнены, и затем сохраняет заказ в таблицу Заказы с начальным статусом «Новый». Менеджер может открыть заказ в своём окне,

изменить статус (например, на «В процессе») и назначить повара. При изменении статуса на «В процессе» система автоматически списывает необходимое сырьё из таблицы Склад на основе данных из таблицы Рецепттуры. Повар, открыв заказ в своём окне, может изменить статус на «Готово», если он назначен и статус — «В процессе». После этого заказ считается выполненным. Заказ нельзя изменить, если его статус — «Завершён» или «Отменён».

При создании заказа и его переводе в статус «В процессе» система производит проверку наличия необходимого сырья на складе (Склад) на основе данных из Рецепттуры. Если сырья недостаточно, система выводит предупреждение и блокирует перевод статуса. Если сырья достаточно, система автоматически списывает требуемое количество из таблицы Склад. При поступлении новой партии сырья через окно «Поставки» — система увеличивает остатки в таблице Склад.

Клиент может проверить статус своего заказа, введя в окне авторизации email и номер заказа. Система проверяет, совпадает ли email с данными в таблице Клиенты, и существует ли заказ с таким номером. Если данные совпадают — открывается окно «Проверить заказ», где отображаются дата создания, статус, состав заказа и общая сумма. Клиент также может просматривать каталог товаров, не проходя авторизацию, и оформлять заказ, если заполнит обязательные поля (email, ФИО, телефон).

Администратор может сформировать аналитический отчёт в окне «Аналитика», указав период. Система фильтрует данные из таблиц Заказы, Позиции заказа, Продукты, Сотрудники, Сырьё по дате, выполняет агрегацию: суммарная прибыль, количество заказов, популярные товары, эффективность сотрудников, и выводит результат в области «Результат анализа».

Все формы ввода данных содержат валидацию на стороне клиента (например, проверка формата email, обязательных полей) и проверки на сервере при сохранении. Система отслеживает нарушения целостности данных (например, удаление товара, который используется в заказе), попытки

недопустимых действий (например, изменение завершённого заказа). В случае ошибки — система выводит понятное сообщение пользователю и не позволяет выполнить некорректное действие.

Таким образом, логика функционирования ПО обеспечивает надёжное и согласованное выполнение всех бизнес-процессов, поддерживает безопасность данных, разграничение прав доступа и удобство использования для всех категорий пользователей.

3. СПЕЦИАЛЬНАЯ ЧАСТЬ

3.1. Формирование требований к ПО

Формирование требований к программному обеспечению (ПО) — это ключевой этап на пути к созданию эффективной и надёжной информационной системы. На этом этапе определяются цели, функции, характеристики и ограничения, которым должно соответствовать будущее приложение. В рамках дипломного проекта, посвящённого разработке информационной системы для предприятия пищевого производства, требования к ПО формировались на основе анализа предметной области, изучения бизнес-процессов, идентификации проблем в текущей организации работы и специфики пищевой отрасли.

Целью разработки программного обеспечения является автоматизация ключевых бизнес-процессов предприятия пищевого производства. Это включает в себя:

- Управление заказами клиентов: приём, учёт, отслеживание статуса, назначение исполнителей.
- Управление складскими запасами: учёт сырья и готовой продукции, контроль минимальных остатков, списание при производстве.
- Управление производственным процессом: планирование, контроль выполнения, учёт расхода материалов по рецептам.
- Управление клиентами, поставщиками, сотрудниками и продуктами: хранение данных, история взаимодействия.
- Формирование аналитических отчётов: прибыль, популярные товары, эффективность сотрудников, расход сырья.

Функциональные требования описывают конкретные задачи, которые система должна выполнять. Они определяют, что делает система. К функциональным требованиям можно отнести:

- Управление заказами: создание, редактирование, изменение статуса, назначение менеджера и повара.

- Управление складом: учёт остатков, автоматическое списание сырья при переводе заказа в статус «В процессе», учёт поставок.

- Управление продуктами и рецептами: хранение информации о товарах, их составе, ценах, сроках годности.

- Управление персоналом: добавление, редактирование, удаление сотрудников, настройка учётных записей и уровней доступа.

- Клиентская зона: возможность проверки статуса заказа по email и номеру, просмотр каталога.

- Аналитика: формирование отчётов по заданному периоду.

Нефункциональные требования определяют качественные характеристики системы. Они описывают, как система должна работать. К нефункциональным требованиям относятся:

- Безопасность: система должна обеспечивать аутентификацию и авторизацию пользователей, разграничение прав доступа по ролям, защиту конфиденциальных данных.

- Надёжность: система должна быть устойчивой к сбоям, обеспечивать целостность данных и иметь возможность восстановления.

- Удобство использования: интерфейс должен быть интуитивно понятным, на русском языке, с логичной навигацией.

- Производительность: система должна отвечать на запросы в приемлемое время (например, открытие списка заказов — не более 3 секунд).

- Совместимость: приложение должно работать в выбранной операционной системе (Windows) и корректно взаимодействовать с СУБД (Microsoft SQL Server).

- Масштабируемость: архитектура должна предусматривать возможность добавления новых функций и роста объёма данных.

При формировании требований были учтены бизнес-правила, характерные для пищевого производства:

- Запрет на изменение заказа после его завершения или отмены.

- Обязательное наличие достаточного количества сырья на складе перед переводом заказа в статус «В процессе».
- Автоматическое списание сырья по рецепту при начале производства.
- Разграничение прав: администратор управляет всем, менеджер — только заказами, повар — только статусом «Готово».
- Обязательные поля при создании заказа и клиента.

Таким образом, сформированные требования к программному обеспечению отражают реальные потребности предприятия, обеспечивают выполнение всех ключевых бизнес-процессов и служат основой для последующих этапов проектирования и разработки.

3.2. Разработка интерфейса ПО

На основе сформированных требований и макетов, разработанных на этапе проектирования, был реализован пользовательский интерфейс информационной системы для предприятия пищевого производства. Интерфейс выполнен в стиле десктопного приложения под Windows с использованием технологии Windows Forms и ориентирован на удобство и эффективность работы сотрудников различных ролей — администратора, менеджера, повара и клиента.

Все окна приложения соответствуют ранее утверждённым макетам: сохранена структура полей, расположение кнопок, логика взаимодействия и разграничение функционала по ролям. Визуальный стиль выбран минималистичным и функциональным: чёткие шрифты, достаточные отступы, интуитивные элементы управления, без избыточной графики. Все формы поддерживают локализацию на русский язык, а ключевые операции снабжены подсказками и валидацией ввода.

Ниже представлены снимки экрана реализованных окон интерфейса, демонстрирующие их внешний вид и основные элементы управления в рабочем состоянии.

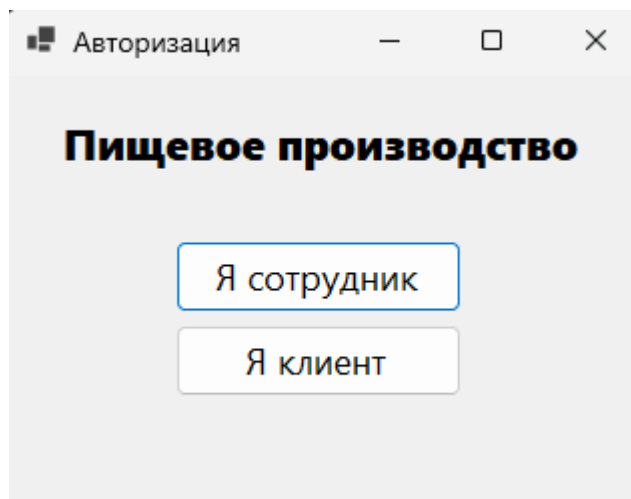


Рисунок 3.1 Окно авторизации

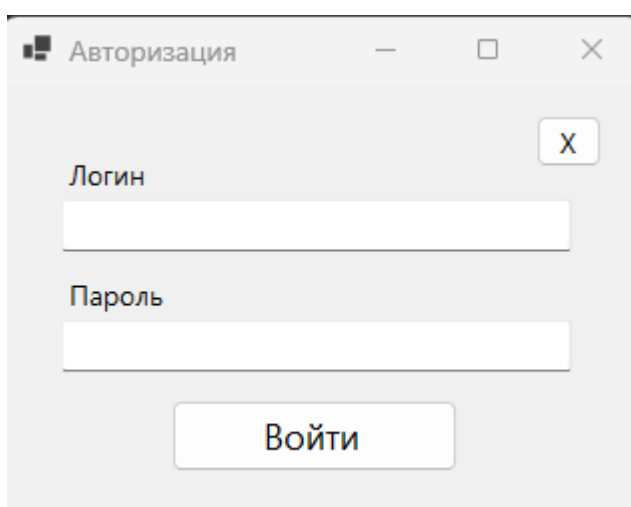


Рисунок 3.2 Окно авторизации

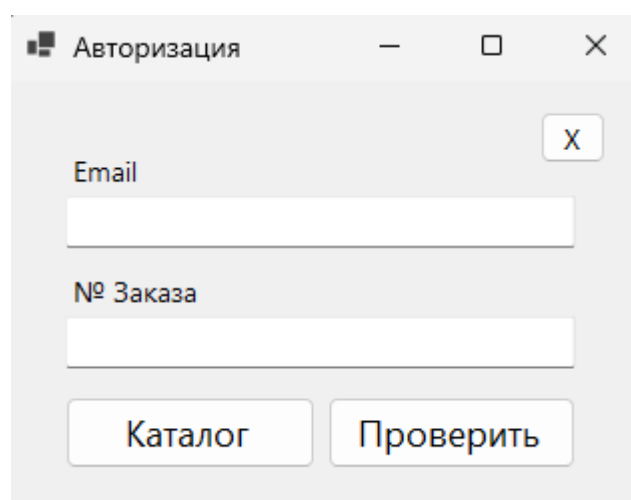


Рисунок 3.3 Окно авторизации

Каталог товаров

Булочка сдобная (45,00 Р)	0	Состав
Йогурт натуральный (80,00 Р)	0	Состав
Пирожок с картошкой (60,00 Р)	0	Состав
Куриная грудка (450,00 Р)	0	Состав
Картофель свежий (40,00 Р)	0	Состав
Сыр твердый (500,00 Р)	0	Состав
Салат "Цезарь" (250,00 Р)	0	Состав
Борщ украинский (120,00 Р)	0	Состав
Пельмени домашние (300,00 Р)	0	Состав

Заказать

Рисунок 3.4 Окно выбора позиций заказа

Заказать

Email **Проверить**

ФИО

Контактное лицо

Телефон

	Название	Цена	Количество	Итого
▶	Булочка сдобн...	45,00 Р	6	270,00 Р
	Йогурт натура...	80,00 Р	1	80,00 Р
	Пирожок с кар...	60,00 Р	2	120,00 Р
	Куриная грудка	450,00 Р	2	900,00 Р
	Картофель све...	40,00 Р	1	40,00 Р
	Сыр твердый	500,00 Р	1	500,00 Р
	Пельмени дом...	300,00 Р	6	1 800,00 Р

Итого: 3 710,00 Р

Заказать

Рисунок 3.5 Окно оформления заказа

Успех

Заказ успешно оформлен!
Номер заказа: 21

ОК

Рисунок 3.6 Окно успешно созданного заказа

Статус заказа				
Дата заказа: 27.01.2026 15:53				
Статус: Новый				
	Название	Цена	Количество	Итого
►	Булочка сдобн...	45,00 Р	6,000	270,00 Р
	Йогурт натура...	80,00 Р	1,000	80,00 Р
	Пирожок с кар...	60,00 Р	2,000	120,00 Р
	Куриная грудка	450,00 Р	2,000	900,00 Р
	Картофель све...	40,00 Р	1,000	40,00 Р
	Сыр твердый	500,00 Р	1,000	500,00 Р
	Пельмени дом...	300,00 Р	6,000	1 800,00 Р
Общая сумма: 3 710,00 Р				

Рисунок 3.7 Окно проверки заказа

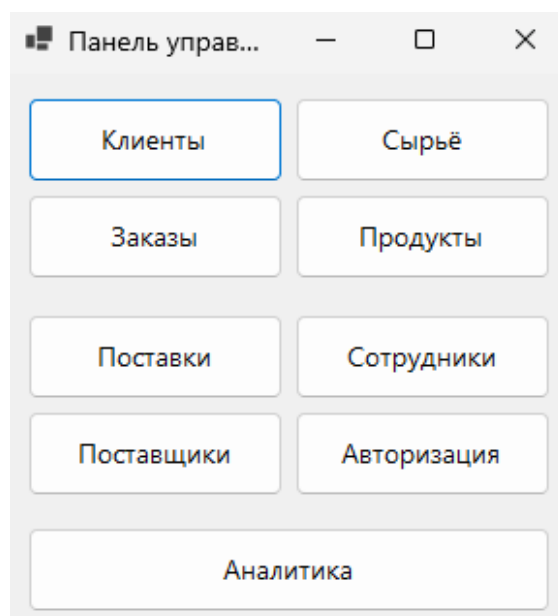


Рисунок 3.7 Окно панель управления администратора

Клиенты					
	ID	ФИО	Контактное лицо	Номер телефона	Электронная почта
▶	1	ООО "Гастрономия"	Иван Петров	+79001111111	contact@gastro.ru
	2	ИП "Шеф-Повар"	Мария Соколова	+79001111112	maria@chef.ru
	3	ООО "Вкуснятина"	Алексей Козлов	+79001111113	info@vkus.ru
	4	ООО "Сытый Кот"	Елена Новикова	+79001111114	elena@sitykot.ru
	5	ИП "Кафе Уют"	Дмитрий Белов	+79001111115	belov@cafeuyt.ru
	6	ООО "Булочная"	Ольга Медведева	+79001111116	medvedeva@bulka.ru
	7	ИП "Суши-Бар"	Сергей Жуков	+79001111117	zhukov@sushi-bar.ru
	8	ООО "Десерт-Палас"	Анна Лапина	+79001111118	lapina@desert-palace.ru
	9	ООО "Бургер-Мастер"	Виктор Семенов	+79001111119	vsemenov@burger.ru
	10	ИП "Пицца-Дом"	Татьяна Власова	+79001111120	tanya@pizza.ru
	13	new	new	new	new@email.com
Добавить					
Изменить					
Удалить					
Обновить					

Рисунок 3.8 Окно клиенты

Изменить	
ФИО	ООО "Гастрономия"
Контактное лицо	Иван Петров
Номер телефона	+79001111111
Электронная почта	contact@gastro.ru
Изменить	

Рисунок 3.9 Окно изменение клиента

Сырьё					
	ID	Наименование	Единица измерения	Минимальный	На складе
▶	1	Мука пшеничная	кг	50,000	90,900
	2	Сахар	кг	30,000	77,200
	3	Соль	кг	20,000	59,720
	4	Яйца	шт	200,000	500,000
	5	Молоко	л	50,000	70,000
	6	Куриное мясо	кг	30,000	64,600
	7	Картофель	кг	100,000	200,000
	8	Лук репчатый	кг	40,000	90,000
	9	Морковь	кг	40,000	100,000
	10	Сыр твердый	кг	20,000	40,000
	11	Сливочное масло	кг	10,000	25,000
	12	Сметана	кг	15,000	29,000
		Картофель			
Добавить					
Изменить					
Удалить					
Обновить					

Рисунок 3.10 Окно сырьё

Изменить

Наименование	Мука пшеничная
Единица измерения	кг
Минимальный	50,000
На складе	90,900

Изменить

Рисунок 3.11 Окно изменение сырья

Заказы

ID	Клиент	Менеджер	Повар	Дата заказа	Дата	Сумма	Статус
8	ООО "Десерт-Пала"	-	-	22.01.2025 11:30:00	-	750,00 Р	Новый
11	ООО "Гастрономия"	-	-	23.01.2026 3:17:51	-	1 230,00 Р	Новый
12	ООО "Гастрономия"	-	-	23.01.2026 3:26:33	-	405,00 Р	Новый
1	ООО "Гастрономия"	Петров Петр Петрович	Сидоров Сидор Сидорович	15.01.2025 10:00:00	15.01.2025 12:00:00	1 500,00 Р	Завершено
2	ИП "Шеф-Повар"	Кузнецова Анна Сергеевна	Смирнова Ольга Николаевна	16.01.2025 9:30:00	16.01.2025 11:00:00	850,00 Р	Завершено
3	ООО "Вкуснятина"	Петров Петр Петрович	Морозов Алексей Владимирович	17.01.2025 11:00:00	-	2 200,00 Р	Готово

Добавить

Изменить

Обновить

Рисунок 3.12 Окно заказы

Редактирование заказа

Заказ №19

Клиент: new

Менеджер: Иванов Иван Иванович

Повар: Сидоров Сидор Сидорович

Дата создания: 24.01.2026 Дата завершения: Не установлена

Статус: Готово

Название	Количество	Цена	Итого
Булочка сдобн...	6,000	45,00 Р	270,00 Р
Йогурт натура...	9,000	80,00 Р	720,00 Р
Пирожок с кар...	9,000	60,00 Р	540,00 Р

Общая сумма: 1 530,00

Применить

Рисунок 3.13 Окно изменения заказа

Продукты

ID	Наименование	Категория	Единица	Срок годности	Цена за ед.
1	Булочка сдобная	Выпечка	шт	2	45,00 Р
2	Йогурт натуральный	Молочные продукты	шт	7	80,00 Р
3	Пирожок с картошкой	Выпечка	шт	1	60,00 Р
4	Куриная грудка	Мясо	кг	5	450,00 Р
5	Картофель свежий	Овощи	кг	14	40,00 Р
6	Сыр твердый	Молочные продукты	кг	30	500,00 Р
7	Салат "Цезарь"	Салаты	шт	1	250,00 Р
	Борщ				

Добавить

Изменить

Удалить

Состав

Обновить

Рисунок 3.14 Окно продукты

Состав "Булочка сдобная"

Сырьё	Количество
Мука пшеничная	0,500 кг
Сахар	0,100 кг
Соль	0,010 кг
*	

Рисунок 3.15 Окно состав продукта

Изменение продукта

Название: Булочка сдобная

Категория: Выпечка

Ед. измерения: шт

Срок годности (дни): 2

Цена за ед.: 45,00

Состав:

Сырьё	Количество
Мука пшеничн...	0,500
Сахар	0,100
Соль	0,010

Добавить Удалить Сохранить

Рисунок 3.16 Окно изменения продукта

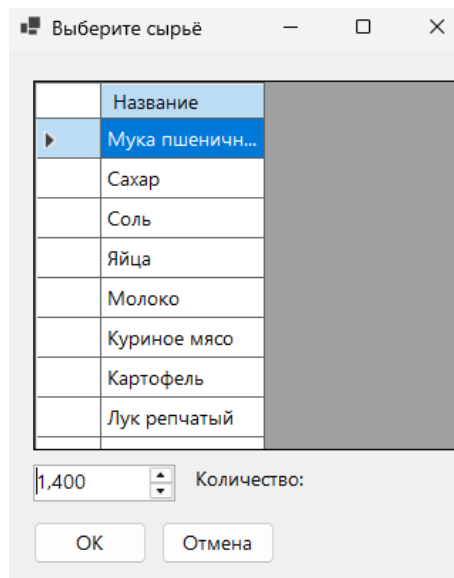


Рисунок 3.17 Окно добавления сырья в продукт

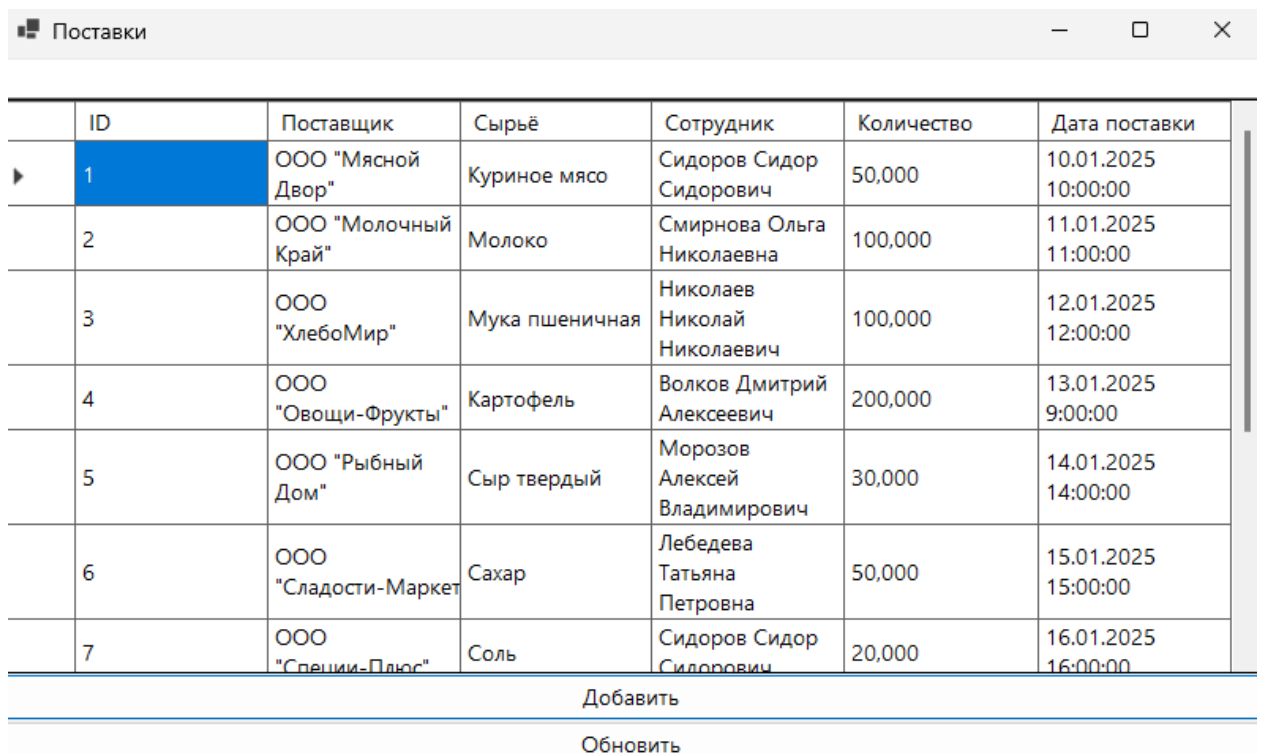


Рисунок 3.18 Окно поставки

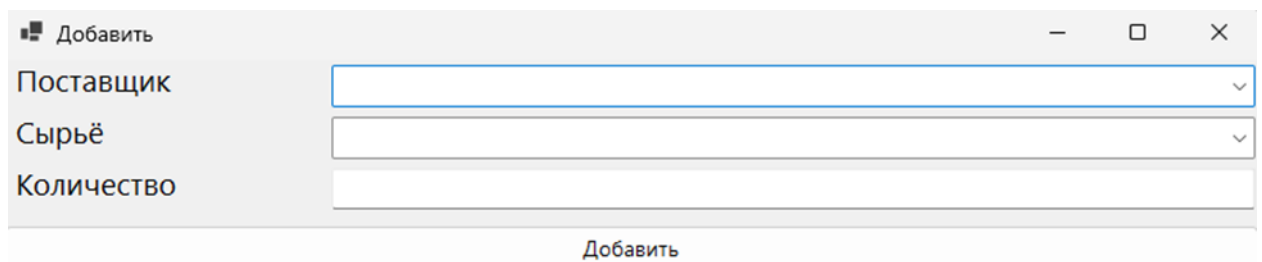


Рисунок 3.19 Окно добавления поставки

Сотрудники

	ID	ФИО	Должность	Номер телефона	Электронная	Дата найма
▶	1	Иванов Иван Иванович	Администратор	+79001234567	ivanov@example.co	15.01.2024 0:00:00
	2	Петров Петр Петрович	Менеджер	+79001234568	petrov@example.co	20.02.2024 0:00:00
	3	Сидоров Сидор Сидорович	Повар	+79001234569	sidorov@example.c	10.03.2024 0:00:00
	4	Кузнецова Анна Сергеевна	Менеджер	+79001234570	kuznetsova@examp	25.01.2024 0:00:00
	5	Волков Дмитрий Алексеевич	Повар	+79001234571	volkov@example.co	10.02.2024 0:00:00
	6	Смирнова Ольга Николаевна	Повар	+79001234572	smirnova@example	05.04.2024 0:00:00
	7	Николаев Николай	Администратор	+79001234573	nikolaev@example	01.01.2024

Добавить

Изменить

Уволить/Восстановить

Обновить

Рисунок 3.20 Окно сотрудники

Изменить

ФИО

Иванов Иван Иванович

Должность

Администратор

Номер телефона

+79001234567

Электронная почта

ivanov@example.com

Изменить

Рисунок 3.21 Окно изменение сотрудников

Поставщики

	ID	Компания	Контактное	Номер телефона	Электронная	Адрес
▶	1	ООО "Мясной Двор"	Андрей Смирнов	+79002222221	andrey@myaso.ru	г. Москва, ул. Мясная, д. 10
	2	ООО "Молочный Край"	Марина Петрова	+79002222222	milk@molochniy.ru	г. Санкт-Петербург, пр. Ленина, д. 25
	3	ООО "ХлебоМир"	Сергей Иванов	+79002222223	sergey@bread.ru	г. Новосибирск, ул. Пекарская, д. 5
	4	ООО "Овощи-Фрукты"	Елена Козлова	+79002222224	elenak@ovosh.ru	г. Казань, ул. Овощная, д. 15
	5	ООО "Рыбный Дом"	Александр Васильев	+79002222225	fish@ryba.ru	г. Ростов-на-Дону, ул. Рыбная, д. 8
						г. Екатеринбург

Добавить

Изменить

Удалить

Обновить

Рисунок 3.22 Окно поставщики

Изменить

Компания	ООО "Мясной Двор"
Контактное лицо	Андрей Смирнов
Номер телефона	+7900222221
Электронная почта	andrey@myaso.ru
Адрес	г. Москва, ул. Мясная, д. 10

Изменить

Рисунок 3.23 Окно изменения поставщиков

Авторизация

Сотрудник	Логин	Пароль	Уровень доступа
Иванов Иван Иванович	admin	admin	Админ
Петров Петр Петрович	manager	manager	Менеджер
Сидоров Сидор Сидорович	cook	cook	Повар
Кузнецова Анна Сергеевна	user1	user1	Нет доступа
Волков Дмитрий Алексеевич	volkovdmitriy	volkovdmitriy	Повар
Смирнова Ольга Николаевна	smirnovaolga	smirnovaolga	Повар
Николаев Николай Николаевич	nikolaevnikolay	nikolaevnikolay	Админ
Федорова Мария Андреевна	fedorovamaria	fedorovamaria	Менеджер
Морозов Алексей Владимирович	morozovalisey	morozovalisey	Повар
Лебедева Татьяна	lebedevatatvana	lebedevatatvana	Повар

Изменить

Обновить

Рисунок 3.24 Окно данные авторизации

Изменить

Логин	volkovdmitriy
Пароль	volkovdmitriy
Уровень доступа	Повар

Изменить

Рисунок 3.25 Окно изменение данных авторизации

Аналитика

Период:

03.12.2025

27.01.2026

Сформировать

=== Отчет по анализу ===

Период: 03.12.2025 — 27.01.2026

--- Лучшие менеджеры ---

Петров Петр Петрович: 4 заказов

Иванов Иван Иванович: 3 заказов

Федорова Мария Андреевна: 2 заказов

--- Лучшие повара ---

Сидоров Сидор Сидорович: 6 заказов

--- Лучшие админы ---

Иванов Иван Иванович: 3 заказов

--- Прибыль ---

Общая прибыль: 3 750,00 Р

--- Самые популярные товары ---

Булочка сдобная: 1062,000 шт.

Йогурт натуральный: 58,000 шт.

Пирожок с картошкой: 40,000 шт.

Салат "Цезарь": 15,000 шт.

Пельмени домашние: 6,000 шт.

Куриная грудка: 2,000 шт.

Картофель свежий: 1,000 шт.

Сыр твердый: 1,000 шт.

--- Расход сырья ---

Мука пшеничная: 543,000 ед.

Сахар: 106,200 ед.

Молоко: 58,000 ед.

Сметана: 11,600 ед.

Соль: 10,620 ед.

Рисунок 3.26 Окно аналитика

ПП - Менеджер

	ID	Клиент	Менеджер	Повар	Дата заказа	Дата	Сумма	Статус
▶	12	ООО "Гастрономия"	-	-	23.01.2026 3:26:33	-	405,00 Р	Новый
	11	ООО "Гастрономия"	-	-	23.01.2026 3:17:51	-	1 230,00 Р	Новый
	8	ООО "Десерт-Палак"	-	-	22.01.2025 11:30:00	-	750,00 Р	Новый
	13	ИП "Шеф-Повар"	Петров Петр Петрович	-	23.01.2026 23:11:11	-	1 170,00 Р	Новый
	5	ИП "Кафе Уют"	Петров Петр Петрович	Сидоров Сидор Сидорович	19.01.2025 14:00:00	-	900,00 Р	Новый
	3	ООО "Вкуснятина"	Петров Петр Петрович	Морозов Алексей Владимирович	17.01.2025 11:00:00	-	2 200,00 Р	Готово
*								

Добавить

Изменить

Обновить

Рисунок 3.27 Окно менеджера

Редактирование заказа

Заказ №12

Клиент: ООО "Гастрономия"

Менеджер: Петров Петр Петрович

Повар:

Дата создания: 23.01.2026 Дата завершения: Не установлена

Статус: Новый

	Название	Количество	Цена	Итого
▶	Булочка сдобн...	9,000	45,00 Р	405,00 Р

Общая сумма: 405,00 Р

Применить

Рисунок 3.28 Окно изменения заказа (менеджером)

ПП - Повар

	ID	Клиент	Менеджер	Повар	Дата заказа	Дата	Сумма	Статус
▶	8	ООО "Десерт-Пала"	Петров Петр Петрович	Сидоров Сидор Сидорович	22.01.2025 11:30:00	-	750,00 Р	В процессе
	11	ООО "Гастрономия"	Петров Петр Петрович	Сидоров Сидор Сидорович	23.01.2026 3:17:51	-	1 230,00 Р	В процессе
	12	ООО "Гастрономия"	Петров Петр Петрович	Сидоров Сидор Сидорович	23.01.2026 3:26:33	-	405,00 Р	В процессе

Изменить

Обновить

Рисунок 3.29 Окно повара

Редактирование заказа

Заказ №8

Дата завершения: Не установлена

Статус: В процессе

Готово

В процессе

Готово

Отменено

	Название	Количество	Цена	Итого
▶	Молоко 3.2%	5		
	Сметана 20%	3,000	180,00 Р	540,00 Р

Применить

Рисунок 3.30 Окно изменения заказа (поваром)

3.3. Проектирование структуры базы данных

База данных (БД) — это структурированное хранилище информации, организованное в электронном виде. Основное её назначение — надёжное хранение, обработка, поиск и управление значительными объёмами данных, связанных с деятельностью предприятия. Использование БД позволяет объединить информацию в единую систему, избегая разрозненного хранения в отдельных файлах или бумажных носителях, обеспечивая при этом её согласованность, доступность и целостность.

Основные цели базы данных:

- 1) Эффективное хранение: организованное и надёжное размещение больших объёмов информации, связанных с заказами, продукцией, сырьём, сотрудниками и клиентами.
- 2) Управление данными: предоставление инструментов для ввода, редактирования, удаления и поиска информации.
- 3) Контролируемый доступ: обеспечение оперативного и безопасного доступа к данным в зависимости от роли пользователя (администратор, менеджер, повар).
- 4) Целостность и защита: поддержание точности и согласованности данных, защита от несанкционированного доступа и ошибок.
- 5) Минимизация дублирования: снижение избыточности за счёт хранения информации в одном месте и использования ссылок между таблицами.

Ключевые компоненты базы данных:

- 1) Таблицы (Tables): основные структурные элементы БД, аналогичные таблицам в электронных таблицах. Каждая таблица содержит данные об определённой сущности (например, "Продукты", "Заказы", "Сырьё", "Сотрудники").
- 2) Поля (Fields / Columns): столбцы таблицы, определяющие атрибуты или характеристики сущности (например, "Название продукта", "Количество на складе", "ФИО сотрудника").

3) Записи (Records / Rows): строки таблицы, каждая из которых представляет один экземпляр сущности с конкретными значениями атрибутов (например, одна строка в таблице "Клиенты" — это данные об одном клиенте).

4) Ключи (Keys):

- Первичный ключ (Primary Key): уникальный идентификатор каждой записи в таблице (например, ID).

- Внешний ключ (Foreign Key): поле, устанавливающее связи между таблицами, ссылаясь на первичный ключ другой (или той же) таблицы (например, КлиентID в таблице "Заказы" ссылается на ID в таблице "Клиенты").

Система управления базами данных (СУБД) — это специализированное программное обеспечение, выступающее посредником между базой данных, пользователями и приложениями. СУБД обеспечивает создание, обслуживание, безопасное хранение и управление БД. Она отвечает за выполнение запросов (например, на языке SQL), поддержание целостности данных, управление доступом нескольких пользователей одновременно и администрирование базы данных.

Примеры СУБД: Microsoft SQL Server, MySQL, PostgreSQL, Oracle Database, SQLite. В рамках дипломного проекта для информационной системы предприятия пищевого производства выбрана Microsoft SQL Server, как надёжная и мощная реляционная СУБД, хорошо интегрируемая с выбранной платформой разработки (C#, Windows Forms, .NET).

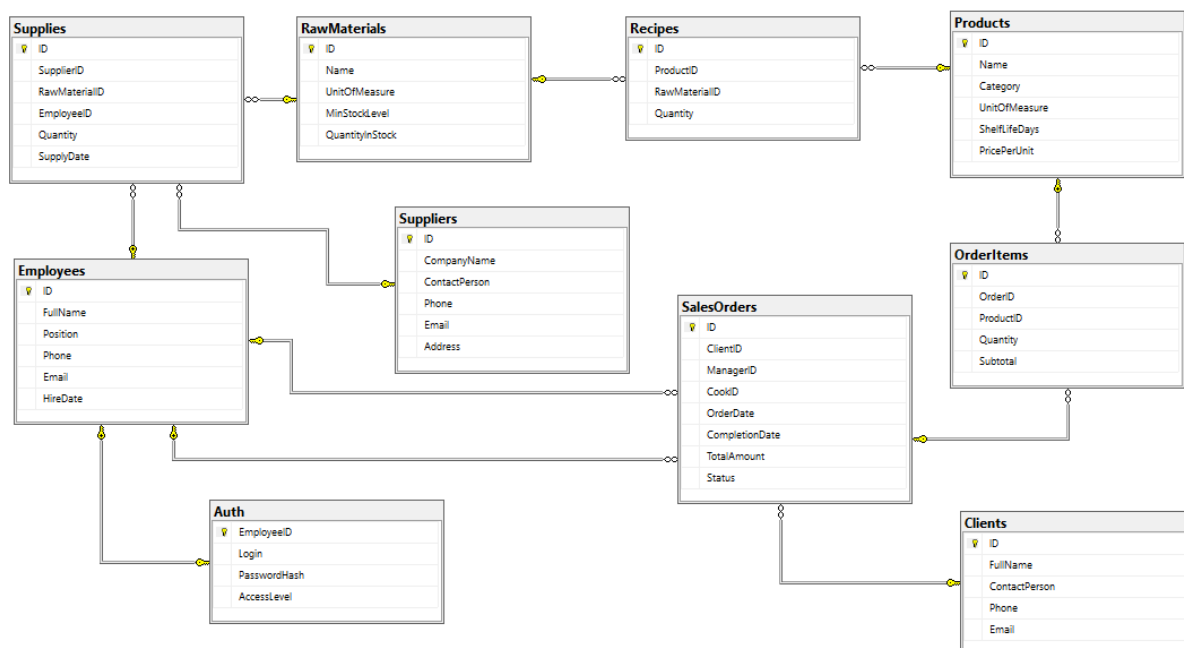


Рисунок 3.31. Физическая ER диаграмма

Описание таблиц базы данных

1) Сотрудники (Employees)

- ID — уникальный идентификатор сотрудника (INT, IDENTITY(1,1), PRIMARY KEY, NOT NULL)
- FullName — полное имя сотрудника (NVARCHAR(255), NOT NULL)
- Position — должность сотрудника (NVARCHAR(100), NOT NULL)
- Phone — контактный телефон (NVARCHAR(50), NULLABLE)
- Email — электронная почта (NVARCHAR(255), NULLABLE)
- HireDate — дата приёма на работу (DATE, NULLABLE)

	Имя столбца	Тип данных	Разрешить знач...
🔑	ID	int	☐
	FullName	nvarchar(255)	☐
	Position	nvarchar(100)	☐
	Phone	nvarchar(50)	☒
	Email	nvarchar(255)	☒
	HireDate	date	☒

Рисунок 3.32 Таблица сотрудники

	ID	FullName	Position	Phone	Email	HireDate
1	1	Иванов Иван Иванович	Администратор	+79001234567	ivanov@example.com	2024-01-15
2	2	Петров Петр Петрович	Менеджер	+79001234568	petrov@example.com	2024-02-20
3	3	Сидоров Сидор Сидорович	Повар	+79001234569	sidorov@example.com	2024-03-10
4	4	Кузнецова Анна Сергеевна	Менеджер	+79001234570	kuznetsova@example.com	2024-01-25
5	5	Волков Дмитрий Алексеевич	Повар	+79001234571	volkov@example.com	2024-02-10
6	6	Смирнова Ольга Николаевна	Повар	+79001234572	smimova@example.com	2024-04-05
7	7	Николаев Николай Николаевич	Администратор	+79001234573	nikolaev@example.com	2024-01-01
8	8	Федорова Мария Андреевна	Менеджер	+79001234574	fedorova@example.com	2024-03-15
9	9	Морозов Алексей Владимирович	Повар	+79001234575	morozov@example.com	2024-04-20
10	10	Лебедева Татьяна Петровна	Повар	+79001234576	lebedeva@example.com	2026-01-23
11	13	1	1	1	1	2026-01-23

Рисунок 3.33 Данные в таблице сотрудники

2) Клиенты (Clients)

- ID — уникальный идентификатор клиента (INT, IDENTITY(1,1), PRIMARY KEY, NOT NULL)
- FullName — полное имя клиента (NVARCHAR(255), NOT NULL)
- ContactPerson — контактное лицо (NVARCHAR(255), NULLABLE)
- Phone — контактный телефон (NVARCHAR(50), NULLABLE)
- Email — электронная почта (NVARCHAR(255), NULLABLE)

	Имя столбца	Тип данных	Разрешить знач...
	ID	int	☐
	FullName	nvarchar(255)	☐
	ContactPerson	nvarchar(255)	☒
	Phone	nvarchar(50)	☒
	Email	nvarchar(255)	☒

Рисунок 3.34 Таблица клиенты

	ID	FullName	ContactPerson	Phone	Email
1	1	ООО "Гастрономия"	Иван Петров	+79001111111	contact@gastro.ru
2	2	ИП "Шеф-Повар"	Мария Соколова	+79001111112	maria@chef.ru
3	3	ООО "Вкуснятина"	Алексей Козлов	+79001111113	info@vkus.ru
4	4	ООО "Сытый Кот"	Елена Новикова	+79001111114	elena@sitykot.ru
5	5	ИП "Кафе Уют"	Дмитрий Белов	+79001111115	belov@cafeuyt.ru
6	6	ООО "Булочная"	Ольга Медведева	+79001111116	medvedeva@bulka.ru
7	7	ИП "Суши-Бар"	Сергей Жуков	+79001111117	zhukov@sushi-bar.ru
8	8	ООО "Десерт-Палас"	Анна Лапина	+79001111118	lapina@desert-palace.ru
9	9	ООО "Бургер-Мастер"	Виктор Семенов	+79001111119	vsemenov@burger.ru
10	10	ИП "Пицца-Дом"	Татьяна Власова	+79001111120	tanya@pizza.ru
11	13	new	new	new	new@email.com

Рисунок 3.35 Данные в таблице клиенты

3) Поставщики (Suppliers)

- ID — уникальный идентификатор поставщика (INT, IDENTITY(1,1), PRIMARY KEY, NOT NULL)
- CompanyName — название компании (NVARCHAR(255), NOT NULL)
- ContactPerson — контактное лицо (NVARCHAR(255), NULLABLE)
- Phone — контактный телефон (NVARCHAR(50), NULLABLE)
- Email — электронная почта (NVARCHAR(255), NULLABLE)
- Address — юридический или фактический адрес (NVARCHAR(500), NULLABLE)

	Имя столбца	Тип данных	Разрешить знач...
	ID	int	☐
	CompanyName	nvarchar(255)	☐
	ContactPerson	nvarchar(255)	☒
	Phone	nvarchar(50)	☒
	Email	nvarchar(255)	☒
	Address	nvarchar(500)	☒

Рисунок 3.36 Таблица поставщики

	ID	CompanyName	ContactPerson	Phone	Email	Address
1	1	ООО "Мясной Двор"	Андрей Смирнов	+79002222221	andrey@myaso.ru	г. Москва, ул. Мясная, д. 10
2	2	ООО "Молочный Край"	Марина Петрова	+79002222222	milk@molochniy.ru	г. Санкт-Петербург, пр. Ленина, д. 25
3	3	ООО "ХлебоМир"	Сергей Иванов	+79002222223	sergey@bread.ru	г. Новосибирск, ул. Пекарская, д. 5
4	4	ООО "Овощи-Фрукты"	Елена Козлова	+79002222224	elenak@ovosh.ru	г. Казань, ул. Овощная, д. 15
5	5	ООО "Рыбный Дом"	Александр Васильев	+79002222225	fish@ryba.ru	г. Ростов-на-Дону, ул. Рыбная, д. 8
6	6	ООО "Сладости-Маркет"	Наталья Соколова	+79002222226	sladosti@market.ru	г. Екатеринбург, ул. Кондитерская, д. 12
7	7	ООО "Специи-Плюс"	Виктория Морозова	+79002222227	spice@specii.ru	г. Нижний Новгород, ул. Специальная, д. 3
8	8	ООО "Зелень-Грин"	Дмитрий Новиков	+79002222228	green@zel.ru	г. Самара, ул. Зеленая, д. 20
9	9	ООО "Консервы-Лайн"	Ольга Петрова	+79002222229	conserv@line.ru	г. Уфа, ул. Консервная, д. 7
10	10	ООО "Масло-Мир"	Игорь Степанов	+79002222230	igor@maslo.ru	г. Краснодар, ул. Масляная, д. 9

Рисунок 3.37 Данные в таблице поставщики

4) Продукты (Products)

- ID — уникальный идентификатор продукта (INT, IDENTITY(1,1), PRIMARY KEY, NOT NULL)
- Name — наименование продукта (NVARCHAR(255), NOT NULL)
- Category — категория продукта (NVARCHAR(100), NOT NULL)
- UnitOfMeasure — единица измерения (NVARCHAR(50), NOT NULL)

- ShelfLifeDays — срок годности в днях (INT, NOT NULL, CHECK >= 0)
- PricePerUnit — цена за единицу (DECIMAL(10,2), NOT NULL, CHECK >= 0)

	Имя столбца	Тип данных	Разрешить знач...
	ID	int	☐
	Name	nvarchar(255)	☐
	Category	nvarchar(100)	☐
	UnitOfMeasure	nvarchar(50)	☐
	ShelfLifeDays	int	☐
	PricePerUnit	decimal(10, 2)	☐

Рисунок 3.38 Таблица продукты

	ID	Name	Category	UnitOfMeasure	ShelfLifeDays	PricePerUnit
1	1	Булочка сдобная	Выпечка	шт	2	45.00
2	2	Йогурт натуральный	Молочные продукты	шт	7	80.00
3	3	Пирожок с картошкой	Выпечка	шт	1	60.00
4	4	Куриная грудка	Мясо	кг	5	450.00
5	5	Картофель свежий	Овощи	кг	14	40.00
6	6	Сыр твердый	Молочные продукты	кг	30	500.00
7	7	Салат "Цезарь"	Салаты	шт	1	250.00
8	8	Борщ украинский	Супы	л	2	120.00
9	9	Пельмени домашние	Замороженные	кг	90	300.00
10	10	Шоколад темный	Сладости	шт	180	150.00
11	11	Сок апельсиновый	Напитки	л	30	100.00
12	12	Капуста белокочанная	Овощи	кг	21	30.00
13	13	Морковь свежая	Овощи	кг	28	25.00
14	14	Молоко 3.2%	Молочные продукты	л	5	80.00
15	15	Сметана 20%	Молочные продукты	кг	10	180.00

Рисунок 3.39 Данные в таблице продукты

5) Сырьё (RawMaterials)

- ID — уникальный идентификатор сырья (INT, IDENTITY(1,1), PRIMARY KEY, NOT NULL)
- Name — наименование сырья (NVARCHAR(255), NOT NULL)
- UnitOfMeasure — единица измерения (NVARCHAR(50), NOT NULL)
- MinStockLevel — минимальный уровень остатка (DECIMAL(10,3), NOT NULL, CHECK >= 0)

- QuantityInStock — количество на складе (DECIMAL(10,3), NOT NULL, CHECK >= 0)

	Имя столбца	Тип данных	Разрешить знач...
	ID	int	☐
	Name	nvarchar(255)	☐
	UnitOfMeasure	nvarchar(50)	☐
	MinStockLevel	decimal(10, 3)	☐
	QuantityInStock	decimal(10, 3)	☐

Рисунок 3.40 Таблица сырьё

	ID	Name	UnitOfMeasure	MinStockLevel	QuantityInStock
1	1	Мука пшеничная	кг	50.000	82.200
2	2	Сахар	кг	30.000	75.700
3	3	Соль	кг	20.000	59.570
4	4	Яйца	шт	200.000	500.000
5	5	Молоко	л	50.000	61.000
6	6	Куриное мясо	кг	30.000	63.800
7	7	Картофель	кг	100.000	200.000
8	8	Лук репчатый	кг	40.000	90.000
9	9	Морковь	кг	40.000	100.000
10	10	Сыр твердый	кг	20.000	40.000
11	11	Сливочное масло	кг	10.000	25.000
12	12	Сметана	кг	15.000	27.200
13	13	Капуста белокочанная	кг	50.000	120.000
14	14	Соус томатный	л	10.000	25.000
15	15	Чеснок	кг	5.000	20.000

Рисунок 3.41 Данные в таблице сырьё

6) Заказы (SalesOrders)

- ID — уникальный идентификатор заказа (INT, IDENTITY(1,1), PRIMARY KEY, NOT NULL)
- ClientID — внешний ключ на клиента (INT, NULLABLE, FK на Clients.ID, ON DELETE SET NULL)
- ManagerID — внешний ключ на менеджера (INT, NULLABLE, FK на Employees.ID, ON DELETE NO ACTION)
- CookID — внешний ключ на повара (INT, NULLABLE, FK на Employees.ID, ON DELETE NO ACTION)
- OrderDate — дата создания заказа (DATETIME2, NOT NULL, DEFAULT GETDATE())

- CompletionDate — дата завершения (DATETIME2, NULLABLE)
- TotalAmount — общая сумма заказа (DECIMAL(12,2), NOT NULL, CHECK >= 0)
- Status — статус заказа (NVARCHAR(50), NOT NULL, DEFAULT 'Создан')

	Имя столбца	Тип данных	Разрешить знач...
	ID	int	☐
	ClientID	int	☒
	ManagerID	int	☒
	CookID	int	☒
	OrderDate	datetime2(7)	☐
	CompletionDate	datetime2(7)	☒
	TotalAmount	decimal(12, 2)	☐
	Status	nvarchar(50)	☐

Рисунок 3.42 Таблица заказы

	ID	ClientID	ManagerID	CookID	OrderDate	CompletionDate	TotalAmount	Status
1	1	1	2	3	2025-01-15 10:00:00.0000000	2025-01-15 12:00:00.0000000	1500.00	Завершено
2	2	2	4	6	2025-01-16 09:30:00.0000000	2025-01-16 11:00:00.0000000	850.00	Завершено
3	3	3	2	9	2025-01-17 11:00:00.0000000	NULL	2200.00	Готово
4	4	4	8	7	2025-01-18 13:00:00.0000000	2025-01-18 14:30:00.0000000	1200.00	Завершено
5	5	5	2	3	2025-01-19 14:00:00.0000000	NULL	900.00	Новый
6	6	6	4	9	2025-01-20 10:00:00.0000000	NULL	3200.00	В процессе
7	7	7	8	10	2025-01-21 12:00:00.0000000	2025-01-21 13:00:00.0000000	1100.00	Завершено
8	8	8	2	3	2025-01-22 11:30:00.0000000	NULL	750.00	В процессе
9	9	9	4	6	2025-01-23 09:00:00.0000000	NULL	1800.00	В процессе
10	10	10	8	7	2025-01-24 15:00:00.0000000	2025-01-24 16:00:00.0000000	2000.00	Завершено
11	11	1	2	3	2026-01-23 03:17:51.2333333	NULL	1230.00	В процессе
12	12	1	2	3	2026-01-23 03:26:33.2800000	NULL	405.00	В процессе
13	13	2	2	NULL	2026-01-23 23:11:11.9133333	NULL	1170.00	Новый
14	14	1	1	3	2026-01-24 02:06:12.5733333	2026-01-24 02:22:20.8400000	45.00	Завершено
15	15	1	1	3	2026-01-24 02:26:13.4000000	2026-01-24 02:29:27.7000000	45.00	Отменено
16	16	1	2	3	2026-01-24 02:30:14.4700000	2026-01-24 02:31:23.7700000	45000.00	Отменено
17	17	3	2	3	2026-01-24 02:35:23.0133333	2026-01-24 02:37:36.3900000	2820.00	Завершено
18	18	13	2	3	2026-01-24 02:58:04.1166667	2026-01-24 02:59:20.5733333	885.00	Завершено
19	19	13	1	3	2026-01-24 15:05:49.1166667	NULL	1530.00	Готово
20	20	13	8	NULL	2026-01-26 10:58:57.1566667	NULL	4980.00	Новый
21	21	13	8	NULL	2026-01-27 15:53:04.5100000	NULL	3710.00	Новый

Рисунок 3.43 Данные в таблице заказы

7) Позиции заказа (OrderItems)

- ID — уникальный идентификатор позиции (INT, IDENTITY(1,1), PRIMARY KEY, NOT NULL)

- OrderID — внешний ключ на заказ (INT, NOT NULL, FK на SalesOrders.ID, ON DELETE CASCADE)
- ProductID — внешний ключ на продукт (INT, NOT NULL, FK на Products.ID, ON DELETE NO ACTION)
- Quantity — количество единиц продукта (DECIMAL(12,3), NOT NULL, CHECK > 0)
- Subtotal — промежуточная сумма (DECIMAL(15,2), NOT NULL)

	Имя столбца	Тип данных	Разрешить знач...
	ID	int	☐
	OrderID	int	☐
	ProductID	int	☐
	Quantity	decimal(12, 3)	☐
	Subtotal	decimal(15, 2)	☐

Рисунок 3.44 Таблица позиции заказа

	ID	OrderID	ProductID	Quantity	Subtotal
1	1	1	1	10.000	450.00
2	2	1	2	5.000	400.00
3	3	2	3	8.000	480.00
4	4	2	4	1.000	450.00
5	5	3	5	20.000	800.00
6	6	3	6	2.000	1000.00
7	7	4	7	3.000	750.00
8	8	4	8	5.000	600.00
9	9	5	9	2.000	600.00
10	10	6	10	10.000	1500.00
11	11	6	11	15.000	1500.00
12	12	7	12	10.000	300.00
13	13	7	13	10.000	250.00
14	14	8	14	5.000	400.00
15	15	8	15	3.000	540.00
16	16	9	1	15.000	675.00
17	17	9	3	10.000	600.00
18	18	10	4	3.000	1350.00
19	19	10	5	10.000	400.00
20	20	10	6	1.000	500.00
21	21	11	1	6.000	270.00
22	22	11	2	9.000	720.00
23	23	11	3	4.000	240.00
24	24	12	1	9.000	405.00
25	25	13	1	6.000	270.00
26	26	13	2	6.000	480.00
27	27	13	3	7.000	420.00
28	28	14	1	1.000	45.00
29	29	15	1	1.000	45.00
30	30	16	1	1000....	45000...
31	31	17	1	12.000	540.00
32	32	17	2	15.000	1200.00
33	33	17	3	18.000	1080.00
34	34	18	1	9.000	405.00
35	35	18	2	6.000	480.00

Рисунок 3.45 Данные в таблице позиции заказа

8) Рецептуры (Recipes)

ID — уникальный идентификатор рецепта (INT, IDENTITY(1,1), PRIMARY KEY, NOT NULL)

ProductID — внешний ключ на продукт (INT, NOT NULL, FK на Products.ID, ON DELETE CASCADE)

RawMaterialID — внешний ключ на сырьё (INT, NOT NULL, FK на RawMaterials.ID)

Quantity — количество сырья, необходимое для производства (DECIMAL(10,3), NOT NULL, CHECK > 0)

	Имя столбца	Тип данных	Разрешить знач...
🔑	ID	int	☐
	ProductID	int	☐
	RawMaterialID	int	☐
	Quantity	decimal(10, 3)	☐

Рисунок 3.46 Таблица рецептура

	ID	ProductID	RawMaterialID	Quantity
1	1	1	1	0.500
2	2	1	2	0.100
3	3	1	3	0.010
4	4	2	5	1.000
5	5	2	12	0.200
6	6	3	1	0.300
7	7	3	6	0.200
8	8	4	6	1.000
9	9	5	7	1.000
10	10	6	10	0.500

Рисунок 3.47 Данные в таблице рецептура

9) Поставки (Supplies)

- ID — уникальный идентификатор поставки (INT, IDENTITY(1,1), PRIMARY KEY, NOT NULL)

- SupplierID — внешний ключ на поставщика (INT, NULLABLE, FK на Suppliers.ID, ON DELETE SET NULL)

- RawMaterialID — внешний ключ на сырьё (INT, NOT NULL, FK на RawMaterials.ID)

- EmployeeID — внешний ключ на сотрудника (INT, NULLABLE, FK на Employees.ID, ON DELETE SET NULL)
- Quantity — количество поставленного сырья (DECIMAL(12,3), NOT NULL, CHECK > 0)
- SupplyDate — дата поставки (DATETIME2, NOT NULL, DEFAULT GETDATE())

	Имя столбца	Тип данных	Разрешить знач...
🔑	ID	int	☐
	SupplierID	int	☒
	RawMaterialID	int	☐
	EmployeeID	int	☒
	Quantity	decimal(12, 3)	☐
	SupplyDate	datetime2(7)	☐

Рисунок 3.48 Таблица поставки

	ID	SupplierID	RawMaterialID	EmployeeID	Quantity	SupplyDate
1	1	1	6	3	50.000	2025-01-10 10:00:00.0000000
2	2	2	5	6	100.000	2025-01-11 11:00:00.0000000
3	3	3	1	7	100.000	2025-01-12 12:00:00.0000000
4	4	4	7	5	200.000	2025-01-13 09:00:00.0000000
5	5	5	10	9	30.000	2025-01-14 14:00:00.0000000
6	6	6	2	10	50.000	2025-01-15 15:00:00.0000000
7	7	7	3	3	20.000	2025-01-16 16:00:00.0000000
8	8	8	9	6	100.000	2025-01-17 10:00:00.0000000
9	9	9	11	7	10.000	2025-01-18 11:00:00.0000000
10	10	10	12	9	20.000	2025-01-19 12:00:00.0000000
11	12	3	1	1	13.000	2026-01-23 17:39:27.6700000

Рисунок 3.49 Данные в таблице поставки

10) Авторизация (Auth)

- EmployeeID — уникальный идентификатор сотрудника (INT, PRIMARY KEY, FK на Employees.ID, ON DELETE CASCADE)
- Login — логин пользователя (NVARCHAR(100), NOT NULL, UNIQUE)
- PasswordHash — хэш пароля (NVARCHAR(255), NOT NULL)
- AccessLevel — уровень доступа (-1: заблокирован, 0: админ, 1: менеджер, 2: повар) (INT, NOT NULL, CHECK IN (-1, 0, 1, 2))

	Имя столбца	Тип данных	Разрешить знач...
🔑	EmployeeID	int	☐
	Login	nvarchar(100)	☐
	PasswordHash	nvarchar(255)	☐
	AccessLevel	int	☐

Рисунок 3.50 Таблица авторизация

	EmployeeID	Login	PasswordHash	AccessLevel
1	1	admin	admin	0
2	2	manager	manager	1
3	3	cook	cook	2
4	4	user1	user1	-1
5	5	volkovdmitriy	volkovdmitriy	2
6	6	smimovaolga	smimovaolga	2
7	7	nikolaevnikolay	nikolaevnikolay	0
8	8	fedorovamaria	fedorovamaria	1
9	9	morozovalisey	morozovalisey	2
10	10	lebedevatyana	lebedevatyana	2
11	13	admin2	admin2	0

Рисунок 3.51 Данные в таблице авторизация

Связи между таблицами базы данных:

1) Таблица Сотрудники связана с таблицей Заказы посредством внешних ключей ManagerID и CookID, что означает, что один сотрудник может быть менеджером или поваром в нескольких заказах.

2) Таблица Сотрудники связана с таблицей Поставки через внешний ключ EmployeeID, что позволяет одному сотруднику оформлять несколько поставок.

3) Таблица Сотрудники связана с таблицей Авторизация посредством внешнего ключа EmployeeID, образуя связь один-к-одному: каждый сотрудник имеет только одну учётную запись.

4) Таблица Клиенты связана с таблицей Заказы через внешний ключ ClientID, что позволяет одному клиенту создавать несколько заказов.

5) Таблица Продукты связана с таблицей Позиции заказа через внешний ключ ProductID, что означает, что один продукт может быть включён в несколько заказов.

6) Таблица Продукты связана с таблицей Рецептуры через внешний ключ ProductID, позволяя одному продукту состоять из нескольких видов сырья.

7) Таблица Сырьё связана с таблицей Рецептуры через внешний ключ RawMaterialID, что позволяет одному виду сырья использоваться в рецептах нескольких продуктов.

8) Таблица Сырьё связана с таблицей Поставки через внешний ключ RawMaterialID, что означает, что одно сырьё может поступать в нескольких поставках.

9) Таблица Поставщики связана с таблицей Поставки через внешний ключ SupplierID, позволяя одному поставщику осуществлять несколько поставок.

10) Таблица Заказы связана с таблицей Позиции заказа через внешний ключ OrderID, что позволяет одному заказу содержать несколько позиций.

3.4. Отладка и тестирование программы

Этап отладки и тестирования является неотъемлемой частью процесса разработки программного обеспечения. Его цель — выявление и устранение ошибок, проверка корректности работы всех модулей приложения, соответствие реализации сформированным требованиям, а также обеспечение стабильности, безопасности и удобства использования системы. В рамках дипломного проекта, посвящённого разработке информационной системы для предприятия пищевого производства, были проведены многоуровневые проверки приложения: от тестирования отдельных компонентов до комплексной проверки взаимодействия с базой данных и проверки бизнес-логики.

Цели тестирования

- Проверка функциональности: убедиться, что все функции системы работают в соответствии с требованиями.
- Проверка надёжности: убедиться, что система стабильно работает при различных сценариях использования и не теряет данные.

- Проверка безопасности: убедиться, что разграничение прав доступа работает корректно, и пользователи не могут получить доступ к запрещённой информации.

- Проверка пользовательского интерфейса: убедиться, что интерфейс интуитивно понятен, удобен и не вызывает трудностей при работе.

- Проверка производительности: убедиться, что система быстро отвечает на запросы и не тормозит при обычной нагрузке.

Виды тестирования

1) Модульное тестирование (Unit Testing): хотя в рамках дипломного проекта не использовались автоматизированные unit-тесты, каждый компонент (окно, форма, метод) разрабатывался и проверялся отдельно на корректность работы. Например, проверялось, что при нажатии кнопки «Добавить» в форме «Клиенты» новая запись действительно появляется в базе данных и отображается в таблице.

2) Интеграционное тестирование: проверялось взаимодействие между различными модулями системы. Например, как изменение статуса заказа в интерфейсе влияет на данные в базе, как при изменении статуса на «В процессе» происходит автоматическое списание сырья.

3) Функциональное тестирование: проверялась корректность выполнения всех заявленных функций системы. Это включало:

- Регистрацию и авторизацию пользователей.
- Создание, редактирование и удаление записей (клиенты, продукты, сотрудники и т.д.).
- Работу с заказами: создание, изменение статуса, назначение повара.
- Формирование аналитических отчётов.
- Проверку статуса заказа клиентом.
- Списание сырья при начале производства.

4) Тестирование на соответствие бизнес-правилам: Особое внимание уделялось проверке соблюдения бизнес-логики. Например:

- Запрет на изменение заказа после его завершения.

- Проверка наличия достаточного количества сырья на складе перед переводом заказа в статус «В процессе».

- Разграничение прав доступа: менеджер не может удалять сотрудников, повар не может назначать других поваров.

5) Тестирование безопасности: проверялось, что пользователи с разными уровнями доступа имеют доступ только к разрешённым им функциям. Также проверялась защита от попыток прямого доступа к данным, которые не должны быть доступны конкретному пользователю.

6) Тестирование пользовательского интерфейса: интерфейс проверялся на соответствие макетам, удобство навигации, корректность отображения данных, валидацию ввода и понятность сообщений об ошибках.

Инструменты и методы отладки

- Встроенный отладчик Visual Studio: использовался для пошагового выполнения кода, анализа значений переменных, поиска и устранения логических ошибок.

- SQL Server Management Studio (SSMS): использовался для проверки корректности выполнения запросов к базе данных, анализа состояния таблиц, проверки триггеров и ограничений.

- Ручное тестирование: основной метод проверки функций, особенно интерфейса и бизнес-логики. Проводилось сценариями, имитирующими реальные действия пользователей.

- Проверка на разных уровнях доступа: тестирование проводилось под учётными записями администратора, менеджера и повара, чтобы убедиться в корректности разграничения прав.

В процессе тестирования были выявлены и устранены следующие типы ошибок:

- Ошибки валидации: например, при вводе отрицательного количества товара или неправильного формата email. Устранены путём добавления проверок на уровне интерфейса и базы данных.

- Ошибки доступа: например, возможность менеджером изменить данные, недоступные ему по правам. Устранены путём доработки логики интерфейса и проверки уровня доступа перед выполнением действия.

- Ошибки в бизнес-логике: например, списание сырья при неверном статусе заказа. Устранены путём добавления дополнительных проверок перед выполнением операций.

- Ошибки синхронизации данных: например, обновление таблицы после изменения данных. Устранены путём добавления обновления данных в интерфейсе после выполнения операций.

В результате проведённых мероприятий по отладке и тестированию было подтверждено, что разработанное программное обеспечение корректно выполняет все основные функции, соответствует сформированным требованиям, соблюдает бизнес-правила и обеспечивает безопасность данных. Система показала стабильную работу при стандартных сценариях использования, а также адекватную реакцию на некорректные действия пользователя. Это позволяет утверждать, что программа готова к практическому применению на предприятии пищевого производства.

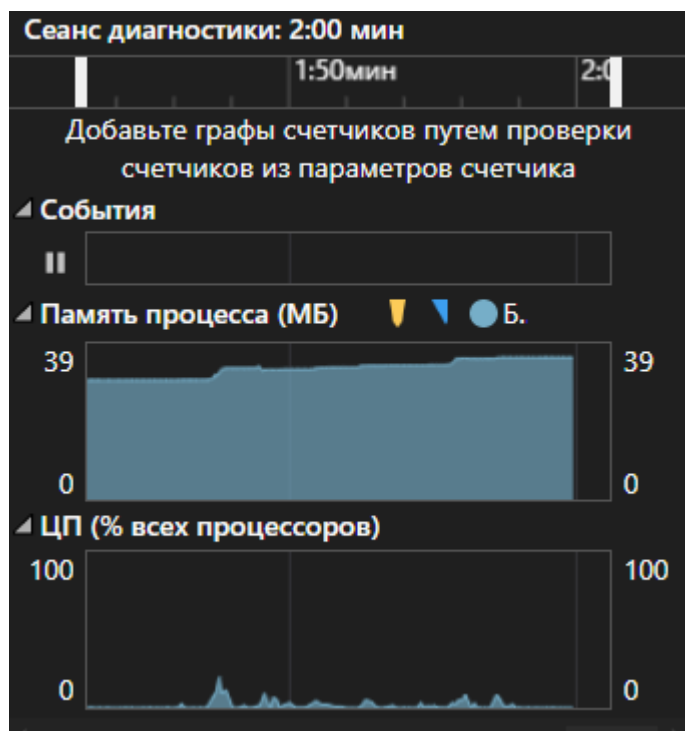


Рисунок 3.52 Диагностика работы программы

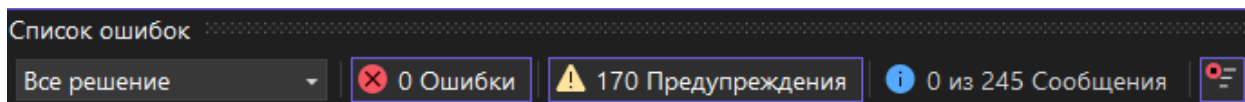


Рисунок 3.53 Список ошибок

3.5. Руководство пользователя

Настоящее руководство пользователя предназначено для ознакомления с основными возможностями, функциями и порядком работы с информационной системой предприятия пищевого производства. Оно поможет как новым, так и опытным пользователям эффективно использовать систему для выполнения своих задач, будь то управление заказами, учёт сырья, взаимодействие с клиентами или формирование отчётов.

Информационная система представляет собой десктопное приложение, разработанное для работы под операционной системой Windows. При запуске программы открывается главное окно авторизации, через которое пользователь может войти в систему или получить ограниченный доступ как клиент.

1) Авторизация и вход в систему

1.1) При запуске приложения откроется окно авторизации.

1.2) Если вы внешний пользователь (клиент):

- Нажмите кнопку «Я клиент».
- Выберите действие: «Проверить заказ» (введите Email и номер заказа) или «Каталог» (для просмотра продукции).

1.3) Если вы внутренний пользователь (сотрудник):

- Нажмите кнопку «Я сотрудник».
- Введите логин и пароль, предоставленные администратором.
- Нажмите «Войти». В зависимости от вашего уровня доступа (администратор, менеджер, повар) откроется соответствующий интерфейс.

2) Работа администратора

После входа администратору открывается «Панель управления», откуда доступны все модули системы:

- Клиенты: просмотр, добавление, редактирование и удаление записей о клиентах.
- Сырьё: учёт и управление остатками сырья, добавление новых позиций.
- Заказы: полное управление заказами, включая статусы и назначение исполнителей.
- Продукты: добавление, изменение и удаление продукции, редактирование рецептур.
- Поставки: фиксация поступлений сырья от поставщиков.
- Сотрудники: управление персоналом, добавление новых, редактирование данных, увольнение.
- Поставщики: учёт и управление поставщиками.
- Авторизация: настройка учётных записей и уровней доступа сотрудников.
- Аналитика: формирование отчётов по различным показателям за выбранный период.

3) Работа менеджера

После входа менеджеру открывается «Окно менеджера», где отображаются заказы, назначенные на него или не имеющие менеджера:

- Используйте поле «Поиск» для фильтрации заказов.
- Нажмите «Изменить», чтобы открыть редактор заказа.
- В редакторе можно изменить статус заказа и назначить повара.
- Нажмите «Применить», чтобы сохранить изменения.
- Нажмите «Обновить», чтобы обновить список заказов.

4) Работа повара

После входа повару открывается «Окно повара», где отображаются заказы, назначенные на него:

- Используйте поле «Поиск» для фильтрации заказов.
- Нажмите «Изменить», чтобы открыть редактор заказа.

- В редакторе можно изменить статус заказа только с «В процессе» на «Готово».

- Нажмите «Применить», чтобы сохранить изменения.
- Нажмите «Обновить», чтобы обновить список заказов.

5) Взаимодействие клиента

- Клиент может проверить статус своего заказа, нажав «Я клиент», введя Email и номер заказа, и нажав «Проверить заказ».

- Клиент может ознакомиться с ассортиментом, нажав «Каталог».

- Для оформления заказа клиент может нажать «Заказать», заполнить контактные данные и выбрать товары.

6) Дополнительные рекомендации

- Всегда выходите из системы, покидая рабочее место, особенно если вы сотрудник.

- При возникновении ошибок или вопросов обращайтесь к администратору системы.

- Регулярно обновляйте данные в системе, чтобы поддерживать их актуальность.

ЗАКЛЮЧЕНИЕ

В рамках дипломного проекта была успешно реализована комплексная информационная система, предназначенная для автоматизации и оптимизации деятельности предприятия пищевого производства. Работа охватывала все ключевые этапы разработки программного обеспечения: от анализа предметной области до тестирования и подготовки руководства пользователя. В процессе выполнения проекта был проведен всесторонний анализ бизнес-процессов предприятия, что позволил выявить основные проблемы в управлении заказами, учёте сырья, взаимодействии с клиентами и сотрудниками. На основе этого анализа были сформированы функциональные и нефункциональные требования к системе, которые легли в основу всей дальнейшей разработки.

Особое внимание было уделено проектированию архитектуры системы. Были разработаны различные модели данных и процессов: ER-диаграмма, отражающая структуру базы данных; контекстная диаграмма, описывающая взаимодействие предприятия с внешней средой; диаграмма декомпозиции IDEF0, детализирующая внутренние бизнес-процессы; диаграмма потоков данных, показывающая движение информации; и диаграмма вариантов использования, определяющая функции системы для различных ролей пользователей. Эти модели стали прочным фундаментом для создания структурированной и логически согласованной системы.

В ходе проектирования интерфейса были разработаны макеты, отражающие потребности и права пользователей разных ролей. Это позволило создать удобный и интуитивно понятный интерфейс, адаптированный под конкретные задачи администратора, менеджера и повара. Особое внимание уделялось обеспечению удобства и безопасности взаимодействия клиента с системой, что особенно важно для внешнего взаимодействия.

База данных, реализованная в Microsoft SQL Server, обеспечивает надёжное хранение, целостность и защиту информации. Строгое соблюдение связей между таблицами, использование ограничений и проверок гарантирует

корректную работу системы и предотвращает ошибки, связанные с нарушением бизнес-логики. Реализация разграничения прав доступа на уровне приложения и базы данных обеспечивает безопасность данных и предотвращает несанкционированные действия.

Разработанное программное обеспечение прошло этапы отладки и тестирования, в ходе которых были выявлены и устранены возможные ошибки, проверена корректность работы всех модулей и подтверждено соответствие системы сформированным требованиям. Система показала стабильность, надёжность и эффективность в работе, что подтверждает её пригодность для практического применения.

Таким образом, выполненная работа представляет собой завершённый программный продукт, который может быть внедрён на предприятии пищевого производства для автоматизации ключевых процессов. Он не только повышает эффективность управления, но и снижает вероятность ошибок, упрощает взаимодействие между различными отделами и клиентами, а также обеспечивает прозрачность и контроль над деятельностью предприятия. Проект стал важным этапом в освоении навыков системного анализа, проектирования, разработки и тестирования программного обеспечения, подтвердив теоретические знания на практике и открыв новые горизонты для дальнейшего профессионального роста.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Справочная и нормативно-правовая литература:

1. ГОСТ Р 54837-2011. Системы обработки информации. Каталоги электронные. Общие требования. — Введ. 2012-07-01. — М.: Стандартинформ, 2011. — 17 с.
2. ГОСТ Р 51907-2020. Информационные технологии. Программные средства. Общие требования к функциональным характеристикам. — Введ. 2021-07-01. — М.: Стандартинформ, 2020. — 20 с.
3. ГОСТ Р 57580.2-2017. Защита информации. Требования к средствам управления ключами. Часть 2. — Введ. 2018-07-01. — М.: Стандартинформ, 2017. — 36 с.
4. ГОСТ Р 50597-2017. Информационная технология. Системы компьютерные. Общие технические условия. — Введ. 2018-07-01. — М.: Стандартинформ, 2017. — 45 с.
5. ГОСТ Р 57580.1-2017. Защита информации. Требования к средствам управления ключами. Часть 1. — Введ. 2018-07-01. — М.: Стандартинформ, 2017. — 30 с.
6. ГОСТ Р 56939-2016. Информационные технологии. Базы данных. Общие требования. — Введ. 2017-07-01. — М.: Стандартинформ, 2016. — 24 с.

Основная литература:

1. Назаров, С.В. Программирование на C# и .NET: от основ до практики / Сергей Викторович Назаров. — М.: ДМК Пресс, 2023. — 672 с. — ISBN 978-5-97061-228-8.
2. Курдюмов, С.П. Разработка приложений на C# и .NET. Практическое руководство / Сергей Петрович Курдюмов. — СПб.: БХВ-Петербург, 2024. — 592 с. — ISBN 978-5-9775-4763-5.
3. Фленов, М.Е. Windows Forms в .NET 6 и .NET 7. Разработка приложений с графическим интерфейсом / Михаил Евгеньевич Фленов. — СПб.: Питер, 2023. — 384 с. — ISBN 978-5-4461-2034-9.

4. Кузнецов, В.Д. Базы данных: проектирование, разработка и сопровождение / Владимир Дмитриевич Кузнецов. — М.: Альфа-Пресс, 2024. — 432 с. — ISBN 978-5-8297-0302-7.

5. Дейт, К.Дж. Введение в системы баз данных / Кристофер Дж. Дейт. — 8-е изд. — М.: Вильямс, 2023. — 1104 с. — ISBN 978-5-8459-1832-4.

6. Бендер, Р. Проектирование баз данных: руководство для разработчиков и архитекторов / Ральф Бендер. — М.: ДМК Пресс, 2024. — 352 с. — ISBN 978-5-97061-187-8.

7. Котеров, Д.А. ASP.NET Core и Entity Framework Core: современная разработка веб-приложений / Дмитрий Александрович Котеров. — СПб.: БХВ-Петербург, 2023. — 560 с. — ISBN 978-5-9775-4711-6.

8. Троелсен, Э. Язык программирования C# 11 и платформа .NET 7. Глубокое изучение / Эндрю Троелсен. — 12-е изд. — М.: Вильямс, 2024. — 1440 с. — ISBN 978-5-8459-2722-7.

9. Шилдт, Г. C# 11.0. Полное руководство / Герберт Шилдт. — М.: Вильямс, 2023. — 1248 с. — ISBN 978-5-8459-2711-1.

10. Макконнелл, С. Совершенный код. Мастер-класс / Стив Макконнелл. — 2-е изд. — М.: Вильямс, 2024. — 896 с. — ISBN 978-5-8459-2615-2.

Электронные ресурсы:

1. Microsoft Docs. Документация по C# и .NET. URL: <https://docs.microsoft.com/ru-ru/dotnet/csharp/> (дата обращения: 15.01.2026).

2. Microsoft Learn. Документация по SQL Server. URL: <https://learn.microsoft.com/ru-ru/sql/sql-server/> (дата обращения: 22.01.2026).

3. Microsoft Docs. Windows Forms для .NET. URL: <https://docs.microsoft.com/ru-ru/dotnet/desktop/winforms/> (дата обращения: 08.01.2026).

4. Stack Overflow. Вопросы и ответы по C#, SQL, WinForms. URL: <https://stackoverflow.com/> (дата обращения: 28.01.2026).

5. Habr. Статьи по разработке ПО, .NET, базам данных. URL: <https://habr.com/ru/> (дата обращения: 05.01.2026).

6. GeeksforGeeks. Учебные материалы по программированию. URL: <https://www.geeksforgeeks.org/> (дата обращения: 19.01.2026).

7. MSDN Magazine. Архив статей по разработке на .NET. URL: <https://msdn.microsoft.com/magazine/> (дата обращения: 12.01.2026).

8. GitHub. Репозитории с примерами кода и проектами. URL: <https://github.com/> (дата обращения: 30.01.2026).

ПРИЛОЖЕНИЕ

Код ClientDoOrderForm.cs

```
public partial class ClientDoOrderForm : Form
{

    private Dictionary<int, int> orderDict;
    private Dictionary<int, NumericUpDown> productQuantities;
    private int? clientId = null; // ID найденного клиента или null, если новый
    public static bool isClose = false;
    private bool isManager = false;

    public ClientDoOrderForm(Dictionary<int, int> order, bool isManager = false)
    {
        isClose = false;
        this.isManager = isManager;

        orderDict = order ?? throw new ArgumentNullException(nameof(order));

        InitializeComponent();

        btnCheck.Click += BtnCheck_Click;
        btnOrder.Click += BtnOrder_Click;

        dgvOrderItems.AllowUserToAddRows = false;
        dgvOrderItems.ReadOnly = true;

        // Subscribe to text change events to enable/disable order button
        txtFullName.TextChanged += OnInputChanged;
        txtContactPerson.TextChanged += OnInputChanged;
        txtPhone.TextChanged += OnInputChanged;

        LoadOrderItemsGridFromDict();
    }

    private void LoadOrderItemsGridFromDict()
    {
        dgvOrderItems.Columns.Clear();

        dgvOrderItems.Columns.Add("Name", "Название");
```

```

dgvOrderItems.Columns.Add("Price", "Цена");
dgvOrderItems.Columns.Add("Quantity", "Количество");
dgvOrderItems.Columns.Add("Quantityt", "Итого");

decimal total = 0;

foreach (var item in orderDict)
{
    int productId = item.Key;
    int quantity = item.Value;

    var product = scr.core.dbt.DBT_Products.GetById(productId);
    if (product == null) continue; // пропускаем, если продукт не найден

    var row = new DataGridViewRow();
    row.CreateCells(dgvOrderItems);
    row.Cells[0].Value = product.Name;
    row.Cells[1].Value = product.PricePerUnit.ToString("C");
    row.Cells[3].Value = (quantity * product.PricePerUnit).ToString("C");
    row.Cells[2].Value = quantity.ToString();

    total += (quantity * product.PricePerUnit);

    dgvOrderItems.Rows.Add(row);
}

label5.Text = "Итого: " + total.ToString("C");
}

private void BtnCheck_Click(object sender, EventArgs e)
{
    var email = txtEmail.Text.Trim();

    if (string.IsNullOrEmpty(email))
    {
        MessageBox.Show("Введите Email.", "Ошибка", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
        return;
    }

    // Проверяем, есть ли клиент с таким email

```

```

var client = scr.core.dbt.DBT_Clients.GetByEmail(email);

if (client != null)
{
    clientId = client.ID;
    MessageBox.Show("Клиент найден!", "Успех", MessageBoxButtons.OK,
    MessageBoxIcon.Information);
    btnOrder.Enabled = true;

    var _client = DBT_Clients.GetById((int)clientId);
    txtContactPerson.Text = _client.ContactPerson;
    txtFullName.Text = _client.FullName;
    txtPhone.Text = _client.Phone;
    txtEmail.Enabled = false;
    btnCheck.Enabled = false;
}
else
{
    clientId = null;
    txtFullName.Enabled = true;
    txtContactPerson.Enabled = true;
    txtPhone.Enabled = true;
    MessageBox.Show("Клиент не найден. Заполните данные.", "Информация",
    MessageBoxButtons.OK, MessageBoxIcon.Information);
}
}

private void OnInputChanged(object sender, EventArgs e)
{
    if (!btnOrder.Enabled && IsNewClientDataValid())
    {
        btnOrder.Enabled = true;
    }
}

private bool IsNewClientDataValid()
{
    return !string.IsNullOrEmpty(txtFullName.Text) &&
        !string.IsNullOrEmpty(txtContactPerson.Text) &&
        !string.IsNullOrEmpty(txtPhone.Text);
}

```

```

private void BtnOrder_Click(object sender, EventArgs e)
{
    if (clientId == null && !IsNewClientDataValid())
    {
        MessageBox.Show("Заполните все поля клиента.", "Ошибка",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    // Создаём клиента, если новый
    if (clientId == null)
    {
        var newClient = new scr.core.dbt.DBT_Clients
        {
            FullName = txtFullName.Text,
            ContactPerson = txtContactPerson.Text,
            Phone = txtPhone.Text,
            Email = txtEmail.Text
        };
        clientId = scr.core.dbt.DBT_Clients.Create(newClient);
        if (clientId == -1)
        {
            MessageBox.Show("Ошибка при создании клиента.", "Ошибка",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
    }

    int orderId = Core.SaveOrder((int)clientId, orderDict, isManager);

    MessageBox.Show($"Заказ успешно оформлен!\nНомер заказа: {orderId}",
    "Успех", MessageBoxButtons.OK, MessageBoxIcon.Information);
    isClose = true;
    Close();
}
}

```

Код Core.cs

```

public static class Core
{

```

```

public static int ThisUser_ID = -1;
public static int ThisUser_AC = -1;

public static string[] ACs = { "Нет доступа", "Админ", "Менеджер", "Повар" };

public static Dictionary<int, decimal>
Get_RawMaterialsIdAndQuantity_ByProductId(int productId)
{
    var objs = new Dictionary<int, decimal>();

    try
    {
        using (SqlConnection connection = new
SqlConnection(SQL._sqlConnectStr))
        {
            connection.Open();
            using (var query = connection.CreateCommand())
            {
                query.CommandText = "SELECT RawMaterialID, Quantity FROM
Recipes WHERE ProductID = @id";
                query.Parameters.AddWithValue("@id", productId);
                using (var reader = query.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        objs.Add(reader.GetInt32(0), reader.GetDecimal(1));
                    }
                }
            }
        }
    }
    catch { objs = null; }
    return objs;
}

public static int SaveOrder(int clientId, Dictionary<int, int> orderDict,
bool isManager = false)
{
    try
    {

```

```

        using (SqlConnection connection = new
SqlConnection(SQL._sqlConnectStr))
        {
            connection.Open();
            using (SqlTransaction transaction =
connection.BeginTransaction())
            {
                try
                {
                    int orderId = -1;
                    int? managerId = null;

                    if (isManager) managerId = Core.ThisUser_ID;
                    else managerId = GetLeastBusyManager(connection,
transaction);

                    if (managerId == null)
                    {
                        throw new Exception("Нет доступных менеджеров для
назначения.");
                    }

                    // 1. Создаём заказ в SalesOrders
                    using (var cmd = new SqlCommand(
@"INSERT INTO SalesOrders (ClientID, ManagerID,
OrderDate, TotalAmount, Status)
OUTPUT INSERTED.ID
VALUES (@ClientID, @ManagerID, @OrderDate,
@TotalAmount, @Status)", connection, transaction))
                    {
                        decimal total = 0;

                        // Предварительно вычисляем сумму
                        foreach (var item in orderDict)
                        {
                            int productId = item.Key;
                            int quantity = item.Value;

                            using (var priceCmd = new SqlCommand(
                                "SELECT PricePerUnit FROM Products WHERE ID =
@ProductID", connection, transaction))

```

```

        {
            priceCmd.Parameters.AddWithValue("@ProductID", productId);
            var result = priceCmd.ExecuteScalar();
            if (result != null)
            {
                decimal price =
Convert.ToDecimal(result);

                total += price * quantity;
            }
            else
            {
                throw new Exception($"Продукт с ID
{productId} не найден.");
            }
        }
    }

    cmd.Parameters.AddWithValue("@ClientID", clientId);
    cmd.Parameters.AddWithValue("@ManagerID", managerId);
    cmd.Parameters.AddWithValue("@OrderDate",
DateTime.Now);

    cmd.Parameters.AddWithValue("@TotalAmount", total);
    cmd.Parameters.AddWithValue("@Status", "Новый");

    orderId = (int)cmd.ExecuteScalar();
}

if (orderId == -1) throw new Exception("Не удалось
создать заказ.");

// 2. Создаём позиции заказа в OrderItems
foreach (var item in orderDict)
{
    int productId = item.Key;
    int quantity = item.Value;

    using (var cmdItem = new SqlCommand(
@"INSERT INTO OrderItems (OrderID, ProductID,
Quantity, Subtotal)

```



```

VALUES (@OrderID, @ProductID, @Quantity,
@Subtotal)", connection, transaction))
    {
        using (var priceCmd = new SqlCommand(
            "SELECT PricePerUnit FROM Products WHERE ID =
@ProductID", connection, transaction))
        {
            priceCmd.Parameters.AddWithValue("@ProductID", productId);
            decimal price =
Convert.ToDecimal(priceCmd.ExecuteScalar());
            decimal subtotal = price * quantity;

            cmdItem.Parameters.AddWithValue("@OrderID",
orderId);
            cmdItem.Parameters.AddWithValue("@ProductID",
productId);
            cmdItem.Parameters.AddWithValue("@Quantity",
quantity);
            cmdItem.Parameters.AddWithValue("@Subtotal",
subtotal);

            cmdItem.ExecuteNonQuery();
        }
    }

    transaction.Commit();
    return orderId;
}
catch
{
    transaction.Rollback();
    throw;
}
}
}
catch
{
    return -1;
}

```

```

    }
}

private static int? GetLeastBusyManager(SqlConnection conn, SqlTransaction
trans)
{
    using (var cmd = new SqlCommand(@"
        SELECT e.ID
        FROM Employees e
        LEFT JOIN (
            SELECT ManagerID, COUNT(*) AS ActiveOrders
            FROM SalesOrders
            WHERE Status IN ('Новый', 'В процессе')
            GROUP BY ManagerID
        ) o ON e.ID = o.ManagerID
        WHERE e.Position = 'Менеджер'
        ORDER BY ISNULL(o.ActiveOrders, 0) ASC
    ", conn, trans))
    {
        using (var reader = cmd.ExecuteReader())
        {
            if (reader.Read())
            {
                return reader.GetInt32(0);
            }
        }
    }
    return null; // Нет доступных менеджеров
}

public static class Auth
{
    public static AuthResult? ValidateCredentials(string login, string
password)
    {
        try
        {
            using (SqlConnection connection = new
SqlConnection(scr.core.SQL._sqlConnectStr))
            {
                connection.Open();
            }
        }
    }
}

```

```

        using (var cmd = new SqlCommand(@"
SELECT a.EmployeeID, a.AccessLevel, e.HireDate
FROM Auth a
INNER JOIN Employees e ON a.EmployeeID = e.ID
WHERE a.Login = @Login AND a.PasswordHash = @Password", connection))
        {
            cmd.Parameters.AddWithValue("@Login", login);
            cmd.Parameters.AddWithValue("@Password", password);

            using (var reader = cmd.ExecuteReader())
            {
                if (reader.Read())
                {
                    int employeeId = reader.GetInt32("EmployeeID");
                    int accessLevel = reader.GetInt32("AccessLevel");
                    DateTime hireDate =
reader.GetDateTime("HireDate");

                    return new AuthResult(employeeId, accessLevel,
hireDate);
                }
            }
        }
    }
}

catch
{
    // В реальной системе логируйте ошибку
}

return null;
}

// Вспомогательная структура
public struct AuthResult
{
    public int EmployeeID;
    public int AccessLevel;
    public DateTime HireDate;

    public AuthResult(int employeeId, int accessLevel, DateTime hireDate)
    {

```

```

        EmployeeID = employeeId;
        AccessLevel = accessLevel;
        HireDate = hireDate;
    }
}
}
}

```

Код AuthForm.cs

```

public partial class AuthForm : Form
{
    public AuthForm()
    {
        InitializeComponent();
        panel_client.Visible = false;
        panel_empl.Visible = false;

        button_client.Click += (s, e) => panel_client.Visible = true;
        button_empl.Click += (s, e) => panel_empl.Visible = true;

        button_c_catalog.Click += (s, e) => { this.Hide(); new
ClientCatalogProductForm().ShowDialog(); this.Show(); };
        button_c_login.Click += (s, e) => {
            if (string.IsNullOrEmpty(textBox_c_order.Text) ||
string.IsNullOrEmpty(textBox_c_email.Text))
            {
                MessageBox.Show("Введите данные для проверки заказа!");
                return;
            }
            if (!int.TryParse(textBox_c_order.Text, out int _Id))
            {
                MessageBox.Show("Введите данные для проверки заказа!");
                return;
            }
            this.Hide();
            try
            {
                new ClientOrderViewForm(textBox_c_email.Text, _Id).ShowDialog();
            }
            catch { }
            this.Show();
        };
    }
}

```

```

button_e_login.Click += (s, e) =>
{
    string login = textBox_e_login.Text.Trim();
    string password = textBox_e_password.Text.Trim();

    if (string.IsNullOrEmpty(login) || string.IsNullOrEmpty(password))
    {
        MessageBox.Show("Введите логин и пароль.", "Ошибка",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    var authResult = Core.Auth.ValidateCredentials(login, password);

    if (authResult == null)
    {
        MessageBox.Show("Неверный логин или пароль.", "Ошибка",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
        return;
    }

    if (((Core.Auth.AuthResult)authResult).AccessLevel == -1)
    {
        MessageBox.Show("У вас нет доступа к системе.", "Доступ
запрещён", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    if (((Core.Auth.AuthResult)authResult).HireDate.Year == 1900)
    {
        MessageBox.Show("Сотрудник уволен. Доступ запрещён.", "Доступ
запрещён", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    Core.ThisUser_AC = ((Core.Auth.AuthResult)authResult).AccessLevel;
    Core.ThisUser_ID = ((Core.Auth.AuthResult)authResult).EmployeeID;

    this.Hide();
}

```

```

        // Открываем форму в зависимости от AccessLevel
        switch (((Core.Auth.AuthResult)authResult).AccessLevel)
        {
            case 0: // Админ
                new AdminForm().ShowDialog();
                break;
            case 1: // Менеджер
                new TopPageForm(new
scr.forms.userControls.SalesOrders_ViewUserControl(), "ПП - Менеджер").ShowDialog();
                break;
            case 2: // Повар
                new TopPageForm(new
scr.forms.userControls.SalesOrders_ViewUserControl(), "ПП - Повар").ShowDialog();
                break;
            default:
                MessageBox.Show("Неизвестный уровень доступа.", "Ошибка",
MessageBoxButtons.OK, MessageBoxIcon.Error);
                return;
        }

        Core.ThisUser_ID = -1;
        Core.ThisUser_AC = -1;

        this.Show();

    };
}

private void button_close_Click(object sender, EventArgs e)
{
    panel_client.Visible = false;
    panel_empl.Visible = false;
}
}

```

Код AnalyticsForm.cs

```

public partial class AnalyticsForm : Form
{
    private Button btnGenerateReport;
    private Button btnSaveHtml;
    private RichTextBox rtbReport;

```

```

private DateTimePicker dtpStart;
private DateTimePicker dtpEnd;

public AnalyticsForm()
{
    InitializeComponent();
    InitializeComponent_();
}

private void InitializeComponent_()
{
    this.Size = new Size(1000, 700);
    this.Text = "Аналитика";
    this.StartPosition = FormStartPosition.CenterScreen;

    // Controls
    var lblPeriod = new Label { Text = "Период:", Location = new Point(20,
20) };

    dtpStart = new DateTimePicker { Format = DateTimePickerFormat.Short,
Location = new Point(120, 20) };
    dtpEnd = new DateTimePicker { Format = DateTimePickerFormat.Short,
Location = new Point(340, 20) };
    dtpEnd.Value = DateTime.Today;
    dtpStart.Value = DateTime.Today.AddMonths(-1);

    btnGenerateReport = new Button { Text = "Сформировать отчет", Location =
new Point(550, 18), Size = new Size(120, 30) };
    btnGenerateReport.Click += BtnGenerateReport_Click;

    btnSaveHtml = new Button { Text = "Сохранить в HTML", Location = new
Point(700, 20), Size = new Size(120, 30) };
    btnSaveHtml.Click += BtnSaveHtml_Click;
    btnSaveHtml.Visible = false;

    rtbReport = new RichTextBox
    {
        Location = new Point(20, 60),
        Size = new Size(950, 600),
        ReadOnly = true,
        ScrollBars = RichTextBoxScrollBars.Vertical,
        Font = new Font("Consolas", 10)
    }

```

```

    };

    this.Controls.AddRange(new Control[] { lblPeriod, dtpStart, dtpEnd,
    btnGenerateReport, btnSaveHtml, rtbReport });
}

private void BtnGenerateReport_Click(object sender, EventArgs e)
{
    var start = dtpStart.Value.Date;
    var end = dtpEnd.Value.Date.AddDays(1).AddSeconds(-1); // Включительно

    var report = GenerateFullReport(start, end);

    rtbReport.Text = report;
    btnSaveHtml.Enabled = true;
}

private string GenerateFullReport(DateTime start, DateTime end)
{
    var sb = new System.Text.StringBuilder();

    sb.AppendLine($"=== Отчет по анализу ===");
    sb.AppendLine($"Период: {start:dd.MM.yyyy} – {end:dd.MM.yyyy}");
    sb.AppendLine();

    sb.AppendLine("--- Лучшие менеджеры ---");
    var bestManagers = GetBestManagers(start, end);
    foreach (var item in bestManagers)
        sb.AppendLine($"{item.FullName}: {item.Count} заказов");
    if (!bestManagers.Any()) sb.AppendLine("Нет данных.");
    sb.AppendLine();

    sb.AppendLine("--- Лучшие повара ---");
    var bestCooks = GetBestCooks(start, end);
    foreach (var item in bestCooks)
        sb.AppendLine($"{item.FullName}: {item.Count} заказов");
    if (!bestCooks.Any()) sb.AppendLine("Нет данных.");
    sb.AppendLine();

    sb.AppendLine("--- Лучшие админы ---");
    var bestAdmins = GetBestAdmins(start, end);

```



```

        foreach (var item in bestAdmins)
            sb.AppendLine($"{item.FullName}: {item.Count} заказов");
        if (!bestAdmins.Any()) sb.AppendLine("Нет данных.");
        sb.AppendLine();

        sb.AppendLine("--- Прибыль ---");
        var profit = GetProfit(start, end);
        sb.AppendLine($"Общая прибыль: {profit:C}");
        sb.AppendLine();

        sb.AppendLine("--- Самые популярные товары ---");
        var popularProducts = GetPopularProducts(start, end);
        foreach (var item in popularProducts)
            sb.AppendLine($"{item.Name}: {item.Quantity} шт.");
        if (!popularProducts.Any()) sb.AppendLine("Нет данных.");
        sb.AppendLine();

        sb.AppendLine("--- Расход сырья ---");
        var rawUsage = GetRawMaterialUsage(start, end);
        foreach (var item in rawUsage)
            sb.AppendLine($"{item.Name}: {item.Quantity:F3} ед.");
        if (!rawUsage.Any()) sb.AppendLine("Нет данных.");
        sb.AppendLine();

        sb.AppendLine("--- Анализ заказов ---");
        var orderStats = GetOrderStats(start, end);
        sb.AppendLine($"Всего заказов: {orderStats.Total}");
        sb.AppendLine($"Завершено: {orderStats.Completed}");
        sb.AppendLine($"Среднее время выполнения: {orderStats.AvgTime:g}");
        if (orderStats.Total == 0) sb.AppendLine("Нет данных.");
        sb.AppendLine();

        return sb.ToString();
    }

    private List<RankedEmployee> GetBestManagers(DateTime start, DateTime end)
    {
        var list = new List<RankedEmployee>();
        using (SqlConnection conn = new
SqlConnection(scr.core.SQL._sqlConnectStr))
        {

```

```

        conn.Open();
        using (var cmd = new SqlCommand(@"
            SELECT e.FullName, COUNT(*) AS Count
            FROM SalesOrders s
            INNER JOIN Employees e ON s.ManagerID = e.ID
            WHERE s.OrderDate BETWEEN @Start AND @End
            GROUP BY e.FullName
            ORDER BY Count DESC", conn))
        {
            cmd.Parameters.AddWithValue("@Start", start);
            cmd.Parameters.AddWithValue("@End", end);
            using (var reader = cmd.ExecuteReader())
            {
                while (reader.Read())
                {
                    list.Add(new RankedEmployee
                    {
                        FullName = reader.GetString("FullName"),
                        Count = reader.GetInt32("Count")
                    });
                }
            }
        }
        return list.Take(5).ToList();
    }

    private List<RankedEmployee> GetBestCooks(DateTime start, DateTime end)
    {
        var list = new List<RankedEmployee>();
        using (SqlConnection conn = new
SqlConnection(scr.core.SQL._sqlConnectStr))
        {
            conn.Open();
            using (var cmd = new SqlCommand(@"
                SELECT e.FullName, COUNT(*) AS Count
                FROM SalesOrders s
                INNER JOIN Employees e ON s.CookID = e.ID
                WHERE s.OrderDate BETWEEN @Start AND @End
                GROUP BY e.FullName
                ORDER BY Count DESC", conn))

```

```

    {
        cmd.Parameters.AddWithValue("@Start", start);
        cmd.Parameters.AddWithValue("@End", end);
        using (var reader = cmd.ExecuteReader())
        {
            while (reader.Read())
            {
                list.Add(new RankedEmployee
                {
                    FullName = reader.GetString("FullName"),
                    Count = reader.GetInt32("Count")
                });
            }
        }
    }
    return list.Take(5).ToList();
}

private List<RankedEmployee> GetBestAdmins(DateTime start, DateTime end)
{
    var list = new List<RankedEmployee>();
    using (SqlConnection conn = new
SqlConnection(scr.core.SQL._sqlConnectStr))
    {
        conn.Open();
        using (var cmd = new SqlCommand(@"
            SELECT e.FullName, COUNT(*) AS Count
            FROM SalesOrders s
            INNER JOIN Employees e ON s.ManagerID = e.ID
            INNER JOIN Auth a ON e.ID = a.EmployeeID
            WHERE a.AccessLevel = 0 AND s.OrderDate BETWEEN @Start AND @End
            GROUP BY e.FullName
            ORDER BY Count DESC", conn))
        {
            cmd.Parameters.AddWithValue("@Start", start);
            cmd.Parameters.AddWithValue("@End", end);
            using (var reader = cmd.ExecuteReader())
            {
                while (reader.Read())
                {

```

```

        list.Add(new RankedEmployee
        {
            FullName = reader.GetString("FullName"),
            Count = reader.GetInt32("Count")
        });
    }
}

return list.Take(5).ToList();
}

private decimal GetProfit(DateTime start, DateTime end)
{
    using (SqlConnection conn = new
SqlConnection(scr.core.SQL._sqlConnectStr))
    {
        conn.Open();
        using (var cmd = new SqlCommand(@"
            SELECT SUM(TotalAmount) AS Profit
            FROM SalesOrders
            WHERE OrderDate BETWEEN @Start AND @End
            AND Status IN ('Завершено')", conn))
        {
            cmd.Parameters.AddWithValue("@Start", start);
            cmd.Parameters.AddWithValue("@End", end);
            var result = cmd.ExecuteScalar();
            return result != DBNull.Value ? Convert.ToDecimal(result) : 0;
        }
    }
}

private List<RankedProduct> GetPopularProducts(DateTime start, DateTime end)
{
    var list = new List<RankedProduct>();
    using (SqlConnection conn = new
SqlConnection(scr.core.SQL._sqlConnectStr))
    {
        conn.Open();
        using (var cmd = new SqlCommand(@"
            SELECT p.Name, SUM(oi.Quantity) AS Quantity

```

```

        FROM OrderItems oi
        INNER JOIN Products p ON oi.ProductID = p.ID
        INNER JOIN SalesOrders s ON oi.OrderID = s.ID
        WHERE s.OrderDate BETWEEN @Start AND @End
        GROUP BY p.Name
        ORDER BY Quantity DESC", conn))
    {
        cmd.Parameters.AddWithValue("@Start", start);
        cmd.Parameters.AddWithValue("@End", end);
        using (var reader = cmd.ExecuteReader())
        {
            while (reader.Read())
            {
                list.Add(new RankedProduct
                {
                    Name = reader.GetString("Name"),
                    Quantity = reader.GetDecimal("Quantity")
                });
            }
        }
    }
    return list.Take(10).ToList();
}

private List<RankedRawMaterial> GetRawMaterialUsage(DateTime start, DateTime
end)
{
    var list = new List<RankedRawMaterial>();
    using (SqlConnection conn = new
SqlConnection(scr.core.SQL._sqlConnectStr))
    {
        conn.Open();
        using (var cmd = new SqlCommand(@"
            SELECT rm.Name, SUM(r.Quantity * oi.Quantity) AS Quantity
            FROM OrderItems oi
            INNER JOIN Products p ON oi.ProductID = p.ID
            INNER JOIN Recipes r ON p.ID = r.ProductID
            INNER JOIN RawMaterials rm ON r.RawMaterialID = rm.ID
            INNER JOIN SalesOrders s ON oi.OrderID = s.ID
            WHERE s.OrderDate BETWEEN @Start AND @End

```

```

        GROUP BY rm.Name
        ORDER BY Quantity DESC", conn))
    {
        cmd.Parameters.AddWithValue("@Start", start);
        cmd.Parameters.AddWithValue("@End", end);
        using (var reader = cmd.ExecuteReader())
        {
            while (reader.Read())
            {
                list.Add(new RankedRawMaterial
                {
                    Name = reader.GetString("Name"),
                    Quantity = reader.GetDecimal("Quantity")
                });
            }
        }
    }
    return list.Take(10).ToList();
}

private OrderStats GetOrderStats(DateTime start, DateTime end)
{
    var stats = new OrderStats();
    using (SqlConnection conn = new
SqlConnection(scr.core.SQL._sqlConnectStr))
    {
        conn.Open();
        using (var cmd = new SqlCommand(@"
            SELECT COUNT(*) AS Total,
                   SUM(CASE WHEN Status = 'Завершено' THEN 1 ELSE 0 END) AS
Completed,
                   AVG(CAST(DATEDIFF(SECOND, OrderDate, CompletionDate) AS
FLOAT)) AS AvgSecs
            FROM SalesOrders
            WHERE OrderDate BETWEEN @Start AND @End
               AND CompletionDate IS NOT NULL", conn))
        {
            cmd.Parameters.AddWithValue("@Start", start);
            cmd.Parameters.AddWithValue("@End", end);
            using (var reader = cmd.ExecuteReader())

```

```

        {
            if (reader.Read())
            {
                stats.Total = reader.GetInt32("Total");
                stats.Completed = reader.IsDBNull("Completed") ? 0 :
reader.GetInt32("Completed");
                var avgSecs = reader.IsDBNull("AvgSecs") ? 0 :
reader.GetDouble("AvgSecs");
                stats.AvgTime = TimeSpan.FromSeconds(avgSecs);
            }
        }
    }
    return stats;
}

private void BtnSaveHtml_Click(object sender, EventArgs e)
{
    var saveFileDialog = new SaveFileDialog
    {
        Filter = "HTML Files (*.html)|*.html",
        FileName = $"Аналитика_{dtpStart.Value:yyyy-MM-
dd}_to_{dtpEnd.Value:yyyy-MM-dd}.html"
    };

    if (saveFileDialog.ShowDialog() == DialogResult.OK)
    {
        var html = GenerateHtmlReport();
        File.WriteAllText(saveFileDialog.FileName, html,
System.Text.Encoding.UTF8);
        MessageBox.Show("Отчет сохранен в HTML.", "Успех",
MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
}

private string GenerateHtmlReport()
{
    var start = dtpStart.Value.Date;
    var end = dtpEnd.Value.Date.AddDays(1).AddSeconds(-1);

    var report = GenerateFullReport(start, end);
}

```

```

        var html = "$@"
<!DOCTYPE html>
<html>
<head>
    <title>Отчет по аналитике</title>
    <meta charset='UTF-8'>
    <style>
        body {{ font-family: Arial, sans-serif; }}
        h1, h2 {{ color: #2c3e50; }}
        pre {{ background-color: #f4f4f4; padding: 10px; border-radius: 5px; }}
    </style>
</head>
<body>
    <h1>Отчет по аналитике</h1>
    <p><strong>Период:</strong> {start:dd.MM.yyyy} – {end:dd.MM.yyyy}</p>
    <pre>{report}</pre>
</body>
</html>";

        return html;
    }
}

```